

Project APEX Annotated Source Code Companion

Every production module, line-numbered source, block explanations and line-by-line Markdown notes

Use this companion with the editor open. First read a function signature and predict its contract. Then follow the numbered code. The annotation states what the line does and what invariant to verify; you should add the concrete tensor shape, unit or artifact path from your own run.

Call Path

1. CLI / UI / API
2. Configuration and source adapter
3. Canonical contract and validation
4. Features, splits, normalization and windows
5. Model factory and world-model architecture
6. Training and checkpointing
7. Evaluation, scenario rollout and plotting
8. Pipeline stages, registry and published artifacts

How to Read a Module

- Role: why the file exists.
- Imports: architectural dependencies.
- Public definitions: boundaries used by other modules.
- State: values that persist between calls or time steps.
- Transformations: equations and tensor shape changes.
- Failure paths: invariants, exceptions and fallbacks.
- Artifacts: what is written, returned or registered.
- Tests: evidence required before changing the implementation.

src/apexsim/__init__.py

Supports the APEX package behind an explicit module boundary.

Trace callers, inputs, outputs and side effects before editing.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-3

```
0001 """Project APEX: an educational F1 telemetry world-model simulation engine."""
0002
0003 __version__ = "0.1.0"
```

Line	Code signal	What it is doing / what to verify
1	"""Project APEX: an educational F1 telemetry world-model simulation engine."""	Begins or ends documentation describing the module, class or function contract.
3	__version__ = "0.1.0"	Creates or updates `__version__`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

src/apexsim/cli.py

Routes human commands to tested domain functions.

A CLI should be thin. It parses intent, validates inputs and calls the same adapters and stages used by tests and orchestration.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```
0001 from __future__ import annotations
0002
0003 import json
0004 from pathlib import Path
0005
0006 import typer
0007
0008 from apexsim.config import load_config
0009 from apexsim.data.fastf1_adapter import ingest_fastf1_session
0010 from apexsim.data.openf1_adapter import ingest_openf1_session
0011 from apexsim.data.synthetic import generate_synthetic_sessions
0012 from apexsim.data.validate import validate_canonical_frame
0013 from apexsim.pipeline.runner import run_pipeline
0014
0015 app = typer.Typer(no_args_is_help=True, help="Project APEX F1 world-simulation engine")
0016
0017
0018 @app.command()
0019 def generate(config: Path = Path("configs/fast.yaml"), output: Path = Path("data/raw/synthetic.csv")) -> None:
0020     cfg = load_config(config)
0021     frame = generate_synthetic_sessions(cfg, output)
0022     typer.echo(f"Generated {len(frame):,} canonical telemetry rows at {output}")
```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	import json	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
4	from pathlib import Path	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
6	import typer	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
8	from apexsim.config import load_config	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
9	from apexsim.data.fastf1_adapter import ingest_fastf1_session	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
10	from apexsim.data.openf1_adapter import ingest_openf1_session	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
11	from apexsim.data.synthetic import generate_synthetic_sessions	Imports a dependency required by the definitions below; keeping imports explicit

Source → contract → runtime evidence → failure test

		reveals architectural coupling.
12	from apexsim.data.validate import validate_canonical_frame	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
13	from apexsim.pipeline.runner import run_pipeline	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
15	app = typer.Typer(no_args_is_help=True, help="Project APEX F1 world-simulation e	Creates or updates `app`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
18	@app.command()	Applies a decorator that changes registration, validation or runtime behaviour of the definition that follows.
19	def generate(config: Path = Path("configs/fast.yaml"), output: Path = Path("data	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
20	cfg = load_config(config)	Creates or updates `cfg`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
21	frame = generate_synthetic_sessions(cfg, output)	Creates or updates `frame`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
22	typer.echo(f"Generated {len(frame):,} canonical telemetry rows at {output}")	Calls a function or method. Trace its input contract, returned value and any persistent side effects.

Lines 23-44

```

0023
0024
0025 @app.command("ingest-fastf1")
0026 def ingest_fastf1(
0027     year: int,
0028     event: str,
0029     session: str,
0030     driver: str,
0031     output: Path = Path("data/raw/fastf1.csv"),
0032     sample_hz: int = 5,
0033 ) -> None:
0034     frame = ingest_fastf1_session(year, event, session, driver, output, sample_hz)
0035     typer.echo(f"Ingested {len(frame):,} FastF1 rows at {output}")
0036
0037
0038 @app.command("ingest-openf1")
0039 def ingest_openf1(
0040     session_key: int,
0041     driver_number: int,
0042     output: Path = Path("data/raw/openf1.csv"),
0043     sample_hz: int = 4,
0044 ) -> None:

```

Line	Code signal	What it is doing / what to verify
25	@app.command("ingest-fastf1")	Applies a decorator that changes registration, validation or runtime behaviour of the definition that follows.
26	def ingest_fastf1(	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
27	year: int,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
28	event: str,	Executes this statement inside the current

		control-flow block; connect it to the nearest function contract and testable invariant.
29	session: str,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
30	driver: str,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
31	output: Path = Path("data/raw/fastf1.csv"),	Creates or updates `output: Path`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
32	sample_hz: int = 5,	Creates or updates `sample_hz: int`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
33	) -> None:	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
34	frame = ingest_fastf1_session(year, event, session, driver, output, sample_hz)	Creates or updates `frame`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
35	typer.echo(f"Ingested {len(frame):,} FastF1 rows at {output}")	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
38	@app.command("ingest-openf1")	Applies a decorator that changes registration, validation or runtime behaviour of the definition that follows.
39	def ingest_openf1(	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
40	session_key: int,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
41	driver_number: int,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
42	output: Path = Path("data/raw/openf1.csv"),	Creates or updates `output: Path`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
43	sample_hz: int = 4,	Creates or updates `sample_hz: int`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
44	) -> None:	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

Lines 45-66

```

0045     frame = ingest_openf1_session(session_key, driver_number, output, sample_hz)
0046     typer.echo(f"Ingested {len(frame):,} OpenF1 rows at {output}")
0047
0048
0049     @app.command()
0050     def validate(path: Path) -> None:
0051         import pandas as pd
0052
0053         report = validate_canonical_frame(pd.read_csv(path), strict=False)
0054         typer.echo(json.dumps(report.to_dict(), indent=2))
0055         if not report.passed:

```

Source → contract → runtime evidence → failure test

```

0056         raise typer.Exit(code=1)
0057
0058
0059 @app.command()
0060 def run(
0061     config: Path = Path("configs/fast.yaml"),
0062     run_id: str = "reference_gru",
0063 ) -> None:
0064     summary = run_pipeline(load_config(config), run_id)
0065     typer.echo(json.dumps(summary, indent=2))
0066

```

Line	Code signal	What it is doing / what to verify
45	frame = ingest_openf1_session(session_key, driver_number, output, sample_hz)	Creates or updates `frame`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
46	typer.echo(f"Ingested {len(frame);} OpenF1 rows at {output}")	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
49	@app.command()	Applies a decorator that changes registration, validation or runtime behaviour of the definition that follows.
50	def validate(path: Path) -> None:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
51	import pandas as pd	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
53	report = validate_canonical_frame(pd.read_csv(path), strict=False)	Loads persisted tabular evidence; validate schema, units and ordering immediately afterward.
54	typer.echo(json.dumps(report.to_dict(), indent=2))	Creates or updates `typer.echo(json.dumps(report.to_dict(), indent)`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
55	if not report.passed:	Branches only when this condition is true; verify both the true and false paths in tests.
56	raise typer.Exit(code=1)	Stops execution because an invariant or supported-condition check failed.
59	@app.command()	Applies a decorator that changes registration, validation or runtime behaviour of the definition that follows.
60	def run(	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
61	config: Path = Path("configs/fast.yaml"),	Creates or updates `config: Path`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
62	run_id: str = "reference_gru",	Creates or updates `run_id: str`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
63	) -> None:	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
64	summary = run_pipeline(load_config(config), run_id)	Creates or updates `summary`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
65	typer.echo(json.dumps(summary, indent=2))	Creates or updates

		<code>`typer.echo(json.dumps(summary, indent`.</code> Check its unit, shape, lifetime and whether it is derived only from information available at this point.
--	--	---

Lines 67-88

```

0067
0068 @app.command("run-canonical")
0069 def run_canonical(
0070     input_path: Path,
0071     config: Path = Path("configs/fast.yaml"),
0072     run_id: str = "historical_world_model",
0073 ) -> None:
0074     """Run the full pipeline from a validated canonical FastF1/OpenF1 CSV."""
0075     cfg = load_config(config)
0076     if not input_path.exists():
0077         raise typer.BadParameter(f"Canonical input does not exist: {input_path}")
0078     cfg.data.canonical_input_path = input_path
0079     summary = run_pipeline(cfg, run_id)
0080     typer.echo(json.dumps(summary, indent=2))
0081
0082
0083 @app.command()
0084 def ui(
0085     run_dir: Path = Path("artifacts/runs/reference_gru"),
0086     config: Path = Path("configs/fast.yaml"),
0087     share: bool = False,
0088 ) -> None:

```

Line	Code signal	What it is doing / what to verify
68	<code>@app.command("run-canonical")</code>	Applies a decorator that changes registration, validation or runtime behaviour of the definition that follows.
69	<code>def run_canonical(</code>	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
70	<code>input_path: Path,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
71	<code>config: Path = Path("configs/fast.yaml"),</code>	Creates or updates <code>`config: Path`</code> . Check its unit, shape, lifetime and whether it is derived only from information available at this point.
72	<code>run_id: str = "historical_world_model",</code>	Creates or updates <code>`run_id: str`</code> . Check its unit, shape, lifetime and whether it is derived only from information available at this point.
73	<code>) -> None:</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
74	<code>"""Run the full pipeline from a validated canonical FastF1/OpenF1 CSV."""</code>	Begins or ends documentation describing the module, class or function contract.
75	<code>cfg = load_config(config)</code>	Creates or updates <code>`cfg`</code> . Check its unit, shape, lifetime and whether it is derived only from information available at this point.
76	<code>if not input_path.exists():</code>	Branches only when this condition is true; verify both the true and false paths in tests.
77	<code>raise typer.BadParameter(f"Canonical input does not exist: {input_path}")</code>	Stops execution because an invariant or supported-condition check failed.
78	<code>cfg.data.canonical_input_path = input_path</code>	Creates or updates <code>`cfg.data.canonical_input_path`</code> . Check its unit, shape, lifetime and whether it is derived only from information available at

		this point.
79	summary = run_pipeline(cfg, run_id)	Creates or updates `summary`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
80	typer.echo(json.dumps(summary, indent=2))	Creates or updates `typer.echo(json.dumps(summary, indent=2))`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
83	@app.command()	Applies a decorator that changes registration, validation or runtime behaviour of the definition that follows.
84	def ui(	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
85	run_dir: Path = Path("artifacts/runs/reference_gru"),	Creates or updates `run_dir: Path`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
86	config: Path = Path("configs/fast.yaml"),	Creates or updates `config: Path`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
87	share: bool = False,	Creates or updates `share: bool`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
88	) -> None:	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

Lines 89-107

```

0089     from apexsim.ui import launch
0090
0091     launch(run_dir, config, share=share)
0092
0093
0094 @app.command()
0095 def api(
0096     artifacts_dir: Path = Path("artifacts/runs"),
0097     host: str = "127.0.0.1",
0098     port: int = 8000,
0099 ) -> None:
0100     import uvicorn
0101     from apexsim.serving import create_api
0102
0103     uvicorn.run(create_api(artifacts_dir), host=host, port=port)
0104
0105
0106 if __name__ == "__main__":
0107     app()

```

Line	Code signal	What it is doing / what to verify
89	from apexsim.ui import launch	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
91	launch(run_dir, config, share=share)	Creates or updates `launch(run_dir, config, share)`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
94	@app.command()	Applies a decorator that changes registration, validation or runtime behaviour of the definition that follows.

95	def api(	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
96	artifacts_dir: Path = Path("artifacts/runs"),	Creates or updates `artifacts_dir: Path`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
97	host: str = "127.0.0.1",	Creates or updates `host: str`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
98	port: int = 8000,	Creates or updates `port: int`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
99	) -> None:	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
100	import uvicorn	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
101	from apexsim.serving import create_api	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
103	uvicorn.run(create_api(artifacts_dir), host=host, port=port)	Creates or updates `uvicorn.run(create_api(artifacts_dir), host)`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
106	if __name__ == "__main__":	Branches only when this condition is true; verify both the true and false paths in tests.
107	app()	Calls a function or method. Trace its input contract, returned value and any persistent side effects.

src/apexsim/config.py

Loads and validates experiment choices before expensive work begins.

Typed configuration turns hidden constants into reproducible evidence and stops incompatible history, horizon or feature choices at the boundary.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```
0001 from __future__ import annotations
0002
0003 from pathlib import Path
0004 from typing import Literal
0005
0006 import yaml
0007 from pydantic import BaseModel, Field, model_validator
0008
0009
0010 class DataConfig(BaseModel):
0011     source: Literal["synthetic", "fastf1", "openf1"] = "synthetic"
0012     raw_dir: Path = Path("data/raw")
0013     processed_dir: Path = Path("data/processed")
0014     canonical_input_path: Path | None = None
0015     sample_hz: int = Field(5, ge=1, le=50)
0016     sessions: int = Field(8, ge=3)
0017     laps_per_session: int = Field(3, ge=1)
0018     track_length_m: float = Field(5200.0, gt=500)
0019     session_seconds: int = Field(210, ge=30)
0020     train_fraction: float = 0.625
0021     val_fraction: float = 0.125
0022     test_fraction: float = 0.25
```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	from pathlib import Path	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
4	from typing import Literal	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
6	import yaml	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
7	from pydantic import BaseModel, Field, model_validator	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
10	class DataConfig(BaseModel):	Defines a type that owns related state and behaviour behind a named interface.
11	source: Literal["synthetic", "fastf1", "openf1"] = "synthetic"	Creates or updates `source: Literal["synthetic", "fastf1", "openf1"]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
12	raw_dir: Path = Path("data/raw")	Creates or updates `raw_dir: Path`. Check its

Source → contract → runtime evidence → failure test

		unit, shape, lifetime and whether it is derived only from information available at this point.
13	processed_dir: Path = Path("data/processed")	Creates or updates `processed_dir: Path`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
14	canonical_input_path: Path None = None	Creates or updates `canonical_input_path: Path   None`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
15	sample_hz: int = Field(5, ge=1, le=50)	Creates or updates `sample_hz: int`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
16	sessions: int = Field(8, ge=3)	Creates or updates `sessions: int`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
17	laps_per_session: int = Field(3, ge=1)	Creates or updates `laps_per_session: int`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
18	track_length_m: float = Field(5200.0, gt=500)	Creates or updates `track_length_m: float`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
19	session_seconds: int = Field(210, ge=30)	Creates or updates `session_seconds: int`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
20	train_fraction: float = 0.625	Creates or updates `train_fraction: float`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
21	val_fraction: float = 0.125	Creates or updates `val_fraction: float`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
22	test_fraction: float = 0.25	Creates or updates `test_fraction: float`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 23-44

```

0023     sequence_length: int = Field(32, ge=4)
0024     prediction_horizon: int = Field(8, ge=1)
0025
0026     @model_validator(mode="after")
0027     def fractions_sum_to_one(self) -> "DataConfig":
0028         total = self.train_fraction + self.val_fraction + self.test_fraction
0029         if abs(total - 1.0) > 1e-6:
0030             raise ValueError("train/val/test fractions must sum to 1.0")
0031         return self
0032
0033
0034     class ModelConfig(BaseModel):
0035         kind: Literal["gru", "rsm", "ssm", "linear"] = "gru"
0036         hidden_dim: int = Field(64, ge=8)
0037         latent_dim: int = Field(16, ge=2)
0038         layers: int = Field(1, ge=1, le=8)
0039         dropout: float = Field(0.0, ge=0.0, lt=1.0)
0040
0041

```

```

0042 class TrainingConfig(BaseModel):
0043     epochs: int = Field(4, ge=1)
0044     batch_size: int = Field(64, ge=1)

```

Line	Code signal	What it is doing / what to verify
23	sequence_length: int = Field(32, ge=4)	Creates or updates `sequence_length: int`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
24	prediction_horizon: int = Field(8, ge=1)	Creates or updates `prediction_horizon: int`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
26	@model_validator(mode="after")	Applies a decorator that changes registration, validation or runtime behaviour of the definition that follows.
27	def fractions_sum_to_one(self) -> "DataConfig":	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
28	total = self.train_fraction + self.val_fraction + self.test_fraction	Creates or updates `total`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
29	if abs(total - 1.0) > 1e-6:	Branches only when this condition is true; verify both the true and false paths in tests.
30	raise ValueError("train/val/test fractions must sum to 1.0")	Stops execution because an invariant or supported-condition check failed.
31	return self	Returns the result to the caller; this is the function output boundary that tests should assert.
34	class ModelConfig(BaseModel):	Defines a type that owns related state and behaviour behind a named interface.
35	kind: Literal["gru", "rsm", "ssm", "linear"] = "gru"	Creates or updates `kind: Literal["gru", "rsm", "ssm", "linear"]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
36	hidden_dim: int = Field(64, ge=8)	Creates or updates `hidden_dim: int`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
37	latent_dim: int = Field(16, ge=2)	Creates or updates `latent_dim: int`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
38	layers: int = Field(1, ge=1, le=8)	Creates or updates `layers: int`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
39	dropout: float = Field(0.0, ge=0.0, lt=1.0)	Creates or updates `dropout: float`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
42	class TrainingConfig(BaseModel):	Defines a type that owns related state and behaviour behind a named interface.
43	epochs: int = Field(4, ge=1)	Creates or updates `epochs: int`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
44	batch_size: int = Field(64, ge=1)	Creates or updates `batch_size: int`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

this point.

Lines 45-66

```

0045     learning_rate: float = Field(1e-3, gt=0)
0046     weight_decay: float = Field(1e-4, ge=0)
0047     grad_clip: float = Field(1.0, gt=0)
0048     patience: int = Field(3, ge=1)
0049     device: Literal["cpu", "cuda", "mps"] = "cpu"
0050     max_train_windows: int | None = Field(3500, ge=32)
0051
0052
0053     class SimulationConfig(BaseModel):
0054         rollout_steps: int = Field(80, ge=1)
0055         throttle_scale: float = Field(1.0, ge=0.0, le=2.0)
0056         brake_scale: float = Field(1.0, ge=0.0, le=2.0)
0057         grip_multiplier: float = Field(1.0, ge=0.3, le=1.5)
0058         rain_delta: float = Field(0.0, ge=-1.0, le=1.0)
0059         tyre_degradation_multiplier: float = Field(1.0, ge=0.1, le=3.0)
0060
0061
0062     class ProjectConfig(BaseModel):
0063         project_name: str = "Project APEX"
0064         seed: int = 42
0065         artifacts_dir: Path = Path("artifacts/runs")
0066         data: DataConfig = DataConfig()

```

Line	Code signal	What it is doing / what to verify
45	learning_rate: float = Field(1e-3, gt=0)	Creates or updates `learning_rate: float`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
46	weight_decay: float = Field(1e-4, ge=0)	Creates or updates `weight_decay: float`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
47	grad_clip: float = Field(1.0, gt=0)	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.
48	patience: int = Field(3, ge=1)	Creates or updates `patience: int`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
49	device: Literal["cpu", "cuda", "mps"] = "cpu"	Creates or updates `device: Literal["cpu", "cuda", "mps"]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
50	max_train_windows: int None = Field(3500, ge=32)	Creates or updates `max_train_windows: int   None`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
53	class SimulationConfig(BaseModel):	Defines a type that owns related state and behaviour behind a named interface.
54	rollout_steps: int = Field(80, ge=1)	Creates or updates `rollout_steps: int`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
55	throttle_scale: float = Field(1.0, ge=0.0, le=2.0)	Creates or updates `throttle_scale: float`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
56	brake_scale: float = Field(1.0, ge=0.0, le=2.0)	Creates or updates `brake_scale: float`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
57	grip_multiplier: float = Field(1.0, ge=0.3,	Creates or updates `grip_multiplier: float`.

	le=1.5)	Check its unit, shape, lifetime and whether it is derived only from information available at this point.
58	rain_delta: float = Field(0.0, ge=-1.0, le=1.0)	Creates or updates `rain_delta: float`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
59	tyre_degradation_multiplier: float = Field(1.0, ge=0.1, le=3.0)	Creates or updates `tyre_degradation_multiplier: float`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
62	class ProjectConfig(BaseModel):	Defines a type that owns related state and behaviour behind a named interface.
63	project_name: str = "Project APEX"	Creates or updates `project_name: str`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
64	seed: int = 42	Creates or updates `seed: int`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
65	artifacts_dir: Path = Path("artifacts/runs")	Creates or updates `artifacts_dir: Path`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
66	data: DataConfig = DataConfig()	Creates or updates `data: DataConfig`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 67-78

```

0067     model: ModelConfig = ModelConfig()
0068     training: TrainingConfig = TrainingConfig()
0069     simulation: SimulationConfig = SimulationConfig()
0070
0071
0072 def load_config(path: str | Path) -> ProjectConfig:
0073     config_path = Path(path)
0074     if not config_path.exists():
0075         raise FileNotFoundError(f"Configuration file does not exist: {config_path}")
0076     with config_path.open("r", encoding="utf-8") as handle:
0077         payload = yaml.safe_load(handle) or {}
0078     return ProjectConfig.model_validate(payload)

```

Line	Code signal	What it is doing / what to verify
67	model: ModelConfig = ModelConfig()	Creates or updates `model: ModelConfig`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
68	training: TrainingConfig = TrainingConfig()	Creates or updates `training: TrainingConfig`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
69	simulation: SimulationConfig = SimulationConfig()	Creates or updates `simulation: SimulationConfig`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
72	def load_config(path: str Path) -> ProjectConfig:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
73	config_path = Path(path)	Creates or updates `config_path`. Check its unit, shape, lifetime and whether it is

		derived only from information available at this point.
74	if not config_path.exists():	Branches only when this condition is true; verify both the true and false paths in tests.
75	raise FileNotFoundError(f"Configuration file does not exist: {config_path}")	Stops execution because an invariant or supported-condition check failed.
76	with config_path.open("r", encoding="utf-8") as handle:	Opens a managed resource or context and guarantees cleanup when the block exits.
77	payload = yaml.safe_load(handle) or {}	Creates or updates `payload`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
78	return ProjectConfig.model_validate(payload)	Returns the result to the caller; this is the function output boundary that tests should assert.

src/apexsim/contracts.py

Defines the canonical telemetry language used by every data source and model.

Treat field order, unit and availability time as part of the API. A model checkpoint is compatible only with the exact contract and preprocessing version it was trained against.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```
0001 from __future__ import annotations
0002
0003 from dataclasses import dataclass
0004 from typing import Final
0005
0006 import pandas as pd
0007
0008 # Canonical columns are deliberately independent of FastF1, OpenF1, and F1 25.
0009 # Every source adapter must translate into this stable contract.
0010 IDENTITY_COLUMNS: Final[list[str]] = [
0011     "session_id",
0012     "source",
0013     "track_id",
0014     "driver_id",
0015     "timestamp_s",
0016     "lap_number",
0017 ]
0018
0019 STATE_COLUMNS: Final[list[str]] = [
0020     "speed_mps",
0021     "acceleration_mps2",
0022     "track_progress_sin",
```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	from dataclasses import dataclass	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
4	from typing import Final	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
6	import pandas as pd	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
8	# Canonical columns are deliberately independent of FastF1, OpenF1, and F1 25.	Documents an assumption, intent or constraint that should remain aligned with the implementation.
9	# Every source adapter must translate into this stable contract.	Documents an assumption, intent or constraint that should remain aligned with the implementation.
10	IDENTITY_COLUMNS: Final[list[str]] = [	Creates or updates `IDENTITY_COLUMNS: Final[list[str]]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
11	"session_id",	Executes this statement inside the current

Source → contract → runtime evidence → failure test

		control-flow block; connect it to the nearest function contract and testable invariant.
12	"source",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
13	"track_id",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
14	"driver_id",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
15	"timestamp_s",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
16	"lap_number",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
17	]	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
19	STATE_COLUMNS: Final[list[str]] = [	Creates or updates `STATE_COLUMNS: Final[list[str]]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
20	"speed_mps",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
21	"acceleration_mps2",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
22	"track_progress_sin",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

Lines 23-44

```

0023     "track_progress_cos",
0024     "tyre_age_laps",
0025 ]
0026
0027 ACTION_COLUMNS: Final[list[str]] = [
0028     "throttle",
0029     "brake",
0030     "gear_norm",
0031     "drs",
0032     "steering_proxy",
0033 ]
0034
0035 CONTEXT_COLUMNS: Final[list[str]] = [
0036     "curvature",
0037     "air_temp_c",
0038     "track_temp_c",
0039     "rainfall",
0040     "wind_speed_mps",
0041     "grip_level",
0042 ]
0043
0044 AUX_COLUMNS: Final[list[str]] = [

```

Line	Code signal	What it is doing / what to verify
23	"track_progress_cos",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
24	"tyre_age_laps",	Executes this statement inside the current control-flow block; connect it to the nearest

		function contract and testable invariant.
25	]	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
27	ACTION_COLUMNS: Final[list[str]] = [	Creates or updates `ACTION_COLUMNS: Final[list[str]]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
28	"throttle",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
29	"brake",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
30	"gear_norm",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
31	"drs",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
32	"steering_proxy",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
33	]	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
35	CONTEXT_COLUMNS: Final[list[str]] = [	Creates or updates `CONTEXT_COLUMNS: Final[list[str]]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
36	"curvature",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
37	"air_temp_c",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
38	"track_temp_c",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
39	"rainfall",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
40	"wind_speed_mps",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
41	"grip_level",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
42	]	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
44	AUX_COLUMNS: Final[list[str]] = [	Creates or updates `AUX_COLUMNS: Final[list[str]]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 45-66

```

0045     "lap_distance_m",
0046     "x_m",
0047     "y_m",

```

```

0048     "rpm",
0049     "gear",
0050     "compound_id",
0051     "is_pit",
0052     "safety_car",
0053 ]
0054
0055 REQUIRED_COLUMNS: Final[list[str]] = (
0056     IDENTITY_COLUMNS + STATE_COLUMNS + ACTION_COLUMNS + CONTEXT_COLUMNS + AUX_COLUMNS
0057 )
0058
0059 MODEL_INPUT_COLUMNS: Final[list[str]] = STATE_COLUMNS + ACTION_COLUMNS + CONTEXT_COLUMNS
0060 TARGET_COLUMNS: Final[list[str]] = STATE_COLUMNS
0061
0062
0063 @dataclass(frozen=True)
0064 class ContractReport:
0065     rows: int
0066     sessions: int

```

Line	Code signal	What it is doing / what to verify
45	"lap_distance_m",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
46	"x_m",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
47	"y_m",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
48	"rpm",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
49	"gear",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
50	"compound_id",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
51	"is_pit",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
52	"safety_car",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
53	]	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
55	REQUIRED_COLUMNS: Final[list[str]] = (	Creates or updates `REQUIRED_COLUMNS: Final[list[str]]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
56	IDENTITY_COLUMNS + STATE_COLUMNS + ACTION_COLUMNS + CONTEXT_COLUMNS + AUX_COLUMN	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
57	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
59	MODEL_INPUT_COLUMNS: Final[list[str]] = STATE_COLUMNS + ACTION_COLUMNS + CONTEXT	Creates or updates `MODEL_INPUT_COLUMNS: Final[list[str]]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
60	TARGET_COLUMNS: Final[list[str]] = STATE_COLUMNS	Creates or updates `TARGET_COLUMNS: Final[list[str]]`. Check its unit, shape, lifetime

		and whether it is derived only from information available at this point.
63	@dataclass(frozen=True)	Applies a decorator that changes registration, validation or runtime behaviour of the definition that follows.
64	class ContractReport:	Defines a type that owns related state and behaviour behind a named interface.
65	rows: int	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
66	sessions: int	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

Lines 67-81

```

0067     drivers: int
0068     tracks: int
0069     null_cells: int
0070     duplicate_frames: int
0071     out_of_range_cells: int
0072     passed: bool
0073
0074     def to_dict(self) -> dict[str, int | bool]:
0075         return self.__dict__.copy()
0076
0077
0078 def assert_canonical_frame(frame: pd.DataFrame) -> None:
0079     missing = sorted(set(REQUIRED_COLUMNS) - set(frame.columns))
0080     if missing:
0081         raise ValueError(f"Canonical frame is missing columns: {missing}")

```

Line	Code signal	What it is doing / what to verify
67	drivers: int	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
68	tracks: int	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
69	null_cells: int	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
70	duplicate_frames: int	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
71	out_of_range_cells: int	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
72	passed: bool	Provides an intentionally empty block; verify that this is a deliberate extension point.
74	def to_dict(self) -> dict[str, int bool]:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
75	return self.__dict__.copy()	Returns the result to the caller; this is the function output boundary that tests should assert.
78	def assert_canonical_frame(frame: pd.DataFrame) -> None:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
79	missing = sorted(set(REQUIRED_COLUMNS) - set(frame.columns))	Creates or updates `missing`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

80	if missing:	Branches only when this condition is true; verify both the true and false paths in tests.
81	raise ValueError(f'Canonical frame is missing columns: {missing}')	Stops execution because an invariant or supported-condition check failed.

src/apexsim/data/__init__.py

Supports the APEX package behind an explicit module boundary.

Trace callers, inputs, outputs and side effects before editing.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-1

```
0001 """Data ingestion, validation, alignment, feature, and windowing utilities."""
```

Line	Code signal	What it is doing / what to verify
1	"""Data ingestion, validation, alignment, feature, and windowing utilities."""	Begins or ends documentation describing the module, class or function contract.

src/apexsim/data/fastf1_adapter.py

Converts FastF1 session objects into the canonical telemetry contract.

Source-specific names, rates and units end here; downstream model code must not depend on FastF1 internals.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```

0001 from __future__ import annotations
0002
0003 from pathlib import Path
0004
0005 import numpy as np
0006 import pandas as pd
0007
0008
0009 def ingest_fastf1_session(
0010     year: int,
0011     event: str,
0012     session_code: str,
0013     driver: str,
0014     output_path: str | Path,
0015     sample_hz: int = 5,
0016 ) -> pd.DataFrame:
0017     """Fetch a historical FastF1 session and translate it into APEX's canonical contract.
0018
0019     FastF1 is an optional dependency because the complete project must remain runnable offline.
0020     """
0021     try:
0022         import fastf1

```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	from pathlib import Path	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
5	import numpy as np	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
6	import pandas as pd	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
9	def ingest_fastf1_session(	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
10	year: int,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
11	event: str,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
12	session_code: str,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

13	driver: str,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
14	output_path: str Path,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
15	sample_hz: int = 5,	Creates or updates `sample_hz: int`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
16	) -> pd.DataFrame:	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
17	"""Fetch a historical FastF1 session and translate it into APEX's canonical cont	Begins or ends documentation describing the module, class or function contract.
19	FastF1 is an optional dependency because the complete project must remain runnab	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
20	"""	Begins or ends documentation describing the module, class or function contract.
21	try:	Begins an operation that may fail for an expected reason.
22	import fastfl	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.

Lines 23-44

```

0023     except ImportError as exc:
0024         raise RuntimeError(
0025             "FastF1 is not installed. Install the project with the 'real-data' extra."
0026         ) from exc
0027
0028     cache_dir = Path(".cache/fastfl")
0029     cache_dir.mkdir(parents=True, exist_ok=True)
0030     fastfl.Cache.enable_cache(str(cache_dir))
0031     session = fastfl.get_session(year, event, session_code)
0032     session.load(telemetry=True, laps=True, weather=True, messages=False)
0033
0034     driver_laps = session.laps.pick_drivers(driver)
0035     rows: list[pd.DataFrame] = []
0036     weather = session.weather_data.copy()
0037     if "Time" in weather:
0038         weather["timestamp_s"] = weather["Time"].dt.total_seconds()
0039
0040     for _, lap in driver_laps.iterlaps():
0041         telemetry = lap.get_telemetry().copy()
0042         if telemetry.empty:
0043             continue
0044         telemetry["timestamp_s"] = telemetry["SessionTime"].dt.total_seconds()

```

Line	Code signal	What it is doing / what to verify
23	except ImportError as exc:	Handles a specific failure mode. The handler should preserve useful diagnostic evidence.
24	raise RuntimeError(	Stops execution because an invariant or supported-condition check failed.
25	"FastF1 is not installed. Install the project with the 'real-data' extra."	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
26	) from exc	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
28	cache_dir = Path(".cache/fastfl")	Creates or updates `cache_dir`. Check its unit, shape, lifetime and whether it is derived only from information available at this

		point.
29	<code>cache_dir.mkdir(parents=True, exist_ok=True)</code>	Creates or updates <code>`cache_dir.mkdir(parents)`</code> . Check its unit, shape, lifetime and whether it is derived only from information available at this point.
30	<code>fastf1.Cache.enable_cache(str(cache_dir))</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
31	<code>session = fastf1.get_session(year, event, session_code)</code>	Creates or updates <code>`session`</code> . Check its unit, shape, lifetime and whether it is derived only from information available at this point.
32	<code>session.load(telemetry=True, laps=True, weather=True, messages=False)</code>	Creates or updates <code>`session.load(telemetry)`</code> . Check its unit, shape, lifetime and whether it is derived only from information available at this point.
34	<code>driver_laps = session.laps.pick_drivers(driver)</code>	Creates or updates <code>`driver_laps`</code> . Check its unit, shape, lifetime and whether it is derived only from information available at this point.
35	<code>rows: list[pd.DataFrame] = []</code>	Creates or updates <code>`rows: list[pd.DataFrame]`</code> . Check its unit, shape, lifetime and whether it is derived only from information available at this point.
36	<code>weather = session.weather_data.copy()</code>	Creates or updates <code>`weather`</code> . Check its unit, shape, lifetime and whether it is derived only from information available at this point.
37	<code>if "Time" in weather:</code>	Branches only when this condition is true; verify both the true and false paths in tests.
38	<code>weather["timestamp_s"] = weather["Time"].dt.total_seconds()</code>	Creates or updates <code>`weather["timestamp_s"]`</code> . Check its unit, shape, lifetime and whether it is derived only from information available at this point.
40	<code>for _, lap in driver_laps.iterlaps():</code>	Iterates over a collection or time dimension. Confirm order, boundaries and whether state should persist between iterations.
41	<code>telemetry = lap.get_telemetry().copy()</code>	Creates or updates <code>`telemetry`</code> . Check its unit, shape, lifetime and whether it is derived only from information available at this point.
42	<code>if telemetry.empty:</code>	Branches only when this condition is true; verify both the true and false paths in tests.
43	<code>continue</code>	Skips the remainder of the current iteration and advances to the next item.
44	<code>telemetry["timestamp_s"] = telemetry["SessionTime"].dt.total_seconds()</code>	Creates or updates <code>`telemetry["timestamp_s"]`</code> . Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 45-66

```

0045     start, end = telemetry.timestamp_s.min(), telemetry.timestamp_s.max()
0046     regular_time = np.arange(start, end, 1.0 / sample_hz)
0047     telemetry = telemetry.set_index("timestamp_s").reindex(regular_time).interpolate().ffill().bfill()
0048     telemetry.index.name = "timestamp_s"
0049     telemetry = telemetry.reset_index()
0050     telemetry["lap_number"] = int(lap.LapNumber)
0051     telemetry["tyre_age_laps"] = float(lap.TyreLife or 0) + (
0052         telemetry["Distance"] / max(float(telemetry["Distance"].max()), 1.0)
0053     )

```

```

0054     rows.append(telemetry)
0055
0056     if not rows:
0057         raise RuntimeError("FastF1 returned no telemetry for the requested driver/session.")
0058     tele = pd.concat(rows, ignore_index=True).sort_values("timestamp_s")
0059     distance = tele["Distance"].fillna(method="ffill").fillna(0.0)
0060     progress = (distance / max(float(distance.max()), 1.0)) % 1.0
0061     speed_mps = tele["Speed"].astype(float) / 3.6
0062     dt = tele.timestamp_s.diff().replace(0, np.nan).fillna(1.0 / sample_hz)
0063     acceleration = speed_mps.diff().fillna(0.0) / dt
0064     x = tele.get("X", pd.Series(np.zeros(len(tele)))).astype(float)
0065     y = tele.get("Y", pd.Series(np.zeros(len(tele)))).astype(float)
0066     heading = np.unwrap(np.arctan2(np.gradient(y), np.gradient(x)))

```

Line	Code signal	What it is doing / what to verify
45	start, end = telemetry.timestamp_s.min(), telemetry.timestamp_s.max()	Creates or updates `start, end`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
46	regular_time = np.arange(start, end, 1.0 / sample_hz)	Creates or updates `regular_time`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
47	telemetry = telemetry.set_index("timestamp_s").reindex(regular_time).interpolate	Creates or updates `telemetry`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
48	telemetry.index.name = "timestamp_s"	Creates or updates `telemetry.index.name`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
49	telemetry = telemetry.reset_index()	Creates or updates `telemetry`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
50	telemetry["lap_number"] = int(lap.LapNumber)	Creates or updates `telemetry["lap_number"]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
51	telemetry["tyre_age_laps"] = float(lap.TyreLife or 0) + (	Creates or updates `telemetry["tyre_age_laps"]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
52	telemetry["Distance"] / max(float(telemetry["Distance"].max()), 1.0)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
53	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
54	rows.append(telemetry)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
56	if not rows:	Branches only when this condition is true; verify both the true and false paths in tests.
57	raise RuntimeError("FastF1 returned no telemetry for the requested driver/sessio	Stops execution because an invariant or supported-condition check failed.
58	tele = pd.concat(rows, ignore_index=True).sort_values("timestamp_ s")	Concatenates tensors or arrays; verify that every non-concatenated axis has identical meaning and size.
59	distance = tele["Distance"].fillna(method="ffill").fillna(0. 0)	Creates or updates `distance`. Check its unit, shape, lifetime and whether it is derived only from information available at this

		point.
60	<code>progress = (distance / max(float(distance.max()), 1.0)) % 1.0</code>	Creates or updates `progress`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
61	<code>speed_mps = tele["Speed"].astype(float) / 3.6</code>	Creates or updates `speed_mps`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
62	<code>dt = tele.timestamp_s.diff().replace(0, np.nan).fillna(1.0 / sample_hz)</code>	Creates or updates `dt`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
63	<code>acceleration = speed_mps.diff().fillna(0.0) / dt</code>	Creates or updates `acceleration`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
64	<code>x = tele.get("X", pd.Series(np.zeros(len(tele))))).astype(float)</code>	Creates or updates `x`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
65	<code>y = tele.get("Y", pd.Series(np.zeros(len(tele))))).astype(float)</code>	Creates or updates `y`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
66	<code>heading = np.unwrap(np.arctan2(np.gradient(y), np.gradient(x)))</code>	Creates or updates `heading`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 67-88

```

0067 steering = np.clip(np.gradient(heading) * sample_hz / 1.8, -1, 1)
0068 curvature = np.clip(np.abs(np.gradient(heading)) * 5.0, 0, 1.5)
0069
0070 canonical = pd.DataFrame(
0071     {
0072         "session_id": f"FF1_{year}_{event}_{session_code}",
0073         "source": "fastf1",
0074         "track_id": str(event),
0075         "driver_id": str(driver),
0076         "timestamp_s": tele.timestamp_s,
0077         "lap_number": tele.lap_number,
0078         "speed_mps": speed_mps,
0079         "acceleration_mps2": acceleration,
0080         "track_progress_sin": np.sin(2 * np.pi * progress),
0081         "track_progress_cos": np.cos(2 * np.pi * progress),
0082         "tyre_age_laps": tele.tyre_age_laps,
0083         "throttle": tele["Throttle"].astype(float) / 100.0,
0084         "brake": tele["Brake"].astype(float),
0085         "gear_norm": tele["nGear"].astype(float).clip(0, 8) / 8.0,
0086         "drs": (tele["DRS"].astype(float) > 9).astype(float),
0087         "steering_proxy": steering,
0088         "curvature": curvature,

```

Line	Code signal	What it is doing / what to verify
67	<code>steering = np.clip(np.gradient(heading) * sample_hz / 1.8, -1, 1)</code>	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.
68	<code>curvature = np.clip(np.abs(np.gradient(heading)) * 5.0, 0, 1.5)</code>	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.
70	<code>canonical = pd.DataFrame(</code>	Creates or updates `canonical`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
71	<code>{</code>	Executes this statement inside the current control-flow block; connect it to the nearest

		function contract and testable invariant.
72	"session_id": f"FF1_{year}_{event}_{session_code}",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
73	"source": "fastf1",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
74	"track_id": str(event),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
75	"driver_id": str(driver),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
76	"timestamp_s": tele.timestamp_s,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
77	"lap_number": tele.lap_number,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
78	"speed_mps": speed_mps,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
79	"acceleration_mps2": acceleration,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
80	"track_progress_sin": np.sin(2 * np.pi * progress),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
81	"track_progress_cos": np.cos(2 * np.pi * progress),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
82	"tyre_age_laps": tele.tyre_age_laps,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
83	"throttle": tele["Throttle"].astype(float) / 100.0,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
84	"brake": tele["Brake"].astype(float),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
85	"gear_norm": tele["nGear"].astype(float).clip(0, 8) / 8.0,	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.
86	"drs": (tele["DRS"].astype(float) > 9).astype(float),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
87	"steering_proxy": steering,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
88	"curvature": curvature,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

Lines 89-107

```

0089     "air_temp_c": 25.0,
0090     "track_temp_c": 35.0,
0091     "rainfall": 0.0,
0092     "wind_speed_mps": 2.0,
0093     "grip_level": 1.0,
0094     "lap_distance_m": distance,
0095     "x_m": x,
```

Source → contract → runtime evidence → failure test

```

0096         "y_m": y,
0097         "rpm": tele["RPM"].astype(float),
0098         "gear": tele["nGear"].astype(int),
0099         "compound_id": 0,
0100         "is_pit": 0,
0101         "safety_car": 0,
0102     }
0103 )
0104 output = Path(output_path)
0105 output.parent.mkdir(parents=True, exist_ok=True)
0106 canonical.to_csv(output, index=False)
0107 return canonical

```

Line	Code signal	What it is doing / what to verify
89	"air_temp_c": 25.0,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
90	"track_temp_c": 35.0,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
91	"rainfall": 0.0,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
92	"wind_speed_mps": 2.0,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
93	"grip_level": 1.0,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
94	"lap_distance_m": distance,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
95	"x_m": x,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
96	"y_m": y,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
97	"rpm": tele["RPM"].astype(float),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
98	"gear": tele["nGear"].astype(int),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
99	"compound_id": 0,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
100	"is_pit": 0,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
101	"safety_car": 0,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
102	}	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
103	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
104	output = Path(output_path)	Creates or updates `output`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

105	<code>output.parent.mkdir(parents=True, exist_ok=True)</code>	Creates or updates <code>`output.parent.mkdir(parents`</code> . Check its unit, shape, lifetime and whether it is derived only from information available at this point.
106	<code>canonical.to_csv(output, index=False)</code>	Persists an artifact. The surrounding code should include run identity, schema and preprocessing lineage.
107	<code>return canonical</code>	Returns the result to the caller; this is the function output boundary that tests should assert.

src/apexsim/data/features.py

Creates causal derived signals and feature arrays.

Every feature must be available at prediction time and have one stable unit/order definition.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```
0001 from __future__ import annotations
0002
0003 from dataclasses import dataclass
0004
0005 import numpy as np
0006 import pandas as pd
0007
0008 from apexsim.contracts import MODEL_INPUT_COLUMNS, TARGET_COLUMNS
0009
0010
0011 @dataclass
0012 class Standardizer:
0013     input_mean: np.ndarray
0014     input_std: np.ndarray
0015     target_mean: np.ndarray
0016     target_std: np.ndarray
0017
0018     @classmethod
0019     def fit(cls, frame: pd.DataFrame) -> "Standardizer":
0020         x = frame[MODEL_INPUT_COLUMNS].to_numpy(np.float32)
0021         y = frame[TARGET_COLUMNS].to_numpy(np.float32)
0022         x_std = x.std(axis=0)
```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	from dataclasses import dataclass	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
5	import numpy as np	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
6	import pandas as pd	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
8	from apexsim.contracts import MODEL_INPUT_COLUMNS, TARGET_COLUMNS	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
11	@dataclass	Applies a decorator that changes registration, validation or runtime behaviour of the definition that follows.
12	class Standardizer:	Defines a type that owns related state and behaviour behind a named interface.
13	input_mean: np.ndarray	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
14	input_std: np.ndarray	Executes this statement inside the current

		control-flow block; connect it to the nearest function contract and testable invariant.
15	target_mean: np.ndarray	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
16	target_std: np.ndarray	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
18	@classmethod	Applies a decorator that changes registration, validation or runtime behaviour of the definition that follows.
19	def fit(cls, frame: pd.DataFrame) -> "Standardizer":	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
20	x = frame[MODEL_INPUT_COLUMNS].to_numpy(np.float32)	Creates or updates `x`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
21	y = frame[TARGET_COLUMNS].to_numpy(np.float32)	Creates or updates `y`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
22	x_std = x.std(axis=0)	Creates or updates `x_std`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 23-44

```

0023     y_std = y.std(axis=0)
0024     return cls(
0025         input_mean=x.mean(axis=0),
0026         input_std=np.where(x_std < 1e-6, 1.0, x_std),
0027         target_mean=y.mean(axis=0),
0028         target_std=np.where(y_std < 1e-6, 1.0, y_std),
0029     )
0030
0031     def transform_inputs(self, values: np.ndarray) -> np.ndarray:
0032         return (values - self.input_mean) / self.input_std
0033
0034     def transform_targets(self, values: np.ndarray) -> np.ndarray:
0035         return (values - self.target_mean) / self.target_std
0036
0037     def inverse_targets(self, values: np.ndarray) -> np.ndarray:
0038         return values * self.target_std + self.target_mean
0039
0040     def to_dict(self) -> dict[str, list[float]]:
0041         return {
0042             "input_mean": self.input_mean.tolist(),
0043             "input_std": self.input_std.tolist(),
0044             "target_mean": self.target_mean.tolist(),

```

Line	Code signal	What it is doing / what to verify
23	y_std = y.std(axis=0)	Creates or updates `y_std`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
24	return cls(	Returns the result to the caller; this is the function output boundary that tests should assert.
25	input_mean=x.mean(axis=0),	Creates or updates `input_mean`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
26	input_std=np.where(x_std < 1e-6, 1.0, x_std),	Creates or updates `input_std`. Check its unit, shape, lifetime and whether it is derived only from information available at this

		point.
27	target_mean=y.mean(axis=0),	Creates or updates `target_mean`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
28	target_std=np.where(y_std < 1e-6, 1.0, y_std),	Creates or updates `target_std`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
29	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
31	def transform_inputs(self, values: np.ndarray) -> np.ndarray:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
32	return (values - self.input_mean) / self.input_std	Returns the result to the caller; this is the function output boundary that tests should assert.
34	def transform_targets(self, values: np.ndarray) -> np.ndarray:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
35	return (values - self.target_mean) / self.target_std	Returns the result to the caller; this is the function output boundary that tests should assert.
37	def inverse_targets(self, values: np.ndarray) -> np.ndarray:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
38	return values * self.target_std + self.target_mean	Returns the result to the caller; this is the function output boundary that tests should assert.
40	def to_dict(self) -> dict[str, list[float]]:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
41	return {	Returns the result to the caller; this is the function output boundary that tests should assert.
42	"input_mean": self.input_mean.tolist(),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
43	"input_std": self.input_std.tolist(),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
44	"target_mean": self.target_mean.tolist(),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

Lines 45-50

```

0045     "target_std": self.target_std.tolist(),
0046     }
0047
0048     @classmethod
0049     def from_dict(cls, payload: dict[str, list[float]]) -> "Standardizer":
0050         return cls(**{key: np.asarray(value, dtype=np.float32) for key, value in payload.items()})

```

Line	Code signal	What it is doing / what to verify
45	"target_std": self.target_std.tolist(),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
46	}	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

48	@classmethod	Applies a decorator that changes registration, validation or runtime behaviour of the definition that follows.
49	def from_dict(cls, payload: dict[str, list[float]]) -> "Standardizer":	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
50	return cls(**{key: np.asarray(value, dtype=np.float32) for key, value in payload	Returns the result to the caller; this is the function output boundary that tests should assert.

src/apexsim/data/openf1_adapter.py

Retrieves and aligns OpenF1 endpoint streams.

Every as-of join needs a direction, tolerance and freshness interpretation; nearest data is not automatically simultaneous data.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```
0001 from __future__ import annotations
0002
0003 from pathlib import Path
0004
0005 import numpy as np
0006 import pandas as pd
0007 import requests
0008
0009
0010 BASE_URL = "https://api.openf1.org/v1"
0011
0012
0013 def _get(endpoint: str, params: dict[str, int | str], timeout: int = 60) -> list[dict]:
0014     response = requests.get(f"{BASE_URL}/{endpoint}", params=params, timeout=timeout)
0015     response.raise_for_status()
0016     payload = response.json()
0017     if not isinstance(payload, list):
0018         raise ValueError(f"Expected a list from OpenF1 {endpoint}, got {type(payload).__name__}")
0019     return payload
0020
0021
0022 def ingest_openf1_session(
```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	from pathlib import Path	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
5	import numpy as np	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
6	import pandas as pd	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
7	import requests	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
10	BASE_URL = "https://api.openf1.org/v1"	Creates or updates `BASE_URL`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
13	def _get(endpoint: str, params: dict[str, int str], timeout: int = 60) -> list	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
14	response =	Creates or updates `response`. Check its unit,

Source → contract → runtime evidence → failure test

	<code>requests.get(f'{BASE_URL}/{endpoint}', params=params, timeout=timeout</code>	shape, lifetime and whether it is derived only from information available at this point.
15	<code>response.raise_for_status()</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
16	<code>payload = response.json()</code>	Creates or updates `payload`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
17	<code>if not isinstance(payload, list):</code>	Branches only when this condition is true; verify both the true and false paths in tests.
18	<code>raise ValueError(f'Expected a list from OpenF1 {endpoint}, got {type(payload)}.')</code>	Stops execution because an invariant or supported-condition check failed.
19	<code>return payload</code>	Returns the result to the caller; this is the function output boundary that tests should assert.
22	<code>def ingest_openf1_session(</code>	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.

Lines 23-44

```

0023     session_key: int,
0024     driver_number: int,
0025     output_path: str | Path,
0026     sample_hz: int = 4,
0027 ) -> pd.DataFrame:
0028     """Translate OpenF1 car/location/weather streams into the canonical frame.
0029
0030     Historical OpenF1 data is rich but the endpoints are sampled independently. We therefore align
0031     streams on time instead of assuming row N in one endpoint matches row N in another.
0032     """
0033     car = pd.DataFrame(_get("car_data", {"session_key": session_key, "driver_number": driver_number}))
0034     location = pd.DataFrame(_get("location", {"session_key": session_key, "driver_number": driver_number}))
0035     laps = pd.DataFrame(_get("laps", {"session_key": session_key, "driver_number": driver_number}))
0036     weather = pd.DataFrame(_get("weather", {"session_key": session_key}))
0037     if car.empty or location.empty:
0038         raise RuntimeError("OpenF1 returned no car or location data for the request.")
0039
0040     for frame in (car, location, weather):
0041         frame["date"] = pd.to_datetime(frame["date"], utc=True)
0042     car = car.sort_values("date")
0043     location = location.sort_values("date")
0044     weather = weather.sort_values("date")

```

Line	Code signal	What it is doing / what to verify
23	<code>session_key: int,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
24	<code>driver_number: int,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
25	<code>output_path: str Path,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
26	<code>sample_hz: int = 4,</code>	Creates or updates `sample_hz: int`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
27	<code>) -> pd.DataFrame:</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
28	<code>"""Translate OpenF1 car/location/weather streams into the canonical frame.</code>	Begins or ends documentation describing the module, class or function contract.

30	Historical OpenF1 data is rich but the endpoints are sampled independently. We t	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
31	streams on time instead of assuming row N in one endpoint matches row N in anoth	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
32	""	Begins or ends documentation describing the module, class or function contract.
33	car = pd.DataFrame(_get("car_data", {"session_key": session_key, "driver_number"	Creates or updates `car`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
34	location = pd.DataFrame(_get("location", {"session_key": session_key, "driver_nu	Creates or updates `location`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
35	laps = pd.DataFrame(_get("laps", {"session_key": session_key, "driver_number": d	Creates or updates `laps`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
36	weather = pd.DataFrame(_get("weather", {"session_key": session_key}))	Creates or updates `weather`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
37	if car.empty or location.empty:	Branches only when this condition is true; verify both the true and false paths in tests.
38	raise RuntimeError("OpenF1 returned no car or location data for the request.")	Stops execution because an invariant or supported-condition check failed.
40	for frame in (car, location, weather):	Iterates over a collection or time dimension. Confirm order, boundaries and whether state should persist between iterations.
41	frame["date"] = pd.to_datetime(frame["date"], utc=True)	Creates or updates `frame["date"]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
42	car = car.sort_values("date")	Creates or updates `car`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
43	location = location.sort_values("date")	Creates or updates `location`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
44	weather = weather.sort_values("date")	Creates or updates `weather`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 45-66

```

0045 merged = pd.merge_asof(car, location, on="date", direction="nearest", tolerance=pd.Timedelta("1s"))
0046 merged = pd.merge_asof(merged, weather, on="date", direction="nearest", tolerance=pd.Timedelta("20s"),
suffixes=("", "_weather"))
0047 merged = merged.dropna(subset=["speed", "x", "y"])
0048 start = merged.date.min()
0049 merged["timestamp_s"] = (merged.date - start).dt.total_seconds()
0050 regular_time = np.arange(0, merged.timestamp_s.max(), 1.0 / sample_hz)
0051 numeric = merged.select_dtypes(include=[np.number]).copy()
0052 numeric.index = merged.timestamp_s
0053 numeric = numeric[~numeric.index.duplicated()].reindex(regular_time).interpolate().ffill().bfill()
0054 numeric.index.name = "timestamp_s"
0055 numeric = numeric.reset_index()
0056
0057 speed_mps = numeric["speed"] / 3.6

```

Source → contract → runtime evidence → failure test

```

0058 dt = numeric.timestamp_s.diff().replace(0, np.nan).fillna(1.0 / sample_hz)
0059 accel = speed_mps.diff().fillna(0.0) / dt
0060 x, y = numeric["x"], numeric["y"]
0061 heading = np.unwrap(np.arctan2(np.gradient(y), np.gradient(x)))
0062 curvature = np.clip(np.abs(np.gradient(heading)) * 5.0, 0, 1.5)
0063 distance = np.cumsum(speed_mps.to_numpy() / sample_hz)
0064 track_length = max(float(np.quantile(distance, 0.99)), 1.0)
0065 progress = (distance / track_length) % 1.0
0066

```

Line	Code signal	What it is doing / what to verify
45	<code>merged = pd.merge_asof(car, location, on="date", direction="nearest", tolerance=</code>	Aligns asynchronous time streams by nearby timestamps; direction, tolerance and sort order are part of the scientific contract.
46	<code>merged = pd.merge_asof(merged, weather, on="date", direction="nearest", toleranc</code>	Aligns asynchronous time streams by nearby timestamps; direction, tolerance and sort order are part of the scientific contract.
47	<code>merged = merged.dropna(subset=["speed", "x", "y"])</code>	Creates or updates `merged`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
48	<code>start = merged.date.min()</code>	Creates or updates `start`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
49	<code>merged["timestamp_s"] = (merged.date - start).dt.total_seconds()</code>	Creates or updates `merged["timestamp_s"]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
50	<code>regular_time = np.arange(0, merged.timestamp_s.max(), 1.0 / sample_hz)</code>	Creates or updates `regular_time`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
51	<code>numeric = merged.select_dtypes(include=[np.number]).copy()</code>	Creates or updates `numeric`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
52	<code>numeric.index = merged.timestamp_s</code>	Creates or updates `numeric.index`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
53	<code>numeric = numeric[~numeric.index.duplicated()].reindex(regular_time).interpolate</code>	Creates or updates `numeric`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
54	<code>numeric.index.name = "timestamp_s"</code>	Creates or updates `numeric.index.name`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
55	<code>numeric = numeric.reset_index()</code>	Creates or updates `numeric`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
57	<code>speed_mps = numeric["speed"] / 3.6</code>	Creates or updates `speed_mps`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
58	<code>dt = numeric.timestamp_s.diff().replace(0, np.nan).fillna(1.0 / sample_hz)</code>	Creates or updates `dt`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
59	<code>accel = speed_mps.diff().fillna(0.0) / dt</code>	Creates or updates `accel`. Check its unit, shape, lifetime and whether it is derived only from information available at this

		point.
60	<code>x, y = numeric["x"], numeric["y"]</code>	Creates or updates `x, y`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
61	<code>heading = np.unwrap(np.arctan2(np.gradient(y), np.gradient(x)))</code>	Creates or updates `heading`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
62	<code>curvature = np.clip(np.abs(np.gradient(heading)) * 5.0, 0, 1.5)</code>	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.
63	<code>distance = np.cumsum(speed_mps.to_numpy0 / sample_hz)</code>	Creates or updates `distance`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
64	<code>track_length = max(float(np.quantile(distance, 0.99)), 1.0)</code>	Creates or updates `track_length`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
65	<code>progress = (distance / track_length) % 1.0</code>	Creates or updates `progress`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 67-88

```

0067     canonical = pd.DataFrame(
0068         {
0069             "session_id": f"OF1_{session_key}",
0070             "source": "openf1",
0071             "track_id": f"session_{session_key}",
0072             "driver_id": str(driver_number),
0073             "timestamp_s": numeric.timestamp_s,
0074             "lap_number": (distance // track_length + 1).astype(int),
0075             "speed_mps": speed_mps,
0076             "acceleration_mps2": accel,
0077             "track_progress_sin": np.sin(2 * np.pi * progress),
0078             "track_progress_cos": np.cos(2 * np.pi * progress),
0079             "tyre_age_laps": distance / track_length,
0080             "throttle": numeric.get("throttle", pd.Series(np.zeros(len(numeric)))) / 100.0,
0081             "brake": numeric.get("brake", pd.Series(np.zeros(len(numeric))).clip(0, 1),
0082             "gear_norm": numeric.get("n_gear", pd.Series(np.ones(len(numeric)))) / 8.0,
0083             "drs": numeric.get("drs", pd.Series(np.zeros(len(numeric))).clip(0, 1),
0084             "steering_proxy": np.clip(np.gradient(heading) * sample_hz / 1.8, -1, 1),
0085             "curvature": curvature,
0086             "air_temp_c": numeric.get("air_temperature", pd.Series(np.full(len(numeric), 25.0))),
0087             "track_temp_c": numeric.get("track_temperature", pd.Series(np.full(len(numeric), 35.0))),
0088             "rainfall": numeric.get("rainfall", pd.Series(np.zeros(len(numeric))).clip(0, 1),

```

Line	Code signal	What it is doing / what to verify
67	<code>canonical = pd.DataFrame(</code>	Creates or updates `canonical`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
68	<code>{</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
69	<code>"session_id": f"OF1_{session_key}",</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
70	<code>"source": "openf1",</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
71	<code>"track_id": f"session_{session_key}",</code>	Executes this statement inside the current control-flow block; connect it to the nearest

		function contract and testable invariant.
72	"driver_id": str(driver_number),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
73	"timestamp_s": numeric.timestamp_s,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
74	"lap_number": (distance // track_length + 1).astype(int),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
75	"speed_mps": speed_mps,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
76	"acceleration_mps2": accel,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
77	"track_progress_sin": np.sin(2 * np.pi * progress),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
78	"track_progress_cos": np.cos(2 * np.pi * progress),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
79	"tyre_age_laps": distance / track_length,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
80	"throttle": numeric.get("throttle", pd.Series(np.zeros(len(numeric)))) / 100.0,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
81	"brake": numeric.get("brake", pd.Series(np.zeros(len(numeric)))) .clip(0, 1),	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.
82	"gear_norm": numeric.get("n_gear", pd.Series(np.ones(len(numeric)))) / 8.0,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
83	"drs": numeric.get("drs", pd.Series(np.zeros(len(numeric)))) .clip(0, 1),	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.
84	"steering_proxy": np.clip(np.gradient(heading) * sample_hz / 1.8, -1, 1),	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.
85	"curvature": curvature,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
86	"air_temp_c": numeric.get("air_temperature", pd.Series(np.full(len(numeric), 25.	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
87	"track_temp_c": numeric.get("track_temperature", pd.Series(np.full(len(numeric),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
88	"rainfall": numeric.get("rainfall", pd.Series(np.zeros(len(numeric)))) .clip(0, 1	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.

Lines 89-104

```

0089     "wind_speed_mps": numeric.get("wind_speed", pd.Series(np.full(len(numeric), 2.0))),
0090     "grip_level": 1.0,
0091     "lap_distance_m": distance % track_length,
0092     "x_m": x,
0093     "y_m": y,
0094     "rpm": numeric.get("rpm", pd.Series(np.full(len(numeric), 9000.0))),
0095     "gear": numeric.get("n_gear", pd.Series(np.ones(len(numeric)))) .astype(int),

```

Source → contract → runtime evidence → failure test


```

0096         "compound_id": 0,
0097         "is_pit": 0,
0098         "safety_car": 0,
0099     }
0100 )
0101 output = Path(output_path)
0102 output.parent.mkdir(parents=True, exist_ok=True)
0103 canonical.to_csv(output, index=False)
0104 return canonical

```

Line	Code signal	What it is doing / what to verify
89	"wind_speed_mps": numeric.get("wind_speed", pd.Series(np.full(len(numeric), 2.0))	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
90	"grip_level": 1.0,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
91	"lap_distance_m": distance % track_length,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
92	"x_m": x,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
93	"y_m": y,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
94	"rpm": numeric.get("rpm", pd.Series(np.full(len(numeric), 9000.0))),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
95	"gear": numeric.get("n_gear", pd.Series(np.ones(len(numeric))))).astype(int) ,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
96	"compound_id": 0,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
97	"is_pit": 0,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
98	"safety_car": 0,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
99	}	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
100	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
101	output = Path(output_path)	Creates or updates `output`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
102	output.parent.mkdir(parents=True, exist_ok=True)	Creates or updates `output.parent.mkdir(parents)`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
103	canonical.to_csv(output, index=False)	Persists an artifact. The surrounding code should include run identity, schema and preprocessing lineage.
104	return canonical	Returns the result to the caller; this is the function output boundary that tests should assert.

src/apexsim/data/synthetic.py

Creates a controlled causal telemetry world for offline learning and tests.

Synthetic data is used to verify contracts and interventions, not to justify real-world accuracy claims.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```
0001 from __future__ import annotations
0002
0003 import math
0004 from pathlib import Path
0005
0006 import numpy as np
0007 import pandas as pd
0008
0009 from apexsim.config import ProjectConfig
0010
0011
0012 def _track_geometry(track_length_m: float, points: int, phase: float) -> pd.DataFrame:
0013     distance = np.linspace(0.0, track_length_m, points, endpoint=False)
0014     theta = 2.0 * np.pi * distance / track_length_m
0015     # A closed, non-circular educational track. It is not an F1 circuit replica.
0016     radius = 780.0 + 135.0 * np.sin(3 * theta + phase) + 70.0 * np.sin(7 * theta)
0017     x = radius * np.cos(theta) + 90.0 * np.sin(2 * theta)
0018     y = 0.78 * radius * np.sin(theta) + 65.0 * np.sin(5 * theta + phase)
0019     dx = np.gradient(x, distance)
0020     dy = np.gradient(y, distance)
0021     heading = np.unwrap(np.arctan2(dy, dx))
0022     curvature = np.abs(np.gradient(heading, distance))
```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	import math	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
4	from pathlib import Path	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
6	import numpy as np	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
7	import pandas as pd	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
9	from apexsim.config import ProjectConfig	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
12	def _track_geometry(track_length_m: float, points: int, phase: float) -> pd.DataFrame:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
13	distance = np.linspace(0.0, track_length_m, points, endpoint=False)	Creates or updates `distance`. Check its unit, shape, lifetime and whether it is derived only from information available at this

		point.
14	<code>theta = 2.0 * np.pi * distance / track_length_m</code>	Creates or updates `theta`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
15	<code># A closed, non-circular educational track. It is not an F1 circuit replica.</code>	Documents an assumption, intent or constraint that should remain aligned with the implementation.
16	<code>radius = 780.0 + 135.0 * np.sin(3 * theta + phase) + 70.0 * np.sin(7 * theta)</code>	Creates or updates `radius`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
17	<code>x = radius * np.cos(theta) + 90.0 * np.sin(2 * theta)</code>	Creates or updates `x`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
18	<code>y = 0.78 * radius * np.sin(theta) + 65.0 * np.sin(5 * theta + phase)</code>	Creates or updates `y`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
19	<code>dx = np.gradient(x, distance)</code>	Creates or updates `dx`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
20	<code>dy = np.gradient(y, distance)</code>	Creates or updates `dy`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
21	<code>heading = np.unwrap(np.arctan2(dy, dx))</code>	Creates or updates `heading`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
22	<code>curvature = np.abs(np.gradient(heading, distance))</code>	Creates or updates `curvature`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 23-44

```

0023     curvature /= max(float(np.quantile(curvature, 0.99)), 1e-6)
0024     curvature = np.clip(curvature, 0.0, 1.5)
0025     return pd.DataFrame(
0026         {
0027             "lap_distance_m": distance,
0028             "x_m": x,
0029             "y_m": y,
0030             "curvature": curvature,
0031             "heading": heading,
0032         }
0033     )
0034
0035
0036 def _gear_from_speed(speed_mps: float) -> int:
0037     # Simple gear map for a teaching simulator.
0038     kmh = speed_mps * 3.6
0039     thresholds = [0, 70, 105, 140, 180, 220, 260, 300]
0040     return int(np.clip(np.searchsorted(thresholds, kmh, side="right"), 1, 8))
0041
0042
0043 def generate_synthetic_sessions(config: ProjectConfig, output_path: str | Path) -> pd.DataFrame:
0044     """Generate causal F1-like telemetry with actions, weather, tyre wear, and track geometry.

```

Line	Code signal	What it is doing / what to verify
23	<code>curvature /= max(float(np.quantile(curvature, 0.99)), 1e-6)</code>	Creates or updates `curvature`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
24	<code>curvature = np.clip(curvature, 0.0, 1.5)</code>	Bounds a value to a supported range. Monitor how often clipping occurs because it

		may hide distribution shift.
25	return pd.DataFrame(	Returns the result to the caller; this is the function output boundary that tests should assert.
26	{	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
27	"lap_distance_m": distance,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
28	"x_m": x,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
29	"y_m": y,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
30	"curvature": curvature,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
31	"heading": heading,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
32	}	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
33	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
36	def gear_from_speed(speed_mps: float) -> int:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
37	# Simple gear map for a teaching simulator.	Documents an assumption, intent or constraint that should remain aligned with the implementation.
38	kmh = speed_mps * 3.6	Creates or updates `kmh`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
39	thresholds = [0, 70, 105, 140, 180, 220, 260, 300]	Creates or updates `thresholds`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
40	return int(np.clip(np.searchsorted(thresholds, kmh, side="right"), 1, 8))	Returns the result to the caller; this is the function output boundary that tests should assert.
43	def generate_synthetic_sessions(config: ProjectConfig, output_path: str Path)	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
44	"""Generate causal F1-like telemetry with actions, weather, tyre wear, and track	Begins or ends documentation describing the module, class or function contract.

Lines 45-66

```

0045
0046     The generator gives us a controlled world where the hidden rules are known. This is invaluable
0047     for testing whether the pipeline and models recover cause-and-effect rather than dataset leakage.
0048     """
0049     rng = np.random.default_rng(config.seed)
0050     hz = config.data.sample_hz
0051     dt = 1.0 / hz
0052     frames_per_session = int(config.data.session_seconds * hz)
0053     output_frames: list[pd.DataFrame] = []

```

Source → contract → runtime evidence → failure test

```

0054
0055     for session_idx in range(config.data.sessions):
0056         track_id = f"synthetic_track_{session_idx % 3}"
0057         session_id = f"SYN_{session_idx:03d}"
0058         driver_id = f"DRV_{session_idx % 5:02d}"
0059         source = "synthetic"
0060         phase = (session_idx % 3) * 0.7
0061         geometry = _track_geometry(config.data.track_length_m, 2048, phase)
0062
0063         driver_aggression = 0.93 + 0.14 * rng.random()
0064         driver_smoothness = 0.85 + 0.14 * rng.random()
0065         wet_session = rng.random() < 0.28
0066         rainfall_base = rng.uniform(0.18, 0.65) if wet_session else rng.uniform(0.0, 0.035)

```

Line	Code signal	What it is doing / what to verify
46	The generator gives us a controlled world where the hidden rules are known. This	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
47	for testing whether the pipeline and models recover cause-and-effect rather than	Iterates over a collection or time dimension. Confirm order, boundaries and whether state should persist between iterations.
48	"""	Begins or ends documentation describing the module, class or function contract.
49	rng = np.random.default_rng(config.seed)	Creates or updates `rng`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
50	hz = config.data.sample_hz	Creates or updates `hz`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
51	dt = 1.0 / hz	Creates or updates `dt`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
52	frames_per_session = int(config.data.session_seconds * hz)	Creates or updates `frames_per_session`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
53	output_frames: list[pd.DataFrame] = []	Creates or updates `output_frames: list[pd.DataFrame]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
55	for session_idx in range(config.data.sessions):	Iterates over a collection or time dimension. Confirm order, boundaries and whether state should persist between iterations.
56	track_id = f"synthetic_track_{session_idx % 3}"	Creates or updates `track_id`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
57	session_id = f"SYN_{session_idx:03d}"	Creates or updates `session_id`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
58	driver_id = f"DRV_{session_idx % 5:02d}"	Creates or updates `driver_id`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
59	source = "synthetic"	Creates or updates `source`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
60	phase = (session_idx % 3) * 0.7	Creates or updates `phase`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

61	geometry = _track_geometry(config.data.track_length_m, 2048, phase)	Creates or updates `geometry`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
63	driver_aggression = 0.93 + 0.14 * rng.random()	Creates or updates `driver_aggression`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
64	driver_smoothness = 0.85 + 0.14 * rng.random()	Creates or updates `driver_smoothness`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
65	wet_session = rng.random() < 0.28	Creates or updates `wet_session`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
66	rainfall_base = rng.uniform(0.18, 0.65) if wet_session else rng.uniform(0.0, 0.0)	Creates or updates `rainfall_base`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 67-88

```

0067     air_temp = rng.uniform(18.0, 31.0)
0068     track_temp = air_temp + rng.uniform(4.0, 18.0)
0069     wind = rng.uniform(0.5, 7.0)
0070     grip = rng.uniform(0.92, 1.06) * (1.0 - 0.35 * rainfall_base)
0071     compound_id = int(rng.integers(0, 3))
0072     tyre_deg_rate = [0.000035, 0.000022, 0.000015][compound_id]
0073
0074     speed = rng.uniform(45.0, 60.0)
0075     distance_total = 0.0
0076     previous_speed = speed
0077     previous_heading = 0.0
0078     records: list[dict[str, float | int | str]] = []
0079
0080     for frame_idx in range(frames_per_session):
0081         timestamp = frame_idx * dt
0082         progress = (distance_total % config.data.track_length_m) / config.data.track_length_m
0083         geo_idx = min(int(progress * len(geometry)), len(geometry) - 1)
0084         point = geometry.iloc[geo_idx]
0085         curvature = float(point.curvature)
0086         tyre_age_laps = distance_total / config.data.track_length_m
0087         lap_number = int(tyre_age_laps) + 1
0088

```

Line	Code signal	What it is doing / what to verify
67	air_temp = rng.uniform(18.0, 31.0)	Creates or updates `air_temp`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
68	track_temp = air_temp + rng.uniform(4.0, 18.0)	Creates or updates `track_temp`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
69	wind = rng.uniform(0.5, 7.0)	Creates or updates `wind`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
70	grip = rng.uniform(0.92, 1.06) * (1.0 - 0.35 * rainfall_base)	Creates or updates `grip`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
71	compound_id = int(rng.integers(0, 3))	Creates or updates `compound_id`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

		this point.
72	<code>tyre_deg_rate = [0.000035, 0.000022, 0.000015][compound_id]</code>	Creates or updates `tyre_deg_rate`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
74	<code>speed = rng.uniform(45.0, 60.0)</code>	Creates or updates `speed`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
75	<code>distance_total = 0.0</code>	Creates or updates `distance_total`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
76	<code>previous_speed = speed</code>	Creates or updates `previous_speed`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
77	<code>previous_heading = 0.0</code>	Creates or updates `previous_heading`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
78	<code>records: list[dict[str, float int str]] = []</code>	Creates or updates `records: list[dict[str, float   int   str]]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
80	<code>for frame_idx in range(frames_per_session):</code>	Iterates over a collection or time dimension. Confirm order, boundaries and whether state should persist between iterations.
81	<code>timestamp = frame_idx * dt</code>	Creates or updates `timestamp`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
82	<code>progress = (distance_total % config.data.track_length_m) / config.data.track_len</code>	Creates or updates `progress`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
83	<code>geo_idx = min(int(progress * len(geometry)), len(geometry) - 1)</code>	Creates or updates `geo_idx`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
84	<code>point = geometry.iloc[geo_idx]</code>	Creates or updates `point`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
85	<code>curvature = float(point.curvature)</code>	Creates or updates `curvature`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
86	<code>tyre_age_laps = distance_total / config.data.track_length_m</code>	Creates or updates `tyre_age_laps`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
87	<code>lap_number = int(tyre_age_laps) + 1</code>	Creates or updates `lap_number`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 89-110

```

0089         # Slowly changing rain lets the model learn weather-conditioned dynamics.
0090         rainfall = float(np.clip(rainfall_base + 0.08 * math.sin(timestamp / 38.0 + phase), 0.0, 1.0))
0091         effective_grip = float(np.clip(grip - tyre_deg_rate * distance_total - 0.3 * rainfall, 0.45, 1.1))
0092         max_corner_speed = 92.0 * effective_grip / (1.0 + 4.8 * curvature)

```

Source → contract → runtime evidence → failure test


```

0093     straight_speed = 93.0 * driver_aggression
0094     target_speed = min(straight_speed, max_corner_speed)
0095     error = target_speed - speed
0096
0097     throttle = float(np.clip(0.5 + 0.045 * error, 0.0, 1.0))
0098     brake = float(np.clip(-0.06 * error - 0.12, 0.0, 1.0))
0099     # Smooth drivers avoid violent simultaneous input changes.
0100     control_noise = rng.normal(0.0, 0.025 * (1.1 - driver_smoothness))
0101     throttle = float(np.clip(throttle + control_noise, 0.0, 1.0))
0102     brake = float(np.clip(brake - control_noise, 0.0, 1.0))
0103     drs = int(curvature < 0.07 and throttle > 0.82 and rainfall < 0.12)
0104
0105     engine_accel = 13.5 * throttle * (1.0 + 0.05 * drs)
0106     brake_accel = 21.5 * brake
0107     aero_drag = 0.00145 * speed * speed
0108     corner_drag = 7.0 * curvature * speed / 80.0
0109     wet_drag = 2.0 * rainfall
0110     stochastic_force = rng.normal(0.0, 0.20)

```

Line	Code signal	What it is doing / what to verify
89	# Slowly changing rain lets the model learn weather-conditioned dynamics.	Documents an assumption, intent or constraint that should remain aligned with the implementation.
90	rainfall = float(np.clip(rainfall_base + 0.08 * math.sin(timestamp / 38.0 + phas	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.
91	effective_grip = float(np.clip(grip - tyre_deg_rate * distance_total - 0.3 * rai	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.
92	max_corner_speed = 92.0 * effective_grip / (1.0 + 4.8 * curvature)	Creates or updates `max_corner_speed`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
93	straight_speed = 93.0 * driver_aggression	Creates or updates `straight_speed`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
94	target_speed = min(straight_speed, max_corner_speed)	Creates or updates `target_speed`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
95	error = target_speed - speed	Creates or updates `error`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
97	throttle = float(np.clip(0.5 + 0.045 * error, 0.0, 1.0))	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.
98	brake = float(np.clip(-0.06 * error - 0.12, 0.0, 1.0))	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.
99	# Smooth drivers avoid violent simultaneous input changes.	Documents an assumption, intent or constraint that should remain aligned with the implementation.
100	control_noise = rng.normal(0.0, 0.025 * (1.1 - driver_smoothness))	Creates or updates `control_noise`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
101	throttle = float(np.clip(throttle + control_noise, 0.0, 1.0))	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.
102	brake = float(np.clip(brake - control_noise, 0.0, 1.0))	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.
103	drs = int(curvature < 0.07 and throttle > 0.82	Creates or updates `drs`. Check its unit,

	and rainfall < 0.12)	shape, lifetime and whether it is derived only from information available at this point.
105	engine_accel = 13.5 * throttle * (1.0 + 0.05 * drs)	Creates or updates `engine_accel`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
106	brake_accel = 21.5 * brake	Creates or updates `brake_accel`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
107	aero_drag = 0.00145 * speed * speed	Creates or updates `aero_drag`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
108	corner_drag = 7.0 * curvature * speed / 80.0	Creates or updates `corner_drag`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
109	wet_drag = 2.0 * rainfall	Creates or updates `wet_drag`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
110	stochastic_force = rng.normal(0.0, 0.20)	Creates or updates `stochastic_force`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 111-132

```

0111 acceleration = engine_accel - brake_accel - aero_drag - corner_drag - wet_drag + stochastic_force
0112 speed = float(np.clip(speed + acceleration * dt, 8.0, 101.0))
0113 distance_total += speed * dt
0114
0115 heading = float(point.heading)
0116 heading_change = (heading - previous_heading + np.pi) % (2 * np.pi) - np.pi
0117 steering_proxy = float(np.clip(heading_change / max(dt, 1e-6) / 1.8, -1.0, 1.0))
0118 previous_heading = heading
0119 gear = _gear_from_speed(speed)
0120 rpm = float(np.clip(4500 + 1100 * gear + 65 * speed + rng.normal(0, 180), 4000, 15000))
0121 acceleration_measured = (speed - previous_speed) / dt
0122 previous_speed = speed
0123
0124 records.append(
0125     {
0126         "session_id": session_id,
0127         "source": source,
0128         "track_id": track_id,
0129         "driver_id": driver_id,
0130         "timestamp_s": timestamp,
0131         "lap_number": lap_number,
0132         "speed_mps": speed,

```

Line	Code signal	What it is doing / what to verify
111	acceleration = engine_accel - brake_accel - aero_drag - corner_drag - wet_drag +	Creates or updates `acceleration`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
112	speed = float(np.clip(speed + acceleration * dt, 8.0, 101.0))	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.
113	distance_total += speed * dt	Creates or updates `distance_total`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
115	heading = float(point.heading)	Creates or updates `heading`. Check its unit,

		shape, lifetime and whether it is derived only from information available at this point.
116	<code>heading_change = (heading - previous_heading + np.pi) % (2 * np.pi) - np.pi</code>	Creates or updates `heading_change`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
117	<code>steering_proxy = float(np.clip(heading_change / max(dt, 1e-6) / 1.8, -1.0, 1.0))</code>	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.
118	<code>previous_heading = heading</code>	Creates or updates `previous_heading`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
119	<code>gear = _gear_from_speed(speed)</code>	Creates or updates `gear`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
120	<code>rpm = float(np.clip(4500 + 1100 * gear + 65 * speed + rng.normal(0, 180), 4000,</code>	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.
121	<code>acceleration_measured = (speed - previous_speed) / dt</code>	Creates or updates `acceleration_measured`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
122	<code>previous_speed = speed</code>	Creates or updates `previous_speed`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
124	<code>records.append(</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
125	<code>{</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
126	<code>"session_id": session_id,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
127	<code>"source": source,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
128	<code>"track_id": track_id,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
129	<code>"driver_id": driver_id,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
130	<code>"timestamp_s": timestamp,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
131	<code>"lap_number": lap_number,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
132	<code>"speed_mps": speed,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

Lines 133-154

```

0133     "acceleration_mps2": acceleration_measured,
0134     "track_progress_sin": math.sin(2 * math.pi * progress),
0135     "track_progress_cos": math.cos(2 * math.pi * progress),

```

Source → contract → runtime evidence → failure test

```

0136     "tyre_age_laps": tyre_age_laps,
0137     "throttle": throttle,
0138     "brake": brake,
0139     "gear_norm": gear / 8.0,
0140     "drs": float(drs),
0141     "steering_proxy": steering_proxy,
0142     "curvature": curvature,
0143     "air_temp_c": air_temp,
0144     "track_temp_c": track_temp,
0145     "rainfall": rainfall,
0146     "wind_speed_mps": wind,
0147     "grip_level": effective_grip,
0148     "lap_distance_m": distance_total % config.data.track_length_m,
0149     "x_m": float(point.x_m),
0150     "y_m": float(point.y_m),
0151     "rpm": rpm,
0152     "gear": gear,
0153     "compound_id": compound_id,
0154     "is_pit": 0,

```

Line	Code signal	What it is doing / what to verify
133	"acceleration_mps2": acceleration_measured,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
134	"track_progress_sin": math.sin(2 * math.pi * progress),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
135	"track_progress_cos": math.cos(2 * math.pi * progress),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
136	"tyre_age_laps": tyre_age_laps,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
137	"throttle": throttle,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
138	"brake": brake,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
139	"gear_norm": gear / 8.0,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
140	"drs": float(drs),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
141	"steering_proxy": steering_proxy,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
142	"curvature": curvature,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
143	"air_temp_c": air_temp,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
144	"track_temp_c": track_temp,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
145	"rainfall": rainfall,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
146	"wind_speed_mps": wind,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
147	"grip_level": effective_grip,	Executes this statement inside the current

		control-flow block; connect it to the nearest function contract and testable invariant.
148	"lap_distance_m": distance_total % config.data.track_length_m,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
149	"x_m": float(point.x_m),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
150	"y_m": float(point.y_m),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
151	"rpm": rpm,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
152	"gear": gear,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
153	"compound_id": compound_id,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
154	"is_pit": 0,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

Lines 155-165

```

0155         "safety_car": 0,
0156     }
0157 )
0158
0159     output_frames.append(pd.DataFrame.from_records(records))
0160
0161     result = pd.concat(output_frames, ignore_index=True)
0162     output = Path(output_path)
0163     output.parent.mkdir(parents=True, exist_ok=True)
0164     result.to_csv(output, index=False)
0165     return result

```

Line	Code signal	What it is doing / what to verify
155	"safety_car": 0,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
156	}	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
157	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
159	output_frames.append(pd.DataFrame.from_records(records))	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
161	result = pd.concat(output_frames, ignore_index=True)	Concatenates tensors or arrays; verify that every non-concatenated axis has identical meaning and size.
162	output = Path(output_path)	Creates or updates `output`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
163	output.parent.mkdir(parents=True, exist_ok=True)	Creates or updates `output.parent.mkdir(parents)`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

164	<code>result.to_csv(output, index=False)</code>	Persists an artifact. The surrounding code should include run identity, schema and preprocessing lineage.
165	<code>return result</code>	Returns the result to the caller; this is the function output boundary that tests should assert.

src/apexsim/data/validate.py

Implements schema, range, time and consistency quality gates.

The earliest cheap check should catch a defect before it reaches windows, normalization or training.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```
0001 from __future__ import annotations
0002
0003 import numpy as np
0004 import pandas as pd
0005
0006 from apexsim.contracts import ContractReport, REQUIRED_COLUMNS, assert_canonical_frame
0007
0008
0009 def validate_canonical_frame(frame: pd.DataFrame, strict: bool = True) -> ContractReport:
0010     assert_canonical_frame(frame)
0011     null_cells = int(frame[REQUIRED_COLUMNS].isna().sum().sum())
0012     duplicate_frames = int(
0013         frame.duplicated(subset=["session_id", "driver_id", "timestamp_s"]).sum()
0014     )
0015
0016     violations = np.zeros(len(frame), dtype=bool)
0017     range_checks = {
0018         "speed_mps": (0.0, 120.0),
0019         "throttle": (0.0, 1.0),
0020         "brake": (0.0, 1.0),
0021         "gear_norm": (0.0, 1.0),
0022         "drs": (0.0, 1.0),
```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	import numpy as np	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
4	import pandas as pd	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
6	from apexsim.contracts import ContractReport, REQUIRED_COLUMNS, assert_canonical	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
9	def validate_canonical_frame(frame: pd.DataFrame, strict: bool = True) -> Contra	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
10	assert_canonical_frame(frame)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
11	null_cells = int(frame[REQUIRED_COLUMNS].isna().sum().sum())	Creates or updates `null_cells`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
12	duplicate_frames = int(	Creates or updates `duplicate_frames`. Check its unit, shape, lifetime and whether it is

		derived only from information available at this point.
13	<code>frame.duplicated(subset=["session_id", "driver_id", "timestamp_s"]).sum()</code>	Creates or updates `frame.duplicated(subset)`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
14	<code>)</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
16	<code>violations = np.zeros(len(frame), dtype=bool)</code>	Creates or updates `violations`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
17	<code>range_checks = {</code>	Creates or updates `range_checks`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
18	<code>"speed_mps": (0.0, 120.0),</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
19	<code>"throttle": (0.0, 1.0),</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
20	<code>"brake": (0.0, 1.0),</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
21	<code>"gear_norm": (0.0, 1.0),</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
22	<code>"drs": (0.0, 1.0),</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

Lines 23-44

```

0023     "steering_proxy": (-1.0, 1.0),
0024     "rainfall": (0.0, 1.0),
0025     "grip_level": (0.2, 1.5),
0026 }
0027 for column, (minimum, maximum) in range_checks.items():
0028     values = pd.to_numeric(frame[column], errors="coerce")
0029     violations |= values.lt(minimum).to_numpy() | values.gt(maximum).to_numpy()
0030 out_of_range_cells = int(violations.sum())
0031
0032 passed = null_cells == 0 and duplicate_frames == 0 and out_of_range_cells == 0
0033 report = ContractReport(
0034     rows=len(frame),
0035     sessions=int(frame.session_id.nunique()),
0036     drivers=int(frame.driver_id.nunique()),
0037     tracks=int(frame.track_id.nunique()),
0038     null_cells=null_cells,
0039     duplicate_frames=duplicate_frames,
0040     out_of_range_cells=out_of_range_cells,
0041     passed=passed,
0042 )
0043 if strict and not passed:
0044     raise ValueError(f"Canonical data contract failed: {report.to_dict()}")

```

Line	Code signal	What it is doing / what to verify
23	<code>"steering_proxy": (-1.0, 1.0),</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
24	<code>"rainfall": (0.0, 1.0),</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

25	"grip_level": (0.2, 1.5),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
26	}	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
27	for column, (minimum, maximum) in range_checks.items():	Iterates over a collection or time dimension. Confirm order, boundaries and whether state should persist between iterations.
28	values = pd.to_numeric(frame[column], errors="coerce")	Creates or updates `values`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
29	violations = values.lt(minimum).to_numpy() values.gt(maximum).to_numpy()	Creates or updates `violations`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
30	out_of_range_cells = int(violations.sum())	Creates or updates `out_of_range_cells`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
32	passed = null_cells == 0 and duplicate_frames == 0 and out_of_range_cells == 0	Provides an intentionally empty block; verify that this is a deliberate extension point.
33	report = ContractReport(	Creates or updates `report`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
34	rows=len(frame),	Creates or updates `rows`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
35	sessions=int(frame.session_id.nunique()),	Creates or updates `sessions`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
36	drivers=int(frame.driver_id.nunique()),	Creates or updates `drivers`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
37	tracks=int(frame.track_id.nunique()),	Creates or updates `tracks`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
38	null_cells=null_cells,	Creates or updates `null_cells`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
39	duplicate_frames=duplicate_frames,	Creates or updates `duplicate_frames`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
40	out_of_range_cells=out_of_range_cells,	Creates or updates `out_of_range_cells`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
41	passed=passed,	Provides an intentionally empty block; verify that this is a deliberate extension point.
42	)	Calls a function or method. Trace its input contract, returned value and any persistent

		side effects.
43	if strict and not passed:	Branches only when this condition is true; verify both the true and false paths in tests.
44	raise ValueError(f"Canonical data contract failed: {report.to_dict()}")	Stops execution because an invariant or supported-condition check failed.

Lines 45-45

0045 return report

Line	Code signal	What it is doing / what to verify
45	return report	Returns the result to the caller; this is the function output boundary that tests should assert.

src/apexsim/data/windows.py

Builds group-safe sequence windows and normalization artifacts.

An off-by-one or cross-session window can make a model appear excellent while invalidating the experiment.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```
0001 from __future__ import annotations
0002
0003 from dataclasses import dataclass
0004
0005 import numpy as np
0006 import pandas as pd
0007 import torch
0008 from torch.utils.data import Dataset
0009
0010 from apexsim.contracts import MODEL_INPUT_COLUMNS, STATE_COLUMNS, TARGET_COLUMNS
0011 from apexsim.data.features import Standardizer
0012
0013
0014 @dataclass(frozen=True)
0015 class WindowIndex:
0016     session_id: str
0017     start: int
0018     stop: int
0019
0020
0021 def split_sessions(
0022     frame: pd.DataFrame,
```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	from dataclasses import dataclass	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
5	import numpy as np	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
6	import pandas as pd	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
7	import torch	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
8	from torch.utils.data import Dataset	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
10	from apexsim.contracts import MODEL_INPUT_COLUMNS, STATE_COLUMNS, TARGET_COLUMNS	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
11	from apexsim.data.features import Standardizer	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.

14	@dataclass(frozen=True)	Applies a decorator that changes registration, validation or runtime behaviour of the definition that follows.
15	class WindowIndex:	Defines a type that owns related state and behaviour behind a named interface.
16	session_id: str	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
17	start: int	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
18	stop: int	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
21	def split_sessions(	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
22	frame: pd.DataFrame,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

Lines 23-44

```

0023     train_fraction: float,
0024     val_fraction: float,
0025     seed: int,
0026 ) -> dict[str, list[str]]:
0027     sessions = sorted(frame.session_id.unique().tolist())
0028     rng = np.random.default_rng(seed)
0029     rng.shuffle(sessions)
0030     n = len(sessions)
0031     n_train = max(1, int(round(n * train_fraction)))
0032     n_val = max(1, int(round(n * val_fraction)))
0033     if n_train + n_val >= n:
0034         n_val = 1
0035         n_train = max(1, n - 2)
0036     return {
0037         "train": sessions[:n_train],
0038         "val": sessions[n_train : n_train + n_val],
0039         "test": sessions[n_train + n_val :],
0040     }
0041
0042
0043 class TelemetryWindowDataset(Dataset):
0044     """Returns histories and next-state sequences without crossing session boundaries."""

```

Line	Code signal	What it is doing / what to verify
23	train_fraction: float,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
24	val_fraction: float,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
25	seed: int,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
26	) -> dict[str, list[str]]:	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
27	sessions = sorted(frame.session_id.unique().tolist())	Creates or updates `sessions`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
28	rng = np.random.default_rng(seed)	Creates or updates `rng`. Check its unit, shape, lifetime and whether it is derived

		only from information available at this point.
29	<code>rng.shuffle(sessions)</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
30	<code>n = len(sessions)</code>	Creates or updates <code>`n`</code> . Check its unit, shape, lifetime and whether it is derived only from information available at this point.
31	<code>n_train = max(1, int(round(n * train_fraction)))</code>	Creates or updates <code>`n_train`</code> . Check its unit, shape, lifetime and whether it is derived only from information available at this point.
32	<code>n_val = max(1, int(round(n * val_fraction)))</code>	Creates or updates <code>`n_val`</code> . Check its unit, shape, lifetime and whether it is derived only from information available at this point.
33	<code>if n_train + n_val >= n:</code>	Branches only when this condition is true; verify both the true and false paths in tests.
34	<code>n_val = 1</code>	Creates or updates <code>`n_val`</code> . Check its unit, shape, lifetime and whether it is derived only from information available at this point.
35	<code>n_train = max(1, n - 2)</code>	Creates or updates <code>`n_train`</code> . Check its unit, shape, lifetime and whether it is derived only from information available at this point.
36	<code>return {</code>	Returns the result to the caller; this is the function output boundary that tests should assert.
37	<code>"train": sessions[:n_train],</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
38	<code>"val": sessions[n_train : n_train + n_val],</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
39	<code>"test": sessions[n_train + n_val :],</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
40	<code>}</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
43	<code>class TelemetryWindowDataset(Dataset):</code>	Defines a type that owns related state and behaviour behind a named interface.
44	<code>"""Returns histories and next-state sequences without crossing session boundarie</code>	Begins or ends documentation describing the module, class or function contract.

Lines 45-66

```

0045
0046     def __init__(
0047         self,
0048         frame: pd.DataFrame,
0049         session_ids: list[str],
0050         sequence_length: int,
0051         prediction_horizon: int,
0052         standardizer: Standardizer,
0053         max_windows: int | None = None,
0054     ) -> None:
0055         self.sequence_length = sequence_length
0056         self.prediction_horizon = prediction_horizon
0057         self.standardizer = standardizer
0058         self.sessions: dict[str, pd.DataFrame] = {}

```

```

0059     self.indices: list[WindowIndex] = []
0060     total = sequence_length + prediction_horizon
0061     for session_id in session_ids:
0062         session = (
0063             frame[frame.session_id == session_id]
0064             .sort_values("timestamp_s")
0065             .reset_index(drop=True)
0066         )

```

Line	Code signal	What it is doing / what to verify
46	def __init__(	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
47	self,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
48	frame: pd.DataFrame,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
49	session_ids: list[str],	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
50	sequence_length: int,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
51	prediction_horizon: int,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
52	standardizer: Standardizer,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
53	max_windows: int None = None,	Creates or updates `max_windows: int   None`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
54	) -> None:	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
55	self.sequence_length = sequence_length	Creates or updates `self.sequence_length`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
56	self.prediction_horizon = prediction_horizon	Runs model inference using learned parameters without fitting on the requested examples.
57	self.standardizer = standardizer	Creates or updates `self.standardizer`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
58	self.sessions: dict[str, pd.DataFrame] = {}	Creates or updates `self.sessions: dict[str, pd.DataFrame]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
59	self.indices: list[WindowIndex] = []	Creates or updates `self.indices: list[WindowIndex]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
60	total = sequence_length + prediction_horizon	Creates or updates `total`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
61	for session_id in session_ids:	Iterates over a collection or time dimension. Confirm order, boundaries and whether

		state should persist between iterations.
62	session = (	Creates or updates `session`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
63	frame[frame.session_id == session_id]	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
64	.sort_values("timestamp_s")	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
65	.reset_index(drop=True)	Creates or updates `.reset_index(drop)`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
66	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.

Lines 67-88

```

0067         self.sessions[session_id] = session
0068         for start in range(0, len(session) - total + 1):
0069             self.indices.append(WindowIndex(session_id, start, start + total))
0070         if max_windows is not None and len(self.indices) > max_windows:
0071             positions = np.linspace(0, len(self.indices) - 1, max_windows).astype(int)
0072             self.indices = [self.indices[i] for i in positions]
0073
0074     def __len__(self) -> int:
0075         return len(self.indices)
0076
0077     def __getitem__(self, index: int) -> dict[str, torch.Tensor | str]:
0078         item = self.indices[index]
0079         window = self.sessions[item.session_id].iloc[item.start : item.stop]
0080         history = window.iloc[: self.sequence_length]
0081         future = window.iloc[self.sequence_length :]
0082         x_history = history[MODEL_INPUT_COLUMNS].to_numpy(np.float32)
0083         x_future = future[MODEL_INPUT_COLUMNS].to_numpy(np.float32)
0084         y_future = future[TARGET_COLUMNS].to_numpy(np.float32)
0085         state_history = history[STATE_COLUMNS].to_numpy(np.float32)
0086         return {
0087             "history": torch.from_numpy(self.standardizer.transform_inputs(x_history).astype(np.float32)),
0088             "future_inputs": torch.from_numpy(self.standardizer.transform_inputs(x_future).astype(np.float32)),

```

Line	Code signal	What it is doing / what to verify
67	self.sessions[session_id] = session	Creates or updates `self.sessions[session_id]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
68	for start in range(0, len(session) - total + 1):	Iterates over a collection or time dimension. Confirm order, boundaries and whether state should persist between iterations.
69	self.indices.append(WindowIndex(session_id, start, start + total))	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
70	if max_windows is not None and len(self.indices) > max_windows:	Branches only when this condition is true; verify both the true and false paths in tests.
71	positions = np.linspace(0, len(self.indices) - 1, max_windows).astype(int)	Creates or updates `positions`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
72	self.indices = [self.indices[i] for i in positions]	Creates or updates `self.indices`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

74	<code>def __len__(self) -> int:</code>	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
75	<code>return len(self.indices)</code>	Returns the result to the caller; this is the function output boundary that tests should assert.
77	<code>def __getitem__(self, index: int) -> dict[str, torch.Tensor str]:</code>	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
78	<code>item = self.indices[index]</code>	Creates or updates `item`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
79	<code>window = self.sessions[item.session_id].iloc[item.start : item.stop]</code>	Creates or updates `window`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
80	<code>history = window.iloc[: self.sequence_length]</code>	Creates or updates `history`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
81	<code>future = window.iloc[self.sequence_length :]</code>	Creates or updates `future`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
82	<code>x_history = history[MODEL_INPUT_COLUMNS].to_numpy(np.float32)</code>	Creates or updates `x_history`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
83	<code>x_future = future[MODEL_INPUT_COLUMNS].to_numpy(np.float32)</code>	Creates or updates `x_future`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
84	<code>y_future = future[TARGET_COLUMNS].to_numpy(np.float32)</code>	Creates or updates `y_future`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
85	<code>state_history = history[STATE_COLUMNS].to_numpy(np.float32)</code>	Creates or updates `state_history`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
86	<code>return {</code>	Returns the result to the caller; this is the function output boundary that tests should assert.
87	<code>"history": torch.from_numpy(self.standardizer.transform_inputs(x_history).astype</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
88	<code>"future_inputs": torch.from_numpy(self.standardizer.transform_inputs(x_future).a</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

Lines 89-92

```

0089         "future_targets": torch.from_numpy(self.standardizer.transform_targets(y_future).astype(np.float32)),
0090         "state_history":
torch.from_numpy(self.standardizer.transform_targets(state_history).astype(np.float32)),
0091         "session_id": item.session_id,
0092     }

```

Line	Code signal	What it is doing / what to verify
89	<code>"future_targets": torch.from_numpy(self.standardizer.transform_targets(y_future)</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

90	"state_history": torch.from_numpy(self.standardizer.transfo rm_targets(state_hist	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
91	"session_id": item.session_id,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
92	}	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

src/apexsim/evaluation.py

Converts standardized model output into physical and horizon-aware evidence.

Evaluation must use held-out groups, inverse transformations and physical guardrails; teacher-forced loss alone is not simulation quality.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```
0001 from __future__ import annotations
0002
0003 import json
0004 from pathlib import Path
0005
0006 import numpy as np
0007 import torch
0008 from torch import nn
0009 from torch.utils.data import DataLoader
0010
0011 from apexsim.contracts import TARGET_COLUMNS
0012 from apexsim.data.features import Standardizer
0013 from apexsim.models.rssm import RSSMWorldModel
0014
0015
0016 def evaluate_world_model(
0017     model: nn.Module,
0018     loader: DataLoader,
0019     standardizer: Standardizer,
0020     output_path: str | Path | None = None,
0021     device: str = "cpu",
0022 ) -> dict:
```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	import json	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
4	from pathlib import Path	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
6	import numpy as np	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
7	import torch	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
8	from torch import nn	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
9	from torch.utils.data import DataLoader	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
11	from apexsim.contracts import TARGET_COLUMNS	Imports a dependency required by the definitions below; keeping imports explicit

Source → contract → runtime evidence → failure test

		reveals architectural coupling.
12	from apexsim.data.features import Standardizer	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
13	from apexsim.models.rssm import RSSMWorldModel	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
16	def evaluate_world_model(	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
17	model: nn.Module,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
18	loader: DataLoader,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
19	standardizer: Standardizer,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
20	output_path: str Path None = None,	Creates or updates `output_path: str   Path   None`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
21	device: str = "cpu",	Creates or updates `device: str`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
22	) -> dict:	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

Lines 23-44

```

0023     model.eval()
0024     model.to(device)
0025     predictions: list[np.ndarray] = []
0026     targets: list[np.ndarray] = []
0027     with torch.no_grad():
0028         for batch in loader:
0029             history = batch["history"].to(device)
0030             future_inputs = batch["future_inputs"].to(device)
0031             if isinstance(model, RSSMWorldModel):
0032                 predicted, _ = model(history, future_inputs, future_targets=None)
0033             else:
0034                 predicted = model(history, future_inputs)
0035             predictions.append(predicted.cpu().numpy())
0036             targets.append(batch["future_targets"].numpy())
0037     pred_z = np.concatenate(predictions, axis=0)
0038     target_z = np.concatenate(targets, axis=0)
0039     pred = standardizer.inverse_targets(pred_z)
0040     target = standardizer.inverse_targets(target_z)
0041     error = pred - target
0042     horizon_rmse = np.sqrt(np.mean(error**2, axis=(0, 2)))
0043     horizon_mae = np.mean(np.abs(error), axis=(0, 2))
0044     per_feature = {}

```

Line	Code signal	What it is doing / what to verify
23	model.eval()	Enables deterministic evaluation behaviour for layers that differ between training and inference.
24	model.to(device)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
25	predictions: list[np.ndarray] = []	Creates or updates `predictions: list[np.ndarray]`. Check its unit, shape,

		lifetime and whether it is derived only from information available at this point.
26	<code>targets: list[np.ndarray] = []</code>	Creates or updates `targets: list[np.ndarray]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
27	<code>with torch.no_grad():</code>	Opens a managed resource or context and guarantees cleanup when the block exits.
28	<code>for batch in loader:</code>	Iterates over a collection or time dimension. Confirm order, boundaries and whether state should persist between iterations.
29	<code>history = batch["history"].to(device)</code>	Creates or updates `history`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
30	<code>future_inputs = batch["future_inputs"].to(device)</code>	Creates or updates `future_inputs`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
31	<code>if isinstance(model, RSSMWorldModel):</code>	Branches only when this condition is true; verify both the true and false paths in tests.
32	<code>predicted, _ = model(history, future_inputs, future_targets=None)</code>	Creates or updates `predicted, _`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
33	<code>else:</code>	Handles all remaining cases; inspect whether this fallback is intentionally broad.
34	<code>predicted = model(history, future_inputs)</code>	Creates or updates `predicted`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
35	<code>predictions.append(predicted.cpu().numpy())</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
36	<code>targets.append(batch["future_targets"].numpy())</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
37	<code>pred_z = np.concatenate(predictions, axis=0)</code>	Concatenates tensors or arrays; verify that every non-concatenated axis has identical meaning and size.
38	<code>target_z = np.concatenate(targets, axis=0)</code>	Concatenates tensors or arrays; verify that every non-concatenated axis has identical meaning and size.
39	<code>pred = standardizer.inverse_targets(pred_z)</code>	Creates or updates `pred`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
40	<code>target = standardizer.inverse_targets(target_z)</code>	Creates or updates `target`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
41	<code>error = pred - target</code>	Creates or updates `error`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
42	<code>horizon_rmse = np.sqrt(np.mean(error**2, axis=(0, 2)))</code>	Creates or updates `horizon_rmse`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
43	<code>horizon_mae = np.mean(np.abs(error), axis=(0, 2))</code>	Creates or updates `horizon_mae`. Check its unit, shape, lifetime and whether it is

		derived only from information available at this point.
44	per_feature = {}	Creates or updates `per_feature`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 45-66

```

0045     for index, feature in enumerate(TARGET_COLUMNS):
0046         per_feature[feature] = {
0047             "mae": float(np.mean(np.abs(error[... , index]))),
0048             "rmse": float(np.sqrt(np.mean(error[... , index] ** 2))),
0049         }
0050     physical = {
0051         "negative_speed_rate": float(np.mean(pred[... , 0] < 0)),
0052         "extreme_speed_rate": float(np.mean(pred[... , 0] > 120)),
0053         "progress_unit_circle_error": float(
0054             np.mean(np.abs(pred[... , 2] ** 2 + pred[... , 3] ** 2 - 1.0))
0055         ),
0056     }
0057     metrics = {
0058         "overall_mae": float(np.mean(np.abs(error))),
0059         "overall_rmse": float(np.sqrt(np.mean(error**2))),
0060         "speed_mae_mps": per_feature["speed_mps"]["mae"],
0061         "speed_rmse_mps": per_feature["speed_mps"]["rmse"],
0062         "per_feature": per_feature,
0063         "horizon_mae": horizon_mae.tolist(),
0064         "horizon_rmse": horizon_rmse.tolist(),
0065         "physical_violations": physical,
0066         "samples": int(pred.shape[0]),

```

Line	Code signal	What it is doing / what to verify
45	for index, feature in enumerate(TARGET_COLUMNS):	Iterates over a collection or time dimension. Confirm order, boundaries and whether state should persist between iterations.
46	per_feature[feature] = {	Creates or updates `per_feature[feature]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
47	"mae": float(np.mean(np.abs(error[... , index]))),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
48	"rmse": float(np.sqrt(np.mean(error[... , index] ** 2))),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
49	}	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
50	physical = {	Creates or updates `physical`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
51	"negative_speed_rate": float(np.mean(pred[... , 0] < 0)),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
52	"extreme_speed_rate": float(np.mean(pred[... , 0] > 120)),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
53	"progress_unit_circle_error": float(np.mean(np.abs(pred[... , 2] ** 2 + pred[... , 3] ** 2 - 1.0))	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
54		Calls a function or method. Trace its input contract, returned value and any persistent side effects.

55	),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
56	}	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
57	metrics = {	Creates or updates `metrics`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
58	"overall_mae": float(np.mean(np.abs(error))),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
59	"overall_rmse": float(np.sqrt(np.mean(error**2))),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
60	"speed_mae_mps": per_feature["speed_mps"] ["mae"],	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
61	"speed_rmse_mps": per_feature["speed_mps"]["rmse"],	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
62	"per_feature": per_feature,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
63	"horizon_mae": horizon_mae.tolist(),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
64	"horizon_rmse": horizon_rmse.tolist(),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
65	"physical_violations": physical,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
66	"samples": int(pred.shape[0]),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

Lines 67-72

```

0067     }
0068     if output_path is not None:
0069         path = Path(output_path)
0070         path.parent.mkdir(parents=True, exist_ok=True)
0071         path.write_text(json.dumps(metrics, indent=2), encoding="utf-8")
0072     return metrics

```

Line	Code signal	What it is doing / what to verify
67	}	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
68	if output_path is not None:	Branches only when this condition is true; verify both the true and false paths in tests.
69	path = Path(output_path)	Creates or updates `path`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
70	path.parent.mkdir(parents=True, exist_ok=True)	Creates or updates `path.parent.mkdir(parents)`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

71	<code>path.write_text(json.dumps(metrics, indent=2), encoding="utf-8")</code>	Persists an artifact. The surrounding code should include run identity, schema and preprocessing lineage.
72	<code>return metrics</code>	Returns the result to the caller; this is the function output boundary that tests should assert.

src/apexsim/models/__init__.py

Supports the APEX package behind an explicit module boundary.

Trace callers, inputs, outputs and side effects before editing.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-1

```
0001 """World-model and baseline architectures."""
```

Line	Code signal	What it is doing / what to verify
1	"""World-model and baseline architectures."""	Begins or ends documentation describing the module, class or function contract.

src/apexsim/models/baselines.py

Provides simple controls that expose leakage and unnecessary complexity.

A world model should not be promoted until it beats persistence and linear dynamics on the exact deployment protocol.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```

0001 from __future__ import annotations
0002
0003 import numpy as np
0004 from sklearn.linear_model import Ridge
0005 from sklearn.multioutput import MultiOutputRegressor
0006
0007
0008 class PersistenceBaseline:
0009     """Predict that the next state is identical to the current state."""
0010
0011     def predict(self, current_state: np.ndarray, horizon: int) -> np.ndarray:
0012         return np.repeat(current_state[None, :], horizon, axis=0)
0013
0014
0015 class LinearTransitionBaseline:
0016     """Ridge regression for one-step state transitions."""
0017
0018     def __init__(self, alpha: float = 1.0) -> None:
0019         self.model = MultiOutputRegressor(Ridge(alpha=alpha))
0020
0021     def fit(self, inputs: np.ndarray, next_states: np.ndarray) -> "LinearTransitionBaseline":
0022         self.model.fit(inputs, next_states)

```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	import numpy as np	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
4	from sklearn.linear_model import Ridge	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
5	from sklearn.multioutput import MultiOutputRegressor	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
8	class PersistenceBaseline:	Defines a type that owns related state and behaviour behind a named interface.
9	"""Predict that the next state is identical to the current state."""	Begins or ends documentation describing the module, class or function contract.
11	def predict(self, current_state: np.ndarray, horizon: int) -> np.ndarray:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
12	return np.repeat(current_state[None, :], horizon, axis=0)	Returns the result to the caller; this is the function output boundary that tests should assert.

15	class LinearTransitionBaseline:	Defines a type that owns related state and behaviour behind a named interface.
16	"""Ridge regression for one-step state transitions."""	Begins or ends documentation describing the module, class or function contract.
18	def __init__(self, alpha: float = 1.0) -> None:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
19	self.model = MultiOutputRegressor(Ridge(alpha=alpha))	Creates or updates `self.model`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
21	def fit(self, inputs: np.ndarray, next_states: np.ndarray) -> "LinearTransitionB	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
22	self.model.fit(inputs, next_states)	Learns parameters or preprocessing statistics from the supplied training evidence only.

Lines 23-26

```

0023         return self
0024
0025     def predict(self, inputs: np.ndarray) -> np.ndarray:
0026         return self.model.predict(inputs)

```

Line	Code signal	What it is doing / what to verify
23	return self	Returns the result to the caller; this is the function output boundary that tests should assert.
25	def predict(self, inputs: np.ndarray) -> np.ndarray:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
26	return self.model.predict(inputs)	Returns the result to the caller; this is the function output boundary that tests should assert.

src/apexsim/models/factory.py

Creates the configured model behind one common interface.

The factory isolates model selection so pipeline and evaluation logic do not fork by architecture.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-18

```
0001 from __future__ import annotations
0002
0003 from torch import nn
0004
0005 from apexsim.config import ModelConfig
0006 from apexsim.models.gru_world_model import GRUWorldModel
0007 from apexsim.models.rssm import RSSMWorldModel
0008 from apexsim.models.ssm_world_model import SSMWorldModel
0009
0010
0011 def build_model(config: ModelConfig, input_dim: int, state_dim: int) -> nn.Module:
0012     if config.kind == "gru":
0013         return GRUWorldModel(input_dim, state_dim, config.hidden_dim, config.layers, config.dropout)
0014     if config.kind == "ssm":
0015         return SSMWorldModel(input_dim, state_dim, config.hidden_dim, config.layers)
0016     if config.kind == "rssm":
0017         return RSSMWorldModel(input_dim, state_dim, config.hidden_dim, config.latent_dim)
0018     raise ValueError(f"Model kind {config.kind!r} is not a neural world model")
```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	from torch import nn	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
5	from apexsim.config import ModelConfig	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
6	from apexsim.models.gru_world_model import GRUWorldModel	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
7	from apexsim.models.rssm import RSSMWorldModel	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
8	from apexsim.models.ssm_world_model import SSMWorldModel	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
11	def build_model(config: ModelConfig, input_dim: int, state_dim: int) -> nn.Module	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
12	if config.kind == "gru":	Branches only when this condition is true; verify both the true and false paths in tests.
13	return GRUWorldModel(input_dim, state_dim, config.hidden_dim, config.layers, con	Returns the result to the caller; this is the function output boundary that tests should assert.
14	if config.kind == "ssm":	Branches only when this condition is true;

Source → contract → runtime evidence → failure test

		verify both the true and false paths in tests.
15	return SSMWorldModel(input_dim, state_dim, config.hidden_dim, config.layers)	Returns the result to the caller; this is the function output boundary that tests should assert.
16	if config.kind == "rssm":	Branches only when this condition is true; verify both the true and false paths in tests.
17	return RSSMWorldModel(input_dim, state_dim, config.hidden_dim, config.latent_dim	Returns the result to the caller; this is the function output boundary that tests should assert.
18	raise ValueError(f"Model kind {config.kind! r} is not a neural world model")	Stops execution because an invariant or supported-condition check failed.

src/apexsim/models/gru_world_model.py

Implements deterministic recurrent history encoding and autoregressive rollout.

At each future step, known controls/context enter the transition while predicted state replaces the unknown future state.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```
0001 from __future__ import annotations
0002
0003 import torch
0004 from torch import nn
0005
0006
0007 class GRUWorldModel(nn.Module):
0008     """Deterministic sequence model that predicts future telemetry states.
0009
0010     The history encoder compresses the observed past into a hidden state. During rollout, the model
0011     consumes the planned future actions/context and its own previous predicted state.
0012     """
0013
0014     def __init__(
0015         self,
0016         input_dim: int,
0017         state_dim: int,
0018         hidden_dim: int = 64,
0019         layers: int = 1,
0020         dropout: float = 0.0,
0021     ) -> None:
0022         super().__init__()
```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	import torch	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
4	from torch import nn	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
7	class GRUWorldModel(nn.Module):	Defines a type that owns related state and behaviour behind a named interface.
8	"""Deterministic sequence model that predicts future telemetry states.	Begins or ends documentation describing the module, class or function contract.
10	The history encoder compresses the observed past into a hidden state. During rol	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
11	consumes the planned future actions/context and its own previous predicted state	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
12	"""	Begins or ends documentation describing the module, class or function contract.
14	def __init__(	Defines a callable boundary. Its parameters

		form the input contract and its return value or side effects form the output contract.
15	self,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
16	input_dim: int,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
17	state_dim: int,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
18	hidden_dim: int = 64,	Creates or updates `hidden_dim: int`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
19	layers: int = 1,	Creates or updates `layers: int`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
20	dropout: float = 0.0,	Creates or updates `dropout: float`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
21	) -> None:	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
22	super().__init__()	Calls a function or method. Trace its input contract, returned value and any persistent side effects.

Lines 23-44

```

0023     self.state_dim = state_dim
0024     self.input_dim = input_dim
0025     self.encoder = nn.GRU(
0026         input_dim,
0027         hidden_dim,
0028         num_layers=layers,
0029         batch_first=True,
0030         dropout=dropout if layers > 1 else 0.0,
0031     )
0032     self.transition = nn.GRUCell(input_dim, hidden_dim)
0033     self.decoder = nn.Sequential(
0034         nn.Linear(hidden_dim, hidden_dim),
0035         nn.SiLU(),
0036         nn.Linear(hidden_dim, state_dim),
0037     )
0038
0039     def forward(self, history: torch.Tensor, future_inputs: torch.Tensor) -> torch.Tensor:
0040         _, hidden = self.encoder(history)
0041         hidden_t = hidden[-1]
0042         predictions = []
0043         future_t = future_inputs.clone()
0044         previous_state = history[:, -1, : self.state_dim]

```

Line	Code signal	What it is doing / what to verify
23	self.state_dim = state_dim	Creates or updates `self.state_dim`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
24	self.input_dim = input_dim	Creates or updates `self.input_dim`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
25	self.encoder = nn.GRU(	Creates or updates `self.encoder`. Check its unit, shape, lifetime and whether it is

		derived only from information available at this point.
26	input_dim,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
27	hidden_dim,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
28	num_layers=layers,	Creates or updates `num_layers`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
29	batch_first=True,	Creates or updates `batch_first`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
30	dropout=dropout if layers > 1 else 0.0,	Creates or updates `dropout`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
31	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
32	self.transition = nn.GRUCell(input_dim, hidden_dim)	Creates or updates `self.transition`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
33	self.decoder = nn.Sequential(	Creates or updates `self.decoder`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
34	nn.Linear(hidden_dim, hidden_dim),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
35	nn.SiLU(),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
36	nn.Linear(hidden_dim, state_dim),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
37	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
39	def forward(self, history: torch.Tensor, future_inputs: torch.Tensor) -> torch.T	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
40	_, hidden = self.encoder(history)	Creates or updates `_, hidden`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
41	hidden_t = hidden[-1]	Creates or updates `hidden_t`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
42	predictions = []	Creates or updates `predictions`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
43	future_t = future_inputs.clone()	Creates or updates `future_t`. Check its unit, shape, lifetime and whether it is derived only from information available at this

		point.
44	<code>previous_state = history[:, -1, : self.state_dim]</code>	Creates or updates `previous_state`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 45-51

```

0045     for step in range(future_t.shape[1]):
0046         step_input = future_t[:, step].clone()
0047         step_input[:, : self.state_dim] = previous_state
0048         hidden_t = self.transition(step_input, hidden_t)
0049         previous_state = self.decoder(hidden_t)
0050         predictions.append(previous_state)
0051     return torch.stack(predictions, dim=1)

```

Line	Code signal	What it is doing / what to verify
45	<code>for step in range(future_t.shape[1]):</code>	Iterates over a collection or time dimension. Confirm order, boundaries and whether state should persist between iterations.
46	<code>step_input = future_t[:, step].clone()</code>	Creates or updates `step_input`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
47	<code>step_input[:, : self.state_dim] = previous_state</code>	Creates or updates `step_input[:, : self.state_dim]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
48	<code>hidden_t = self.transition(step_input, hidden_t)</code>	Creates or updates `hidden_t`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
49	<code>previous_state = self.decoder(hidden_t)</code>	Creates or updates `previous_state`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
50	<code>predictions.append(previous_state)</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
51	<code>return torch.stack(predictions, dim=1)</code>	Returns the result to the caller; this is the function output boundary that tests should assert.

src/apexsim/models/rssm.py

Implements deterministic memory plus stochastic prior/posterior latent state.

Training may use observations through the posterior; imagination must proceed through the prior without future evidence.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```

0001 from __future__ import annotations
0002
0003 import torch
0004 from torch import nn
0005 from torch.distributions import Normal, kl_divergence
0006
0007
0008 class RSSMWorldModel(nn.Module):
0009     """Compact Dreamer-style recurrent state-space model for telemetry.
0010
0011     h_t is deterministic memory. z_t is a stochastic latent state. The posterior sees the observed
0012     state during training; the prior is used for imagination when future observations are unavailable.
0013     """
0014
0015     def __init__(self, input_dim: int, state_dim: int, hidden_dim: int = 64, latent_dim: int = 12) -> None:
0016         super().__init__()
0017         self.input_dim = input_dim
0018         self.state_dim = state_dim
0019         self.hidden_dim = hidden_dim
0020         self.latent_dim = latent_dim
0021         self.action_context_dim = input_dim - state_dim
0022         self.gru = nn.GRUCell(latent_dim + self.action_context_dim, hidden_dim)

```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	import torch	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
4	from torch import nn	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
5	from torch.distributions import Normal, kl_divergence	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
8	class RSSMWorldModel(nn.Module):	Defines a type that owns related state and behaviour behind a named interface.
9	"""Compact Dreamer-style recurrent state-space model for telemetry.	Begins or ends documentation describing the module, class or function contract.
11	h_t is deterministic memory. z_t is a stochastic latent state. The posterior see	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
12	state during training; the prior is used for imagination when future observation	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

13	"""	Begins or ends documentation describing the module, class or function contract.
15	def __init__(self, input_dim: int, state_dim: int, hidden_dim: int = 64, latent_dim: int = 64):	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
16	super().__init__()	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
17	self.input_dim = input_dim	Creates or updates `self.input_dim`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
18	self.state_dim = state_dim	Creates or updates `self.state_dim`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
19	self.hidden_dim = hidden_dim	Creates or updates `self.hidden_dim`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
20	self.latent_dim = latent_dim	Creates or updates `self.latent_dim`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
21	self.action_context_dim = input_dim - state_dim	Creates or updates `self.action_context_dim`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
22	self.gru = nn.GRUCell(latent_dim + self.action_context_dim, hidden_dim)	Creates or updates `self.gru`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 23-44

```

0023     self.prior = nn.Linear(hidden_dim, 2 * latent_dim)
0024     self.posterior = nn.Linear(hidden_dim + state_dim, 2 * latent_dim)
0025     self.decoder = nn.Sequential(
0026         nn.Linear(hidden_dim + latent_dim, hidden_dim),
0027         nn.SiLU(),
0028         nn.Linear(hidden_dim, state_dim),
0029     )
0030
0031     @staticmethod
0032     def _distribution(parameters: torch.Tensor) -> Normal:
0033         mean, raw_std = parameters.chunk(2, dim=-1)
0034         std = torch.nn.functional.softplus(raw_std) + 0.1
0035         return Normal(mean, std)
0036
0037     def forward(
0038         self,
0039         history: torch.Tensor,
0040         future_inputs: torch.Tensor,
0041         future_targets: torch.Tensor | None = None,
0042     ) -> tuple[torch.Tensor, torch.Tensor]:
0043         batch = history.shape[0]
0044         h = history.new_zeros((batch, self.hidden_dim))

```

Line	Code signal	What it is doing / what to verify
23	self.prior = nn.Linear(hidden_dim, 2 * latent_dim)	Creates or updates `self.prior`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
24	self.posterior = nn.Linear(hidden_dim + state_dim, 2 * latent_dim)	Creates or updates `self.posterior`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

		this point.
25	self.decoder = nn.Sequential(	Creates or updates `self.decoder`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
26	nn.Linear(hidden_dim + latent_dim, hidden_dim),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
27	nn.SiLU(),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
28	nn.Linear(hidden_dim, state_dim),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
29	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
31	@staticmethod	Applies a decorator that changes registration, validation or runtime behaviour of the definition that follows.
32	def _distribution(parameters: torch.Tensor) -> Normal:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
33	mean, raw_std = parameters.chunk(2, dim=-1)	Creates or updates `mean, raw_std`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
34	std = torch.nn.functional.softplus(raw_std) + 0.1	Creates or updates `std`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
35	return Normal(mean, std)	Returns the result to the caller; this is the function output boundary that tests should assert.
37	def forward(	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
38	self,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
39	history: torch.Tensor,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
40	future_inputs: torch.Tensor,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
41	future_targets: torch.Tensor None = None,	Creates or updates `future_targets: torch.Tensor   None`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
42	) -> tuple[torch.Tensor, torch.Tensor]:	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
43	batch = history.shape[0]	Creates or updates `batch`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
44	h = history.new_zeros((batch, self.hidden_dim))	Creates or updates `h`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 45-66

```

0045     z = history.new_zeros((batch, self.latent_dim))
0046     # Observe history to initialize belief state.
0047     for step in range(history.shape[1]):
0048         state = history[:, step, : self.state_dim]
0049         action_context = history[:, step, self.state_dim :]
0050         h = self.gru(torch.cat([z, action_context], dim=-1), h)
0051         posterior = self._distribution(self.posterior(torch.cat([h, state], dim=-1)))
0052         z = posterior.rsample()
0053
0054     outputs = []
0055     kl_terms = []
0056     previous_state = history[:, -1, : self.state_dim]
0057     for step in range(future_inputs.shape[1]):
0058         action_context = future_inputs[:, step, self.state_dim :]
0059         h = self.gru(torch.cat([z, action_context], dim=-1), h)
0060         prior = self._distribution(self.prior(h))
0061         if future_targets is not None:
0062             posterior = self._distribution(
0063                 self.posterior(torch.cat([h, future_targets[:, step]], dim=-1))
0064             )
0065         z = posterior.rsample()
0066         kl_terms.append(kl_divergence(posterior, prior).sum(dim=-1))

```

Line	Code signal	What it is doing / what to verify
45	<code>z = history.new_zeros((batch, self.latent_dim))</code>	Creates or updates `z`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
46	<code># Observe history to initialize belief state.</code>	Documents an assumption, intent or constraint that should remain aligned with the implementation.
47	<code>for step in range(history.shape[1]):</code>	Iterates over a collection or time dimension. Confirm order, boundaries and whether state should persist between iterations.
48	<code>state = history[:, step, : self.state_dim]</code>	Creates or updates `state`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
49	<code>action_context = history[:, step, self.state_dim :]</code>	Creates or updates `action_context`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
50	<code>h = self.gru(torch.cat([z, action_context], dim=-1), h)</code>	Concatenates tensors or arrays; verify that every non-concatenated axis has identical meaning and size.
51	<code>posterior = self._distribution(self.posterior(torch.cat([h, state], dim=-1)))</code>	Concatenates tensors or arrays; verify that every non-concatenated axis has identical meaning and size.
52	<code>z = posterior.rsample()</code>	Creates or updates `z`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
54	<code>outputs = []</code>	Creates or updates `outputs`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
55	<code>kl_terms = []</code>	Creates or updates `kl_terms`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
56	<code>previous_state = history[:, -1, : self.state_dim]</code>	Creates or updates `previous_state`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
57	<code>for step in range(future_inputs.shape[1]):</code>	Iterates over a collection or time dimension. Confirm order, boundaries and whether

		state should persist between iterations.
58	<code>action_context = future_inputs[:, step, self.state_dim :]</code>	Creates or updates `action_context`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
59	<code>h = self.gru(torch.cat([z, action_context], dim=-1), h)</code>	Concatenates tensors or arrays; verify that every non-concatenated axis has identical meaning and size.
60	<code>prior = self._distribution(self.prior(h))</code>	Creates or updates `prior`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
61	<code>if future_targets is not None:</code>	Branches only when this condition is true; verify both the true and false paths in tests.
62	<code>posterior = self._distribution(</code>	Creates or updates `posterior`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
63	<code>self.posterior(torch.cat([h, future_targets[:, step]], dim=-1))</code>	Concatenates tensors or arrays; verify that every non-concatenated axis has identical meaning and size.
64	<code>)</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
65	<code>z = posterior.rsample()</code>	Creates or updates `z`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
66	<code>kl_terms.append(kl_divergence(posterior, prior).sum(dim=-1))</code>	Creates or updates `kl_terms.append(kl_divergence(posterior, prior).sum(dim=-1)`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 67-73

```

0067         else:
0068             z = prior.rsample()
0069             previous_state = self.decoder(torch.cat([h, z], dim=-1))
0070             outputs.append(previous_state)
0071         prediction = torch.stack(outputs, dim=1)
0072         kl = torch.stack(kl_terms, dim=1).mean() if kl_terms else prediction.new_tensor(0.0)
0073         return prediction, kl

```

Line	Code signal	What it is doing / what to verify
67	<code>else:</code>	Handles all remaining cases; inspect whether this fallback is intentionally broad.
68	<code>z = prior.rsample()</code>	Creates or updates `z`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
69	<code>previous_state = self.decoder(torch.cat([h, z], dim=-1))</code>	Concatenates tensors or arrays; verify that every non-concatenated axis has identical meaning and size.
70	<code>outputs.append(previous_state)</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
71	<code>prediction = torch.stack(outputs, dim=1)</code>	Creates a new axis by stacking aligned values; document what the new axis represents.
72	<code>kl = torch.stack(kl_terms, dim=1).mean() if kl_terms else prediction.new_tensor(</code>	Creates a new axis by stacking aligned values; document what the new axis represents.
73	<code>return prediction, kl</code>	Returns the result to the caller; this is the

		function output boundary that tests should assert.
--	--	--

src/apexsim/models/ssm_world_model.py

Implements an educational stable selective state-space challenger.

It demonstrates content-dependent memory but does not claim full optimized Mamba fidelity.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```
0001 from __future__ import annotations
0002
0003 import torch
0004 from torch import nn
0005
0006
0007 class SelectiveSSMCell(nn.Module):
0008     """Small educational selective state-space cell.
0009
0010     This is intentionally not a full Mamba kernel. It demonstrates the central idea: a persistent
0011     state with input-dependent write/forget gates and stable diagonal decay.
0012     """
0013
0014     def __init__(self, input_dim: int, hidden_dim: int) -> None:
0015         super().__init__()
0016         self.logit_decay = nn.Parameter(torch.zeros(hidden_dim))
0017         self.input_projection = nn.Linear(input_dim, hidden_dim)
0018         self.gate_projection = nn.Linear(input_dim, hidden_dim)
0019
0020     def forward(self, x: torch.Tensor, state: torch.Tensor) -> torch.Tensor:
0021         decay = torch.sigmoid(self.logit_decay).unsqueeze(0)
0022         candidate = torch.tanh(self.gate_projection(x))
```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	import torch	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
4	from torch import nn	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
7	class SelectiveSSMCell(nn.Module):	Defines a type that owns related state and behaviour behind a named interface.
8	"""Small educational selective state-space cell.	Begins or ends documentation describing the module, class or function contract.
10	This is intentionally not a full Mamba kernel. It demonstrates the central idea:	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
11	state with input-dependent write/forget gates and stable diagonal decay.	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
12	"""	Begins or ends documentation describing the module, class or function contract.
14	def __init__(self, input_dim: int, hidden_dim: int) -> None:	Defines a callable boundary. Its parameters form the input contract and its return value

		or side effects form the output contract.
15	<code>super().__init__()</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
16	<code>self.logit_decay = nn.Parameter(torch.zeros(hidden_dim))</code>	Creates or updates <code>'self.logit_decay'</code> . Check its unit, shape, lifetime and whether it is derived only from information available at this point.
17	<code>self.input_projection = nn.Linear(input_dim, hidden_dim)</code>	Creates or updates <code>'self.input_projection'</code> . Check its unit, shape, lifetime and whether it is derived only from information available at this point.
18	<code>self.gate_projection = nn.Linear(input_dim, hidden_dim)</code>	Creates or updates <code>'self.gate_projection'</code> . Check its unit, shape, lifetime and whether it is derived only from information available at this point.
20	<code>def forward(self, x: torch.Tensor, state: torch.Tensor) -> torch.Tensor:</code>	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
21	<code>decay = torch.sigmoid(self.logit_decay).unsqueeze(0)</code>	Maps logits to values between zero and one; interpret them as probabilities only after calibration evidence.
22	<code>candidate = torch.tanh(self.input_projection(x))</code>	Creates or updates <code>'candidate'</code> . Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 23-44

```

0023     gate = torch.sigmoid(self.gate_projection(x))
0024     return decay * state + (1.0 - decay) * gate * candidate
0025
0026
0027 class SSMWorldModel(nn.Module):
0028     def __init__(self, input_dim: int, state_dim: int, hidden_dim: int = 64, layers: int = 2) -> None:
0029         super().__init__()
0030         self.state_dim = state_dim
0031         self.cells = nn.ModuleList(
0032             [SelectiveSSMCell(input_dim if i == 0 else hidden_dim, hidden_dim) for i in range(layers)]
0033         )
0034         self.decoder = nn.Sequential(nn.LayerNorm(hidden_dim), nn.Linear(hidden_dim, state_dim))
0035
0036     def _step(self, x: torch.Tensor, states: list[torch.Tensor]) -> tuple[torch.Tensor, list[torch.Tensor]]:
0037         updated: list[torch.Tensor] = []
0038         value = x
0039         for cell, state in zip(self.cells, states):
0040             state = cell(value, state)
0041             updated.append(state)
0042             value = state
0043         return self.decoder(value), updated
0044

```

Line	Code signal	What it is doing / what to verify
23	<code>gate = torch.sigmoid(self.gate_projection(x))</code>	Maps logits to values between zero and one; interpret them as probabilities only after calibration evidence.
24	<code>return decay * state + (1.0 - decay) * gate * candidate</code>	Returns the result to the caller; this is the function output boundary that tests should assert.
27	<code>class SSMWorldModel(nn.Module):</code>	Defines a type that owns related state and behaviour behind a named interface.
28	<code>def __init__(self, input_dim: int, state_dim: int, hidden_dim: int = 64, layers:</code>	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
29	<code>super().__init__()</code>	Calls a function or method. Trace its input

		contract, returned value and any persistent side effects.
30	<code>self.state_dim = state_dim</code>	Creates or updates `self.state_dim`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
31	<code>self.cells = nn.ModuleList(</code>	Creates or updates `self.cells`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
32	<code>[SelectiveSSMCell(input_dim if i == 0 else hidden_dim, hidden_dim) for i in rang</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
33	<code>)</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
34	<code>self.decoder = nn.Sequential(nn.LayerNorm(hidden_dim), nn.Linear(hidden_dim, sta</code>	Creates or updates `self.decoder`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
36	<code>def _step(self, x: torch.Tensor, states: list[torch.Tensor]) -> tuple[torch.Tens</code>	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
37	<code>updated: list[torch.Tensor] = []</code>	Creates or updates `updated: list[torch.Tensor]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
38	<code>value = x</code>	Creates or updates `value`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
39	<code>for cell, state in zip(self.cells, states):</code>	Iterates over a collection or time dimension. Confirm order, boundaries and whether state should persist between iterations.
40	<code>state = cell(value, state)</code>	Creates or updates `state`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
41	<code>updated.append(state)</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
42	<code>value = state</code>	Creates or updates `value`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
43	<code>return self.decoder(value), updated</code>	Returns the result to the caller; this is the function output boundary that tests should assert.

Lines 45-58

```

0045     def forward(self, history: torch.Tensor, future_inputs: torch.Tensor) -> torch.Tensor:
0046         batch = history.shape[0]
0047         hidden_dim = self.cells[0].logit_decay.shape[0]
0048         states = [history.new_zeros((batch, hidden_dim)) for _ in self.cells]
0049         for step in range(history.shape[1]):
0050             _, states = self._step(history[:, step], states)
0051             previous_state = history[:, -1, : self.state_dim]
0052             outputs = []
0053             for step in range(future_inputs.shape[1]):
0054                 x = future_inputs[:, step].clone()
0055                 x[:, : self.state_dim] = previous_state
0056                 previous_state, states = self._step(x, states)
0057                 outputs.append(previous_state)

```

Source → contract → runtime evidence → failure test

0058 return torch.stack(outputs, dim=1)

Line	Code signal	What it is doing / what to verify
45	def forward(self, history: torch.Tensor, future_inputs: torch.Tensor) -> torch.T	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
46	batch = history.shape[0]	Creates or updates `batch`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
47	hidden_dim = self.cells[0].logit_decay.shape[0]	Creates or updates `hidden_dim`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
48	states = [history.new_zeros((batch, hidden_dim)) for _ in self.cells]	Creates or updates `states`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
49	for step in range(history.shape[1]):	Iterates over a collection or time dimension. Confirm order, boundaries and whether state should persist between iterations.
50	_, states = self._step(history[:, step], states)	Creates or updates `_, states`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
51	previous_state = history[:, -1, : self.state_dim]	Creates or updates `previous_state`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
52	outputs = []	Creates or updates `outputs`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
53	for step in range(future_inputs.shape[1]):	Iterates over a collection or time dimension. Confirm order, boundaries and whether state should persist between iterations.
54	x = future_inputs[:, step].clone()	Creates or updates `x`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
55	x[:, : self.state_dim] = previous_state	Creates or updates `x[:, : self.state_dim]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
56	previous_state, states = self._step(x, states)	Creates or updates `previous_state, states`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
57	outputs.append(previous_state)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
58	return torch.stack(outputs, dim=1)	Returns the result to the caller; this is the function output boundary that tests should assert.

src/apexsim/pipeline/__init__.py

Supports the APEX package behind an explicit module boundary.

Trace callers, inputs, outputs and side effects before editing.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-1

```
0001 """Reusable idempotent pipeline stages and local orchestration."""
```

Line	Code signal	What it is doing / what to verify
1	"""Reusable idempotent pipeline stages and local orchestration."""	Begins or ends documentation describing the module, class or function contract.

src/apexsim/pipeline/runner.py

Coordinates run lifecycle and stage transitions.

The runner owns ordering, failure state and summary publication while stage functions own domain logic.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```
0001 from __future__ import annotations
0002
0003 import json
0004 from datetime import datetime, timezone
0005 from pathlib import Path
0006
0007 from apexsim.config import ProjectConfig
0008 from apexsim.pipeline.stages import (
0009     ablation_stage,
0010     dataset_stage,
0011     evaluate_stage,
0012     ingest_stage,
0013     publish_stage,
0014     quality_stage,
0015     train_stage,
0016 )
0017 from apexsim.registry import RunRegistry
0018
0019
0020 def run_pipeline(config: ProjectConfig, run_id: str) -> dict:
0021     run_dir = config.artifacts_dir / run_id
0022     run_dir.mkdir(parents=True, exist_ok=True)
```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	import json	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
4	from datetime import datetime, timezone	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
5	from pathlib import Path	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
7	from apexsim.config import ProjectConfig	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
8	from apexsim.pipeline.stages import (	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
9	ablation_stage,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
10	dataset_stage,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

11	evaluate_stage,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
12	ingest_stage,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
13	publish_stage,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
14	quality_stage,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
15	train_stage,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
16	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
17	from apexsim.registry import RunRegistry	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
20	def run_pipeline(config: ProjectConfig, run_id: str) -> dict:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
21	run_dir = config.artifacts_dir / run_id	Creates or updates `run_dir`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
22	run_dir.mkdir(parents=True, exist_ok=True)	Creates or updates `run_dir.mkdir(parents)`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 23-44

```

0023 registry = RunRegistry(config.artifacts_dir / "runs.sqlite")
0024 registry.start(run_id, config.model.kind, str(run_dir))
0025 started = datetime.now(timezone.utc).isoformat()
0026 try:
0027     canonical = ingest_stage(config, run_dir)
0028     quality = quality_stage(canonical, run_dir)
0029     dataset = dataset_stage(config, canonical, run_dir)
0030     model = train_stage(config, canonical, run_dir)
0031     metrics = evaluate_stage(config, canonical, run_dir)
0032     ablations = ablation_stage(config, canonical, run_dir)
0033     publication = publish_stage(config, canonical, run_dir)
0034     summary = {
0035         "run_id": run_id,
0036         "status": "succeeded",
0037         "started_at": started,
0038         "finished_at": datetime.now(timezone.utc).isoformat(),
0039         "canonical_data": str(canonical),
0040         "quality": quality,
0041         "splits": dataset["splits"],
0042         "model": model,
0043         "metrics": metrics,
0044         "best_ablation": ablations[0],

```

Line	Code signal	What it is doing / what to verify
23	registry = RunRegistry(config.artifacts_dir / "runs.sqlite")	Creates or updates `registry`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
24	registry.start(run_id, config.model.kind, str(run_dir))	Calls a function or method. Trace its input contract, returned value and any persistent

		side effects.
25	<code>started = datetime.now(timezone.utc).isoformat()</code>	Creates or updates `started`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
26	<code>try:</code>	Begins an operation that may fail for an expected reason.
27	<code>canonical = ingest_stage(config, run_dir)</code>	Creates or updates `canonical`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
28	<code>quality = quality_stage(canonical, run_dir)</code>	Creates or updates `quality`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
29	<code>dataset = dataset_stage(config, canonical, run_dir)</code>	Creates or updates `dataset`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
30	<code>model = train_stage(config, canonical, run_dir)</code>	Creates or updates `model`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
31	<code>metrics = evaluate_stage(config, canonical, run_dir)</code>	Creates or updates `metrics`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
32	<code>ablations = ablation_stage(config, canonical, run_dir)</code>	Creates or updates `ablations`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
33	<code>publication = publish_stage(config, canonical, run_dir)</code>	Creates or updates `publication`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
34	<code>summary = {</code>	Creates or updates `summary`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
35	<code>"run_id": run_id,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
36	<code>"status": "succeeded",</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
37	<code>"started_at": started,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
38	<code>"finished_at": datetime.now(timezone.utc).isoformat(),</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
39	<code>"canonical_data": str(canonical),</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
40	<code>"quality": quality,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
41	<code>"splits": dataset["splits"],</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
42	<code>"model": model,</code>	Executes this statement inside the current

		control-flow block; connect it to the nearest function contract and testable invariant.
43	"metrics": metrics,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
44	"best_ablation": ablations[0],	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

Lines 45-52

```

0045     "publication": publication,
0046     }
0047     (run_dir / "summary.json").write_text(json.dumps(summary, indent=2), encoding="utf-8")
0048     registry.finish(run_id, "succeeded", metrics)
0049     return summary
0050 except Exception:
0051     registry.finish(run_id, "failed")
0052     raise

```

Line	Code signal	What it is doing / what to verify
45	"publication": publication,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
46	}	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
47	(run_dir / "summary.json").write_text(json.dumps(summary, indent=2), encoding="u	Persists an artifact. The surrounding code should include run identity, schema and preprocessing lineage.
48	registry.finish(run_id, "succeeded", metrics)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
49	return summary	Returns the result to the caller; this is the function output boundary that tests should assert.
50	except Exception:	Handles a specific failure mode. The handler should preserve useful diagnostic evidence.
51	registry.finish(run_id, "failed")	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
52	raise	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

src/apexsim/pipeline/stages.py

Defines reusable idempotent pipeline operations.

Each stage consumes paths/contracts and writes immutable evidence that can be retried independently.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```
0001 from __future__ import annotations
0002
0003 import json
0004 import shutil
0005 from pathlib import Path
0006
0007 import joblib
0008 import numpy as np
0009 import pandas as pd
0010 import torch
0011 from torch.utils.data import DataLoader
0012
0013 from apexsim.config import ProjectConfig
0014 from apexsim.contracts import MODEL_INPUT_COLUMNS, STATE_COLUMNS, TARGET_COLUMNS
0015 from apexsim.data.features import Standardizer
0016 from apexsim.data.synthetic import generate_synthetic_sessions
0017 from apexsim.data.validate import validate_canonical_frame
0018 from apexsim.data.windows import TelemetryWindowDataset, split_sessions
0019 from apexsim.evaluation import evaluate_world_model
0020 from apexsim.models.factory import build_model
0021 from apexsim.training import train_world_model
0022
```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	import json	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
4	import shutil	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
5	from pathlib import Path	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
7	import joblib	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
8	import numpy as np	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
9	import pandas as pd	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
10	import torch	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.

Source → contract → runtime evidence → failure test

11	from torch.utils.data import DataLoader	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
13	from apexsim.config import ProjectConfig	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
14	from apexsim.contracts import MODEL_INPUT_COLUMNS, STATE_COLUMNS, TARGET_COLUMNS	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
15	from apexsim.data.features import Standardizer	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
16	from apexsim.data.synthetic import generate_synthetic_sessions	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
17	from apexsim.data.validate import validate_canonical_frame	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
18	from apexsim.data.windows import TelemetryWindowDataset, split_sessions	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
19	from apexsim.evaluation import evaluate_world_model	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
20	from apexsim.models.factory import build_model	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
21	from apexsim.training import train_world_model	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.

Lines 23-44

```

0023
0024 def ingest_stage(config: ProjectConfig, run_dir: Path) -> Path:
0025     """Materialize one canonical telemetry file inside the immutable run directory.
0026
0027     Synthetic runs generate the file directly. Real-data runs first use a source
0028     adapter (FastF1 or OpenF1) to create the canonical contract, then provide that
0029     file through ``canonical_input_path``. Copying it into the run directory makes
0030     the exact training input part of run lineage and keeps downstream stages
0031     source-independent.
0032     """
0033     output = run_dir / "canonical_telemetry.csv"
0034     if output.exists():
0035         return output
0036
0037     if config.data.canonical_input_path is not None:
0038         source = Path(config.data.canonical_input_path)
0039         if not source.exists():
0040             raise FileNotFoundError(f"Canonical input does not exist: {source}")
0041         if source.resolve() != output.resolve():
0042             shutil.copy2(source, output)
0043         return output
0044

```

Line	Code signal	What it is doing / what to verify
24	def ingest_stage(config: ProjectConfig, run_dir: Path) -> Path:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
25	"""Materialize one canonical telemetry file inside the immutable run directory.	Begins or ends documentation describing the module, class or function contract.
27	Synthetic runs generate the file directly. Real-data runs first use a source	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

28	adapter (FastF1 or OpenF1) to create the canonical contract, then provide that	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
29	file through ``canonical_input_path``. Copying it into the run directory makes	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
30	the exact training input part of run lineage and keeps downstream stages	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
31	source-independent.	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
32	"""	Begins or ends documentation describing the module, class or function contract.
33	output = run_dir / "canonical_telemetry.csv"	Creates or updates `output`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
34	if output.exists():	Branches only when this condition is true; verify both the true and false paths in tests.
35	return output	Returns the result to the caller; this is the function output boundary that tests should assert.
37	if config.data.canonical_input_path is not None:	Branches only when this condition is true; verify both the true and false paths in tests.
38	source = Path(config.data.canonical_input_path)	Creates or updates `source`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
39	if not source.exists():	Branches only when this condition is true; verify both the true and false paths in tests.
40	raise FileNotFoundError(f"Canonical input does not exist: {source}")	Stops execution because an invariant or supported-condition check failed.
41	if source.resolve() != output.resolve():	Branches only when this condition is true; verify both the true and false paths in tests.
42	shutil.copy2(source, output)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
43	return output	Returns the result to the caller; this is the function output boundary that tests should assert.

Lines 45-66

```

0045     if config.data.source == "synthetic":
0046         generate_synthetic_sessions(config, output)
0047         return output
0048
0049     raise ValueError(
0050         "FastF1/OpenF1 sources require a canonical input file. Run the matching "
0051         "ingestion command, then use `apexsim run-canonical --input-path ...`."
0052     )
0053
0054
0055 def quality_stage(canonical_path: Path, run_dir: Path) -> dict:
0056     output = run_dir / "quality_report.json"
0057     if output.exists():
0058         return json.loads(output.read_text(encoding="utf-8"))
0059     frame = pd.read_csv(canonical_path)
0060     report = validate_canonical_frame(frame, strict=True).to_dict()
0061     output.write_text(json.dumps(report, indent=2), encoding="utf-8")
0062     return report
0063
0064

```

```

0065 def dataset_stage(config: ProjectConfig, canonical_path: Path, run_dir: Path) -> dict:
0066     split_path = run_dir / "splits.json"

```

Line	Code signal	What it is doing / what to verify
45	if config.data.source == "synthetic":	Branches only when this condition is true; verify both the true and false paths in tests.
46	generate_synthetic_sessions(config, output)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
47	return output	Returns the result to the caller; this is the function output boundary that tests should assert.
49	raise ValueError(	Stops execution because an invariant or supported-condition check failed.
50	"FastF1/OpenF1 sources require a canonical input file. Run the matching "	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
51	"ingestion command, then use `apexsim run-canonical -input-path ...`."	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
52	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
55	def quality_stage(canonical_path: Path, run_dir: Path) -> dict:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
56	output = run_dir / "quality_report.json"	Creates or updates `output`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
57	if output.exists():	Branches only when this condition is true; verify both the true and false paths in tests.
58	return json.loads(output.read_text(encoding="utf-8"))	Returns the result to the caller; this is the function output boundary that tests should assert.
59	frame = pd.read_csv(canonical_path)	Loads persisted tabular evidence; validate schema, units and ordering immediately afterward.
60	report = validate_canonical_frame(frame, strict=True).to_dict()	Creates or updates `report`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
61	output.write_text(json.dumps(report, indent=2), encoding="utf-8")	Persists an artifact. The surrounding code should include run identity, schema and preprocessing lineage.
62	return report	Returns the result to the caller; this is the function output boundary that tests should assert.
65	def dataset_stage(config: ProjectConfig, canonical_path: Path, run_dir: Path) ->	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
66	split_path = run_dir / "splits.json"	Creates or updates `split_path`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 67-88

```

0067     scaler_path = run_dir / "standardizer.json"
0068     if split_path.exists() and scaler_path.exists():
0069         return {
0070             "splits": json.loads(split_path.read_text(encoding="utf-8")),

```

Source → contract → runtime evidence → failure test

```

0071         "standardizer": json.loads(scaler_path.read_text(encoding="utf-8")),
0072     }
0073     frame = pd.read_csv(canonical_path)
0074     splits = split_sessions(
0075         frame,
0076         config.data.train_fraction,
0077         config.data.val_fraction,
0078         config.seed,
0079     )
0080     train_frame = frame[frame.session_id.isin(splits["train"])]
0081     standardizer = Standardizer.fit(train_frame)
0082     split_path.write_text(json.dumps(splits, indent=2), encoding="utf-8")
0083     scaler_path.write_text(json.dumps(standardizer.to_dict(), indent=2), encoding="utf-8")
0084     return {"splits": splits, "standardizer": standardizer.to_dict()}
0085
0086
0087 def _load_datasets(config: ProjectConfig, canonical_path: Path, run_dir: Path):
0088     frame = pd.read_csv(canonical_path)

```

Line	Code signal	What it is doing / what to verify
67	scaler_path = run_dir / "standardizer.json"	Creates or updates `scaler_path`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
68	if split_path.exists() and scaler_path.exists():	Branches only when this condition is true; verify both the true and false paths in tests.
69	return {	Returns the result to the caller; this is the function output boundary that tests should assert.
70	"splits": json.loads(split_path.read_text(encoding="utf-8")),	Creates or updates ``splits``: json.loads(split_path.read_text(encoding=`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
71	"standardizer": json.loads(scaler_path.read_text(encoding="utf-8")),	Creates or updates ``standardizer``: json.loads(scaler_path.read_text(encoding=`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
72	}	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
73	frame = pd.read_csv(canonical_path)	Loads persisted tabular evidence; validate schema, units and ordering immediately afterward.
74	splits = split_sessions(	Creates or updates `splits`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
75	frame,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
76	config.data.train_fraction,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
77	config.data.val_fraction,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
78	config.seed,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
79	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
80	train_frame =	Creates or updates `train_frame`. Check its

	<code>frame[frame.session_id.isin(splits["train"])]</code>	unit, shape, lifetime and whether it is derived only from information available at this point.
81	<code>standardizer = Standardizer.fit(train_frame)</code>	Learns parameters or preprocessing statistics from the supplied training evidence only.
82	<code>split_path.write_text(json.dumps(splits, indent=2), encoding="utf-8")</code>	Persists an artifact. The surrounding code should include run identity, schema and preprocessing lineage.
83	<code>scaler_path.write_text(json.dumps(standardizer.to_dict(), indent=2), encoding="u</code>	Persists an artifact. The surrounding code should include run identity, schema and preprocessing lineage.
84	<code>return {"splits": splits, "standardizer": standardizer.to_dict()}</code>	Returns the result to the caller; this is the function output boundary that tests should assert.
87	<code>def load_datasets(config: ProjectConfig, canonical_path: Path, run_dir: Path):</code>	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
88	<code>frame = pd.read_csv(canonical_path)</code>	Loads persisted tabular evidence; validate schema, units and ordering immediately afterward.

Lines 89-110

```

0089     splits = json.loads((run_dir / "splits.json").read_text(encoding="utf-8"))
0090     standardizer = Standardizer.from_dict(
0091         json.loads((run_dir / "standardizer.json").read_text(encoding="utf-8"))
0092     )
0093     datasets = {
0094         name: TelemetryWindowDataset(
0095             frame,
0096             session_ids,
0097             config.data.sequence_length,
0098             config.data.prediction_horizon,
0099             standardizer,
0100             max_windows=config.training.max_train_windows if name == "train" else 1200,
0101         )
0102         for name, session_ids in splits.items()
0103     }
0104     return frame, standardizer, datasets
0105
0106
0107 def train_stage(config: ProjectConfig, canonical_path: Path, run_dir: Path) -> dict:
0108     model_dir = run_dir / "model"
0109     model_dir.mkdir(parents=True, exist_ok=True)
0110     checkpoint = model_dir / "world_model.pt"

```

Line	Code signal	What it is doing / what to verify
89	<code>splits = json.loads((run_dir / "splits.json").read_text(encoding="utf-8"))</code>	Creates or updates `splits`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
90	<code>standardizer = Standardizer.from_dict(</code>	Creates or updates `standardizer`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
91	<code>json.loads((run_dir / "standardizer.json").read_text(encoding="utf-8"))</code>	Creates or updates `json.loads((run_dir / "standardizer.json").read_text(encoding='`'. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
92	<code>)</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
93	<code>datasets = {</code>	Creates or updates `datasets`. Check its unit, shape, lifetime and whether it is derived

		only from information available at this point.
94	name: TelemetryWindowDataset(	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
95	frame,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
96	session_ids,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
97	config.data.sequence_length,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
98	config.data.prediction_horizon,	Runs model inference using learned parameters without fitting on the requested examples.
99	standardizer,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
100	max_windows=config.training.max_train_windows if name == "train" else 1200,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
101	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
102	for name, session_ids in splits.items()	Iterates over a collection or time dimension. Confirm order, boundaries and whether state should persist between iterations.
103	}	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
104	return frame, standardizer, datasets	Returns the result to the caller; this is the function output boundary that tests should assert.
107	def train_stage(config: ProjectConfig, canonical_path: Path, run_dir: Path) -> d	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
108	model_dir = run_dir / "model"	Creates or updates `model_dir`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
109	model_dir.mkdir(parents=True, exist_ok=True)	Creates or updates `model_dir.mkdir(parents)`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
110	checkpoint = model_dir / "world_model.pt"	Creates or updates `checkpoint`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 111-132

```

0111     meta_path = model_dir / "model_meta.json"
0112     if checkpoint.exists() and meta_path.exists():
0113         return json.loads(meta_path.read_text(encoding="utf-8"))
0114     _, standardizer, datasets = _load_datasets(config, canonical_path, run_dir)
0115     if len(datasets["train"]) == 0 or len(datasets["val"]) == 0:
0116         raise RuntimeError("Not enough windows after session split; increase session duration or count.")
0117     train_loader = DataLoader(datasets["train"], batch_size=config.training.batch_size, shuffle=True)
0118     val_loader = DataLoader(datasets["val"], batch_size=config.training.batch_size, shuffle=False)
0119     model = build_model(config.model, len(MODEL_INPUT_COLUMNS), len(TARGET_COLUMNS))

```

Source → contract → runtime evidence → failure test

```

0120     result = train_world_model(model, train_loader, val_loader, config, model_dir)
0121     meta = {
0122         "kind": config.model.kind,
0123         "input_dim": len(MODEL_INPUT_COLUMNS),
0124         "state_dim": len(TARGET_COLUMNS),
0125         "hidden_dim": config.model.hidden_dim,
0126         "latent_dim": config.model.latent_dim,
0127         "layers": config.model.layers,
0128         "dropout": config.model.dropout,
0129         "best_val_loss": result.best_val_loss,
0130         "epochs_completed": result.epochs_completed,
0131     }
0132     meta_path.write_text(json.dumps(meta, indent=2), encoding="utf-8")

```

Line	Code signal	What it is doing / what to verify
111	meta_path = model_dir / "model_meta.json"	Creates or updates `meta_path`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
112	if checkpoint.exists() and meta_path.exists():	Branches only when this condition is true; verify both the true and false paths in tests.
113	return json.loads(meta_path.read_text(encoding="utf-8"))	Returns the result to the caller; this is the function output boundary that tests should assert.
114	_, standardizer, datasets = _load_datasets(config, canonical_path, run_dir)	Creates or updates `_, standardizer, datasets`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
115	if len(datasets["train"]) == 0 or len(datasets["val"]) == 0:	Branches only when this condition is true; verify both the true and false paths in tests.
116	raise RuntimeError("Not enough windows after session split; increase session dur	Stops execution because an invariant or supported-condition check failed.
117	train_loader = DataLoader(datasets["train"], batch_size=config.training.batch_si	Creates or updates `train_loader`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
118	val_loader = DataLoader(datasets["val"], batch_size=config.training.batch_size,	Creates or updates `val_loader`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
119	model = build_model(config.model, len(MODEL_INPUT_COLUMNS), len(TARGET_COLUMNS))	Creates or updates `model`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
120	result = train_world_model(model, train_loader, val_loader, config, model_dir)	Creates or updates `result`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
121	meta = {	Creates or updates `meta`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
122	"kind": config.model.kind,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
123	"input_dim": len(MODEL_INPUT_COLUMNS),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
124	"state_dim": len(TARGET_COLUMNS),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
125	"hidden_dim": config.model.hidden_dim,	Executes this statement inside the current control-flow block; connect it to the nearest

		function contract and testable invariant.
126	"latent_dim": config.model.latent_dim,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
127	"layers": config.model.layers,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
128	"dropout": config.model.dropout,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
129	"best_val_loss": result.best_val_loss,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
130	"epochs_completed": result.epochs_completed,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
131	}	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
132	meta_path.write_text(json.dumps(meta, indent=2), encoding="utf-8")	Persists an artifact. The surrounding code should include run identity, schema and preprocessing lineage.

Lines 133-154

```

0133     return meta
0134
0135
0136 def load_trained_model(config: ProjectConfig, run_dir: Path):
0137     meta = json.loads((run_dir / "model" / "model_meta.json").read_text(encoding="utf-8"))
0138     config.model.kind = meta["kind"]
0139     config.model.hidden_dim = meta["hidden_dim"]
0140     config.model.latent_dim = meta["latent_dim"]
0141     config.model.layers = meta["layers"]
0142     config.model.dropout = meta["dropout"]
0143     model = build_model(config.model, meta["input_dim"], meta["state_dim"])
0144     state = torch.load(run_dir / "model" / "world_model.pt", map_location="cpu", weights_only=True)
0145     model.load_state_dict(state)
0146     return model
0147
0148
0149 def evaluate_stage(config: ProjectConfig, canonical_path: Path, run_dir: Path) -> dict:
0150     output = run_dir / "metrics.json"
0151     if output.exists():
0152         return json.loads(output.read_text(encoding="utf-8"))
0153     _, standardizer, datasets = _load_datasets(config, canonical_path, run_dir)
0154     test_loader = DataLoader(datasets["test"], batch_size=config.training.batch_size, shuffle=False)

```

Line	Code signal	What it is doing / what to verify
133	return meta	Returns the result to the caller; this is the function output boundary that tests should assert.
136	def load_trained_model(config: ProjectConfig, run_dir: Path):	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
137	meta = json.loads((run_dir / "model" / "model_meta.json").read_text(encoding="ut	Creates or updates `meta`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
138	config.model.kind = meta["kind"]	Creates or updates `config.model.kind`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
139	config.model.hidden_dim = meta["hidden_dim"]	Creates or updates `config.model.hidden_dim`. Check its unit,

		shape, lifetime and whether it is derived only from information available at this point.
140	<code>config.model.latent_dim = meta["latent_dim"]</code>	Creates or updates `config.model.latent_dim`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
141	<code>config.model.layers = meta["layers"]</code>	Creates or updates `config.model.layers`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
142	<code>config.model.dropout = meta["dropout"]</code>	Creates or updates `config.model.dropout`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
143	<code>model = build_model(config.model, meta["input_dim"], meta["state_dim"])</code>	Creates or updates `model`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
144	<code>state = torch.load(run_dir / "model" / "world_model.pt", map_location="cpu", wei</code>	Creates or updates `state`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
145	<code>model.load_state_dict(state)</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
146	<code>return model</code>	Returns the result to the caller; this is the function output boundary that tests should assert.
149	<code>def evaluate_stage(config: ProjectConfig, canonical_path: Path, run_dir: Path) -</code>	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
150	<code>output = run_dir / "metrics.json"</code>	Creates or updates `output`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
151	<code>if output.exists():</code>	Branches only when this condition is true; verify both the true and false paths in tests.
152	<code>return json.loads(output.read_text(encoding="utf-8"))</code>	Returns the result to the caller; this is the function output boundary that tests should assert.
153	<code>_, standardizer, datasets = _load_datasets(config, canonical_path, run_dir)</code>	Creates or updates `_, standardizer, datasets`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
154	<code>test_loader = DataLoader(datasets["test"], batch_size=config.training.batch_size</code>	Creates or updates `test_loader`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 155-176

```

0155     model = load_trained_model(config, run_dir)
0156     return evaluate_world_model(model, test_loader, standardizer, output, config.training.device)
0157
0158
0159 def ablation_stage(config: ProjectConfig, canonical_path: Path, run_dir: Path) -> list[dict]:
0160     output = run_dir / "ablations.json"
0161     if output.exists():
0162         return json.loads(output.read_text(encoding="utf-8"))
0163     frame = pd.read_csv(canonical_path)
0164     splits = json.loads((run_dir / "splits.json").read_text(encoding="utf-8"))
0165     train = frame[frame.session_id.isin(splits["train"])]
0166     test = frame[frame.session_id.isin(splits["test"])]

```

Source → contract → runtime evidence → failure test

```

0167     feature_sets = {
0168         "state_only": STATE_COLUMNS,
0169         "state_plus_actions": STATE_COLUMNS + ["throttle", "brake", "gear_norm", "drs"],
0170         "full_context": MODEL_INPUT_COLUMNS,
0171         "no_weather": [c for c in MODEL_INPUT_COLUMNS if c not in {"rainfall", "air_temp_c", "track_temp_c",
"wind_speed_mps"}],
0172         "no_track_geometry": [c for c in MODEL_INPUT_COLUMNS if c not in {"curvature", "steering_proxy",
"track_progress_sin", "track_progress_cos"}],
0173     }
0174     results = []
0175     for name, features in feature_sets.items():
0176         # One-step ridge is intentionally used as a cheap, interpretable ablation probe.

```

Line	Code signal	What it is doing / what to verify
155	model = load_trained_model(config, run_dir)	Creates or updates `model`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
156	return evaluate_world_model(model, test_loader, standardizer, output, config.tra	Returns the result to the caller; this is the function output boundary that tests should assert.
159	def ablation_stage(config: ProjectConfig, canonical_path: Path, run_dir: Path) -	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
160	output = run_dir / "ablations.json"	Creates or updates `output`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
161	if output.exists():	Branches only when this condition is true; verify both the true and false paths in tests.
162	return json.loads(output.read_text(encoding="utf-8"))	Returns the result to the caller; this is the function output boundary that tests should assert.
163	frame = pd.read_csv(canonical_path)	Loads persisted tabular evidence; validate schema, units and ordering immediately afterward.
164	splits = json.loads(run_dir / "splits.json").read_text(encoding="utf-8"))	Creates or updates `splits`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
165	train = frame[frame.session_id.isin(splits["train"])] copy()	Creates or updates `train`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
166	test = frame[frame.session_id.isin(splits["test"])] copy()	Creates or updates `test`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
167	feature_sets = {	Creates or updates `feature_sets`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
168	"state_only": STATE_COLUMNS,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
169	"state_plus_actions": STATE_COLUMNS + ["throttle", "brake", "gear_norm", "drs"],	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
170	"full_context": MODEL_INPUT_COLUMNS,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

171	"no_weather": [c for c in MODEL_INPUT_COLUMNS if c not in {"rainfall", "air_temp	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
172	"no_track_geometry": [c for c in MODEL_INPUT_COLUMNS if c not in {"curvature", "	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
173	}	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
174	results = []	Creates or updates `results`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
175	for name, features in feature_sets.items():	Iterates over a collection or time dimension. Confirm order, boundaries and whether state should persist between iterations.
176	# One-step ridge is intentionally used as a cheap, interpretable ablation probe.	Documents an assumption, intent or constraint that should remain aligned with the implementation.

Lines 177-198

```

0177     x_train = train[features].iloc[:-1].to_numpy(np.float32)
0178     y_train = train[TARGET_COLUMNS].iloc[1:].to_numpy(np.float32)
0179     same_session = train.session_id.iloc[:-1].to_numpy() == train.session_id.iloc[1:].to_numpy()
0180     x_train, y_train = x_train[same_session], y_train[same_session]
0181     x_test = test[features].iloc[:-1].to_numpy(np.float32)
0182     y_test = test[TARGET_COLUMNS].iloc[1:].to_numpy(np.float32)
0183     same_test = test.session_id.iloc[:-1].to_numpy() == test.session_id.iloc[1:].to_numpy()
0184     x_test, y_test = x_test[same_test], y_test[same_test]
0185     from sklearn.linear_model import Ridge
0186     from sklearn.preprocessing import StandardScaler
0187     from sklearn.pipeline import make_pipeline
0188     from sklearn.multioutput import MultiOutputRegressor
0189
0190     model = make_pipeline(StandardScaler(), MultiOutputRegressor(Ridge(alpha=1.0)))
0191     model.fit(x_train, y_train)
0192     prediction = model.predict(x_test)
0193     results.append(
0194         {
0195             "ablation": name,
0196             "features": features,
0197             "speed_mae_mps": float(np.mean(np.abs(prediction[:, 0] - y_test[:, 0]))),
0198             "overall_mae": float(np.mean(np.abs(prediction - y_test))),

```

Line	Code signal	What it is doing / what to verify
177	x_train = train[features].iloc[:-1].to_numpy(np.float32)	Creates or updates `x_train`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
178	y_train = train[TARGET_COLUMNS].iloc[1:].to_numpy(np.float32)	Creates or updates `y_train`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
179	same_session = train.session_id.iloc[:-1].to_numpy() == train.session_id.iloc[1:]	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
180	x_train, y_train = x_train[same_session], y_train[same_session]	Creates or updates `x_train, y_train`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
181	x_test = test[features].iloc[:-1].to_numpy(np.float32)	Creates or updates `x_test`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
182	y_test =	Creates or updates `y_test`. Check its unit,

	test[TARGET_COLUMNS].iloc[1:].to_numpy(np.float32)	shape, lifetime and whether it is derived only from information available at this point.
183	same_test = test.session_id.iloc[:-1].to_numpy() == test.session_id.iloc[1:].to_	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
184	x_test, y_test = x_test[same_test], y_test[same_test]	Creates or updates `x_test, y_test`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
185	from sklearn.linear_model import Ridge	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
186	from sklearn.preprocessing import StandardScaler	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
187	from sklearn.pipeline import make_pipeline	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
188	from sklearn.multioutput import MultiOutputRegressor	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
190	model = make_pipeline(StandardScaler(), MultiOutputRegressor(Ridge(alpha=1.0)))	Creates or updates `model`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
191	model.fit(x_train, y_train)	Learns parameters or preprocessing statistics from the supplied training evidence only.
192	prediction = model.predict(x_test)	Runs model inference using learned parameters without fitting on the requested examples.
193	results.append(	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
194	{	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
195	"ablation": name,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
196	"features": features,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
197	"speed_mae_mps": float(np.mean(np.abs(prediction[:, 0] - y_test[:, 0]))),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
198	"overall_mae": float(np.mean(np.abs(prediction - y_test))),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

Lines 199-220

```

0199         }
0200     )
0201     results.sort(key=lambda item: item["speed_mae_mps"])
0202     output.write_text(json.dumps(results, indent=2), encoding="utf-8")
0203     return results
0204
0205
0206 def publish_stage(config: ProjectConfig, canonical_path: Path, run_dir: Path) -> dict:
0207     output = run_dir / "publication.json"
0208     if output.exists():

```

Source → contract → runtime evidence → failure test

```

0209     return json.loads(output.read_text(encoding="utf-8"))
0210     frame, standardizer, datasets = _load_datasets(config, canonical_path, run_dir)
0211     model = load_trained_model(config, run_dir)
0212     sample = datasets["test"][0]
0213     history = sample["history"].unsqueeze(0)
0214     future_inputs = sample["future_inputs"].unsqueeze(0)
0215     with torch.no_grad():
0216         if config.model.kind == "rssm":
0217             prediction, _ = model(history, future_inputs, future_targets=None)
0218         else:
0219             prediction = model(history, future_inputs)
0220     predicted = standardizer.inverse_targets(prediction.numpy()[0])

```

Line	Code signal	What it is doing / what to verify
199	}	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
200	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
201	results.sort(key=lambda item: item["speed_mae_mps"])	Creates or updates `results.sort(key)`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
202	output.write_text(json.dumps(results, indent=2), encoding="utf-8")	Persists an artifact. The surrounding code should include run identity, schema and preprocessing lineage.
203	return results	Returns the result to the caller; this is the function output boundary that tests should assert.
206	def publish_stage(config: ProjectConfig, canonical_path: Path, run_dir: Path) ->	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
207	output = run_dir / "publication.json"	Creates or updates `output`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
208	if output.exists():	Branches only when this condition is true; verify both the true and false paths in tests.
209	return json.loads(output.read_text(encoding="utf-8"))	Returns the result to the caller; this is the function output boundary that tests should assert.
210	frame, standardizer, datasets = _load_datasets(config, canonical_path, run_dir)	Creates or updates `frame, standardizer, datasets`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
211	model = load_trained_model(config, run_dir)	Creates or updates `model`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
212	sample = datasets["test"][0]	Creates or updates `sample`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
213	history = sample["history"].unsqueeze(0)	Creates or updates `history`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
214	future_inputs = sample["future_inputs"].unsqueeze(0)	Creates or updates `future_inputs`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
215	with torch.no_grad():	Opens a managed resource or context and

		guarantees cleanup when the block exits.
216	if config.model.kind == "rssm":	Branches only when this condition is true; verify both the true and false paths in tests.
217	prediction, _ = model(history, future_inputs, future_targets=None)	Creates or updates `prediction, _`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
218	else:	Handles all remaining cases; inspect whether this fallback is intentionally broad.
219	prediction = model(history, future_inputs)	Creates or updates `prediction`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
220	predicted = standardizer.inverse_targets(prediction.numpy()[0])	Creates or updates `predicted`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 221-241

```

0221     truth = standardizer.inverse_targets(sample["future_targets"].numpy())
0222     comparison = pd.DataFrame(
0223         {
0224             "step": np.arange(len(predicted)),
0225             "predicted_speed_mps": predicted[:, 0],
0226             "actual_speed_mps": truth[:, 0],
0227             "predicted_accel_mps2": predicted[:, 1],
0228             "actual_accel_mps2": truth[:, 1],
0229         }
0230     )
0231     comparison_path = run_dir / "rollout_preview.csv"
0232     comparison.to_csv(comparison_path, index=False)
0233     payload = {
0234         "run_dir": str(run_dir),
0235         "model_kind": config.model.kind,
0236         "rollout_preview": str(comparison_path),
0237         "test_sessions": json.loads((run_dir / "splits.json").read_text())["test"],
0238         "ui_command": f"apexsim ui --run-dir {run_dir}",
0239     }
0240     output.write_text(json.dumps(payload, indent=2), encoding="utf-8")
0241     return payload

```

Line	Code signal	What it is doing / what to verify
221	truth = standardizer.inverse_targets(sample["future_targets"].numpy())	Creates or updates `truth`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
222	comparison = pd.DataFrame(	Creates or updates `comparison`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
223	{	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
224	"step": np.arange(len(predicted)),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
225	"predicted_speed_mps": predicted[:, 0],	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
226	"actual_speed_mps": truth[:, 0],	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
227	"predicted_accel_mps2": predicted[:, 1],	Executes this statement inside the current control-flow block; connect it to the nearest

		function contract and testable invariant.
228	"actual_accel_mps2": truth[:, 1],	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
229	}	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
230	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
231	comparison_path = run_dir / "rollout_preview.csv"	Creates or updates `comparison_path`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
232	comparison.to_csv(comparison_path, index=False)	Persists an artifact. The surrounding code should include run identity, schema and preprocessing lineage.
233	payload = {	Creates or updates `payload`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
234	"run_dir": str(run_dir),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
235	"model_kind": config.model.kind,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
236	"rollout_preview": str(comparison_path),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
237	"test_sessions": json.loads((run_dir / "splits.json").read_text())["test"],	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
238	"ui_command": f"apexsim ui --run-dir {run_dir}",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
239	}	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
240	output.write_text(json.dumps(payload, indent=2, encoding="utf-8"))	Persists an artifact. The surrounding code should include run identity, schema and preprocessing lineage.
241	return payload	Returns the result to the caller; this is the function output boundary that tests should assert.

src/apexsim/plotting.py

Turns telemetry and rollout evidence into diagnostic figures.

Plots should expose alignment, horizon drift and intervention effects rather than merely decorate the UI.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```
0001 from __future__ import annotations
0002
0003 from pathlib import Path
0004
0005 import matplotlib.pyplot as plt
0006 import pandas as pd
0007
0008
0009 def save_track_map(frame: pd.DataFrame, output_path: str | Path) -> Path:
0010     path = Path(output_path)
0011     path.parent.mkdir(parents=True, exist_ok=True)
0012     fig, ax = plt.subplots(figsize=(6, 4.5))
0013     session = frame[frame.session_id == frame.session_id.iloc[0]]
0014     ax.plot(session.x_m, session.y_m, linewidth=2)
0015     ax.scatter(session.x_m.iloc[0], session.y_m.iloc[0], s=50, label="Start")
0016     ax.set_title("Synthetic track geometry")
0017     ax.set_xlabel("x (m)")
0018     ax.set_ylabel("y (m)")
0019     ax.set_aspect("equal", adjustable="box")
0020     ax.legend()
0021     fig.tight_layout()
0022     fig.savefig(path, dpi=160)
```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	from pathlib import Path	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
5	import matplotlib.pyplot as plt	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
6	import pandas as pd	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
9	def save_track_map(frame: pd.DataFrame, output_path: str Path) -> Path:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
10	path = Path(output_path)	Creates or updates `path`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
11	path.parent.mkdir(parents=True, exist_ok=True)	Creates or updates `path.parent.mkdir(parents)`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

12	<code>fig, ax = plt.subplots(figsize=(6, 4.5))</code>	Creates or updates `fig, ax`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
13	<code>session = frame[frame.session_id == frame.session_id.iloc[0]]</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
14	<code>ax.plot(session.x_m, session.y_m, linewidth=2)</code>	Creates or updates `ax.plot(session.x_m, session.y_m, linewidth=2)`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
15	<code>ax.scatter(session.x_m.iloc[0], session.y_m.iloc[0], s=50, label="Start")</code>	Creates or updates `ax.scatter(session.x_m.iloc[0], session.y_m.iloc[0], s=50, label="Start")`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
16	<code>ax.set_title("Synthetic track geometry")</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
17	<code>ax.set_xlabel("x (m)")</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
18	<code>ax.set_ylabel("y (m)")</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
19	<code>ax.set_aspect("equal", adjustable="box")</code>	Creates or updates `ax.set_aspect("equal", adjustable="box")`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
20	<code>ax.legend()</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
21	<code>fig.tight_layout()</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
22	<code>fig.savefig(path, dpi=160)</code>	Creates or updates `fig.savefig(path, dpi)`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 23-41

```

0023     plt.close(fig)
0024     return path
0025
0026
0027 def save_training_curve(history: list[dict], output_path: str | Path) -> Path:
0028     path = Path(output_path)
0029     path.parent.mkdir(parents=True, exist_ok=True)
0030     frame = pd.DataFrame(history)
0031     fig, ax = plt.subplots(figsize=(6, 4))
0032     ax.plot(frame.epoch, frame.train_loss, marker="o", label="Train")
0033     ax.plot(frame.epoch, frame.val_loss, marker="o", label="Validation")
0034     ax.set_xlabel("Epoch")
0035     ax.set_ylabel("Loss")
0036     ax.set_title("World-model training")
0037     ax.legend()
0038     fig.tight_layout()
0039     fig.savefig(path, dpi=160)
0040     plt.close(fig)
0041     return path

```

Line	Code signal	What it is doing / what to verify
23	<code>plt.close(fig)</code>	Calls a function or method. Trace its input

		contract, returned value and any persistent side effects.
24	return path	Returns the result to the caller; this is the function output boundary that tests should assert.
27	def save_training_curve(history: list[dict], output_path: str Path) -> Path:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
28	path = Path(output_path)	Creates or updates `path`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
29	path.parent.mkdir(parents=True, exist_ok=True)	Creates or updates `path.parent.mkdir(parents)`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
30	frame = pd.DataFrame(history)	Creates or updates `frame`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
31	fig, ax = plt.subplots(figsize=(6, 4))	Creates or updates `fig, ax`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
32	ax.plot(frame.epoch, frame.train_loss, marker="o", label="Train")	Creates or updates `ax.plot(frame.epoch, frame.train_loss, marker)`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
33	ax.plot(frame.epoch, frame.val_loss, marker="o", label="Validation")	Creates or updates `ax.plot(frame.epoch, frame.val_loss, marker)`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
34	ax.set_xlabel("Epoch")	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
35	ax.set_ylabel("Loss")	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
36	ax.set_title("World-model training")	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
37	ax.legend()	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
38	fig.tight_layout()	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
39	fig.savefig(path, dpi=160)	Creates or updates `fig.savefig(path, dpi)`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
40	plt.close(fig)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
41	return path	Returns the result to the caller; this is the function output boundary that tests should assert.

src/apexsim/registry.py

Persists run identity, status, configuration, metrics and artifact locations.

A registry is evidence, not a leaderboard. It makes promotion and rollback auditable.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```
0001 from __future__ import annotations
0002
0003 import json
0004 import sqlite3
0005 from datetime import datetime, timezone
0006 from pathlib import Path
0007
0008
0009 class RunRegistry:
0010     def __init__(self, database_path: str | Path) -> None:
0011         self.path = Path(database_path)
0012         self.path.parent.mkdir(parents=True, exist_ok=True)
0013         with self._connect() as connection:
0014             connection.execute(
0015                 """
0016                 CREATE TABLE IF NOT EXISTS runs (
0017                     run_id TEXT PRIMARY KEY,
0018                     status TEXT NOT NULL,
0019                     model_kind TEXT NOT NULL,
0020                     started_at TEXT NOT NULL,
0021                     finished_at TEXT,
0022                     run_dir TEXT NOT NULL,
```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	import json	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
4	import sqlite3	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
5	from datetime import datetime, timezone	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
6	from pathlib import Path	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
9	class RunRegistry:	Defines a type that owns related state and behaviour behind a named interface.
10	def __init__(self, database_path: str Path) -> None:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
11	self.path = Path(database_path)	Creates or updates `self.path`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Source → contract → runtime evidence → failure test

12	<code>self.path.parent.mkdir(parents=True, exist_ok=True)</code>	Creates or updates <code>self.path.parent.mkdir(parents=True)</code> . Check its unit, shape, lifetime and whether it is derived only from information available at this point.
13	<code>with self._connect() as connection:</code>	Opens a managed resource or context and guarantees cleanup when the block exits.
14	<code>connection.execute(</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
15	<code>"""</code>	Begins or ends documentation describing the module, class or function contract.
16	<code>CREATE TABLE IF NOT EXISTS runs (</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
17	<code>run_id TEXT PRIMARY KEY,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
18	<code>status TEXT NOT NULL,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
19	<code>model_kind TEXT NOT NULL,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
20	<code>started_at TEXT NOT NULL,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
21	<code>finished_at TEXT,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
22	<code>run_dir TEXT NOT NULL,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

Lines 23-44

```

0023         metrics_json TEXT
0024     )
0025     """
0026 )
0027
0028 def _connect(self) -> sqlite3.Connection:
0029     return sqlite3.connect(self.path)
0030
0031 def start(self, run_id: str, model_kind: str, run_dir: str) -> None:
0032     now = datetime.now(timezone.utc).isoformat()
0033     with self._connect() as connection:
0034         connection.execute(
0035             "INSERT OR REPLACE INTO runs VALUES (?, ?, ?, ?, ?, ?, ?)",
0036             (run_id, "running", model_kind, now, None, run_dir, None),
0037         )
0038
0039 def finish(self, run_id: str, status: str, metrics: dict | None = None) -> None:
0040     now = datetime.now(timezone.utc).isoformat()
0041     with self._connect() as connection:
0042         connection.execute(
0043             "UPDATE runs SET status=?, finished_at=?, metrics_json=? WHERE run_id=?",
0044             (status, now, json.dumps(metrics) if metrics else None, run_id),

```

Line	Code signal	What it is doing / what to verify
23	<code>metrics_json TEXT</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
24	<code>)</code>	Calls a function or method. Trace its input contract, returned value and any persistent

		side effects.
25	"""	Begins or ends documentation describing the module, class or function contract.
26	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
28	def _connect(self) -> sqlite3.Connection:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
29	return sqlite3.connect(self.path)	Returns the result to the caller; this is the function output boundary that tests should assert.
31	def start(self, run_id: str, model_kind: str, run_dir: str) -> None:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
32	now = datetime.now(timezone.utc).isoformat()	Creates or updates `now`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
33	with self._connect() as connection:	Opens a managed resource or context and guarantees cleanup when the block exits.
34	connection.execute(	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
35	"INSERT OR REPLACE INTO runs VALUES (?, ?, ?, ?, ?, ?)",	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
36	(run_id, "running", model_kind, now, None, run_dir, None),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
37	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
39	def finish(self, run_id: str, status: str, metrics: dict None = None) -> None:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
40	now = datetime.now(timezone.utc).isoformat()	Creates or updates `now`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
41	with self._connect() as connection:	Opens a managed resource or context and guarantees cleanup when the block exits.
42	connection.execute(	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
43	"UPDATE runs SET status=?, finished_at=?, metrics_json=? WHERE run_id=?",	Creates or updates `UPDATE runs SET status`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
44	(status, now, json.dumps(metrics) if metrics else None, run_id),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

Lines 45-56

```

0045         )
0046
0047     def list_runs(self) -> list[dict]:
0048         with self._connect() as connection:
0049             connection.row_factory = sqlite3.Row
0050             rows = connection.execute("SELECT * FROM runs ORDER BY started_at DESC").fetchall()
0051             result = []

```

Source → contract → runtime evidence → failure test

```

0052     for row in rows:
0053         item = dict(row)
0054         item["metrics"] = json.loads(item.pop("metrics_json")) if item.get("metrics_json") else None
0055         result.append(item)
0056     return result

```

Line	Code signal	What it is doing / what to verify
45	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
47	def list_runs(self) -> list[dict]:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
48	with self._connect() as connection:	Opens a managed resource or context and guarantees cleanup when the block exits.
49	connection.row_factory = sqlite3.Row	Creates or updates `connection.row_factory`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
50	rows = connection.execute("SELECT * FROM runs ORDER BY started_at DESC").fetchall	Creates or updates `rows`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
51	result = []	Creates or updates `result`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
52	for row in rows:	Iterates over a collection or time dimension. Confirm order, boundaries and whether state should persist between iterations.
53	item = dict(row)	Creates or updates `item`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
54	item["metrics"] = json.loads(item.pop("metrics_json")) if item.get("metrics_json	Creates or updates `item["metrics"]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
55	result.append(item)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
56	return result	Returns the result to the caller; this is the function output boundary that tests should assert.

src/apexsim/serving.py

Creates typed API boundaries for health, runs and scenarios.

Serving should load compatible artifacts once, validate units and avoid exposing internal tensors as public contracts.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```
0001 from __future__ import annotations
0002
0003 import json
0004 from pathlib import Path
0005
0006 from fastapi import FastAPI, HTTPException
0007
0008 from apexsim.registry import RunRegistry
0009
0010
0011 def create_api(artifacts_dir: str | Path = "artifacts/runs") -> FastAPI:
0012     root = Path(artifacts_dir)
0013     registry = RunRegistry(root / "runs.sqlite")
0014     app = FastAPI(title="Project APEX API", version="0.1.0")
0015
0016     @app.get("/health")
0017     def health() -> dict[str, str]:
0018         return {"status": "ok"}
0019
0020     @app.get("/runs")
0021     def runs() -> list[dict]:
0022         return registry.list_runs()
```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	import json	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
4	from pathlib import Path	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
6	from fastapi import FastAPI, HTTPException	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
8	from apexsim.registry import RunRegistry	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
11	def create_api(artifacts_dir: str Path = "artifacts/runs") -> FastAPI:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
12	root = Path(artifacts_dir)	Creates or updates `root`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
13	registry = RunRegistry(root / "runs.sqlite")	Creates or updates `registry`. Check its unit,

Source → contract → runtime evidence → failure test

		shape, lifetime and whether it is derived only from information available at this point.
14	<code>app = FastAPI(title="Project APEX API", version="0.1.0")</code>	Creates or updates `app`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
16	<code>@app.get("/health")</code>	Applies a decorator that changes registration, validation or runtime behaviour of the definition that follows.
17	<code>def health() -> dict[str, str]:</code>	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
18	<code>return {"status": "ok"}</code>	Returns the result to the caller; this is the function output boundary that tests should assert.
20	<code>@app.get("/runs")</code>	Applies a decorator that changes registration, validation or runtime behaviour of the definition that follows.
21	<code>def runs() -> list[dict]:</code>	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
22	<code>return registry.list_runs()</code>	Returns the result to the caller; this is the function output boundary that tests should assert.

Lines 23-40

```

0023
0024     @app.get("/runs/{run_id}")
0025     def run_details(run_id: str) -> dict:
0026         path = root / run_id / "summary.json"
0027         if not path.exists():
0028             raise HTTPException(status_code=404, detail="Run not found")
0029         return json.loads(path.read_text())
0030
0031     @app.get("/runs/{run_id}/rollout")
0032     def rollout_preview(run_id: str) -> list[dict]:
0033         path = root / run_id / "rollout_preview.csv"
0034         if not path.exists():
0035             raise HTTPException(status_code=404, detail="Rollout preview not found")
0036         import pandas as pd
0037
0038         return pd.read_csv(path).to_dict(orient="records")
0039
0040     return app

```

Line	Code signal	What it is doing / what to verify
24	<code>@app.get("/runs/{run_id}")</code>	Applies a decorator that changes registration, validation or runtime behaviour of the definition that follows.
25	<code>def run_details(run_id: str) -> dict:</code>	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
26	<code>path = root / run_id / "summary.json"</code>	Creates or updates `path`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
27	<code>if not path.exists():</code>	Branches only when this condition is true; verify both the true and false paths in tests.
28	<code>raise HTTPException(status_code=404, detail="Run not found")</code>	Stops execution because an invariant or supported-condition check failed.
29	<code>return json.loads(path.read_text())</code>	Returns the result to the caller; this is the function output boundary that tests should

		assert.
31	@app.get("/runs/{run_id}/rollout")	Applies a decorator that changes registration, validation or runtime behaviour of the definition that follows.
32	def rollout_preview(run_id: str) -> list[dict]:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
33	path = root / run_id / "rollout_preview.csv"	Creates or updates `path`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
34	if not path.exists():	Branches only when this condition is true; verify both the true and false paths in tests.
35	raise HTTPException(status_code=404, detail="Rollout preview not found")	Stops execution because an invariant or supported-condition check failed.
36	import pandas as pd	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
38	return pd.read_csv(path).to_dict(orient="records")	Returns the result to the caller; this is the function output boundary that tests should assert.
40	return app	Returns the result to the caller; this is the function output boundary that tests should assert.

src/apexsim/simulation.py

Applies bounded counterfactual interventions and runs imagined futures.

Actions and context can be changed only within explicit support; the scenario engine must preserve the model contract and communicate extrapolation.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```
0001 from __future__ import annotations
0002
0003 from dataclasses import dataclass
0004
0005 import numpy as np
0006 import torch
0007 from torch import nn
0008
0009 from apexsim.contracts import ACTION_COLUMNS, CONTEXT_COLUMNS, MODEL_INPUT_COLUMNS, STATE_COLUMNS
0010 from apexsim.data.features import Standardizer
0011 from apexsim.models.rssm import RSSMWorldModel
0012
0013
0014 @dataclass(frozen=True)
0015 class Scenario:
0016     throttle_scale: float = 1.0
0017     brake_scale: float = 1.0
0018     grip_multiplier: float = 1.0
0019     rain_delta: float = 0.0
0020     tyre_degradation_multiplier: float = 1.0
0021
0022
```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	from dataclasses import dataclass	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
5	import numpy as np	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
6	import torch	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
7	from torch import nn	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
9	from apexsim.contracts import ACTION_COLUMNS, CONTEXT_COLUMNS, MODEL_INPUT_COLUMNS	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
10	from apexsim.data.features import Standardizer	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
11	from apexsim.models.rssm import RSSMWorldModel	Imports a dependency required by the definitions below; keeping imports explicit

Source → contract → runtime evidence → failure test

		reveals architectural coupling.
14	@dataclass(frozen=True)	Applies a decorator that changes registration, validation or runtime behaviour of the definition that follows.
15	class Scenario:	Defines a type that owns related state and behaviour behind a named interface.
16	throttle_scale: float = 1.0	Creates or updates `throttle_scale: float`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
17	brake_scale: float = 1.0	Creates or updates `brake_scale: float`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
18	grip_multiplier: float = 1.0	Creates or updates `grip_multiplier: float`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
19	rain_delta: float = 0.0	Creates or updates `rain_delta: float`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
20	tyre_degradation_multiplier: float = 1.0	Creates or updates `tyre_degradation_multiplier: float`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 23-44

```

0023 def apply_scenario(future_inputs_raw: np.ndarray, scenario: Scenario) -> np.ndarray:
0024     modified = future_inputs_raw.copy()
0025     index = {name: i for i, name in enumerate(MODEL_INPUT_COLUMNS)}
0026     modified[:, index["throttle"]] = np.clip(
0027         modified[:, index["throttle"]] * scenario.throttle_scale, 0.0, 1.0
0028     )
0029     modified[:, index["brake"]] = np.clip(
0030         modified[:, index["brake"]] * scenario.brake_scale, 0.0, 1.0
0031     )
0032     modified[:, index["grip_level"]] = np.clip(
0033         modified[:, index["grip_level"]] * scenario.grip_multiplier, 0.2, 1.5
0034     )
0035     modified[:, index["rainfall"]] = np.clip(
0036         modified[:, index["rainfall"]] + scenario.rain_delta, 0.0, 1.0
0037     )
0038     tyre_index = index["tyre_age_laps"]
0039     base_age = modified[0, tyre_index]
0040     modified[:, tyre_index] = base_age + (
0041         modified[:, tyre_index] - base_age
0042     ) * scenario.tyre_degradation_multiplier
0043     return modified
0044

```

Line	Code signal	What it is doing / what to verify
23	def apply_scenario(future_inputs_raw: np.ndarray, scenario: Scenario) -> np.ndarray	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
24	modified = future_inputs_raw.copy()	Creates or updates `modified`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
25	index = {name: i for i, name in enumerate(MODEL_INPUT_COLUMNS)}	Creates or updates `index`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
26	modified[:, index["throttle"]] = np.clip(	Bounds a value to a supported range.

		Monitor how often clipping occurs because it may hide distribution shift.
27	<code>modified[:, index["throttle"]] * scenario.throttle_scale, 0.0, 1.0</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
28	<code>)</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
29	<code>modified[:, index["brake"]] = np.clip(</code>	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.
30	<code>modified[:, index["brake"]] * scenario.brake_scale, 0.0, 1.0</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
31	<code>)</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
32	<code>modified[:, index["grip_level"]] = np.clip(</code>	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.
33	<code>modified[:, index["grip_level"]] * scenario.grip_multiplier, 0.2, 1.5</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
34	<code>)</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
35	<code>modified[:, index["rainfall"]] = np.clip(</code>	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.
36	<code>modified[:, index["rainfall"]] + scenario.rain_delta, 0.0, 1.0</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
37	<code>)</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
38	<code>tyre_index = index["tyre_age_laps"]</code>	Creates or updates `tyre_index`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
39	<code>base_age = modified[0, tyre_index]</code>	Creates or updates `base_age`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
40	<code>modified[:, tyre_index] = base_age + (</code>	Creates or updates `modified[:, tyre_index]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
41	<code>modified[:, tyre_index] - base_age</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
42	<code>) * scenario.tyre_degradation_multiplier</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
43	<code>return modified</code>	Returns the result to the caller; this is the function output boundary that tests should assert.

Lines 45-66

```
0045
0046 def rollout(
```

Source → contract → runtime evidence → failure test

```

0047     model: nn.Module,
0048     history_raw: np.ndarray,
0049     future_inputs_raw: np.ndarray,
0050     standardizer: Standardizer,
0051     scenario: Scenario,
0052     device: str = "cpu",
0053 ) -> np.ndarray:
0054     model.eval().to(device)
0055     future_modified = apply_scenario(future_inputs_raw, scenario)
0056     history = torch.from_numpy(
0057         standardizer.transform_inputs(history_raw).astype(np.float32)[None, ...]
0058     ).to(device)
0059     future = torch.from_numpy(
0060         standardizer.transform_inputs(future_modified).astype(np.float32)[None, ...]
0061     ).to(device)
0062     with torch.no_grad():
0063         if isinstance(model, RSSMWorldModel):
0064             prediction_z, _ = model(history, future, future_targets=None)
0065         else:
0066             prediction_z = model(history, future)

```

Line	Code signal	What it is doing / what to verify
46	def rollout(	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
47	model: nn.Module,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
48	history_raw: np.ndarray,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
49	future_inputs_raw: np.ndarray,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
50	standardizer: Standardizer,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
51	scenario: Scenario,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
52	device: str = "cpu",	Creates or updates `device: str`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
53	) -> np.ndarray:	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
54	model.eval().to(device)	Enables deterministic evaluation behaviour for layers that differ between training and inference.
55	future_modified = apply_scenario(future_inputs_raw, scenario)	Creates or updates `future_modified`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
56	history = torch.from_numpy(	Creates or updates `history`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
57	standardizer.transform_inputs(history_raw). astype(np.float32)[None, ...]	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
58	).to(device)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
59	future = torch.from_numpy(	Creates or updates `future`. Check its unit,

		shape, lifetime and whether it is derived only from information available at this point.
60	<code>standardizer.transform_inputs(future_modified).astype(np.float32)[None, ...]</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
61	<code>).to(device)</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
62	<code>with torch.no_grad():</code>	Opens a managed resource or context and guarantees cleanup when the block exits.
63	<code>if isinstance(model, RSSMWorldModel):</code>	Branches only when this condition is true; verify both the true and false paths in tests.
64	<code>prediction_z, _ = model(history, future, future_targets=None)</code>	Creates or updates `prediction_z, _`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
65	<code>else:</code>	Handles all remaining cases; inspect whether this fallback is intentionally broad.
66	<code>prediction_z = model(history, future)</code>	Creates or updates `prediction_z`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 67-67

```
0067     return standardizer.inverse_targets(prediction_z.cpu().numpy()[0])
```

Line	Code signal	What it is doing / what to verify
67	<code>return standardizer.inverse_targets(prediction_z.cpu().numpy()[0])</code>	Returns the result to the caller; this is the function output boundary that tests should assert.

src/apexsim/training.py

Owns optimization, validation, checkpoints and model-specific loss paths.

The training loop is a state machine: clear gradients, forward, measure loss, backpropagate, update, validate, compare and persist.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```
0001 from __future__ import annotations
0002
0003 import json
0004 import random
0005 from dataclasses import dataclass
0006 from pathlib import Path
0007
0008 import numpy as np
0009 import torch
0010 from torch import nn
0011 from torch.utils.data import DataLoader
0012
0013 from apexsim.config import ProjectConfig
0014 from apexsim.models.rssm import RSSMWorldModel
0015
0016
0017 @dataclass
0018 class TrainingResult:
0019     best_val_loss: float
0020     epochs_completed: int
0021     checkpoint_path: Path
0022     history_path: Path
```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	import json	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
4	import random	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
5	from dataclasses import dataclass	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
6	from pathlib import Path	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
8	import numpy as np	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
9	import torch	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
10	from torch import nn	Imports a dependency required by the definitions below; keeping imports explicit

Source → contract → runtime evidence → failure test

		reveals architectural coupling.
11	from torch.utils.data import DataLoader	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
13	from apexsim.config import ProjectConfig	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
14	from apexsim.models.rssm import RSSMWorldModel	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
17	@dataclass	Applies a decorator that changes registration, validation or runtime behaviour of the definition that follows.
18	class TrainingResult:	Defines a type that owns related state and behaviour behind a named interface.
19	best_val_loss: float	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
20	epochs_completed: int	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
21	checkpoint_path: Path	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
22	history_path: Path	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

Lines 23-44

```

0023
0024
0025 def seed_everything(seed: int) -> None:
0026     random.seed(seed)
0027     np.random.seed(seed)
0028     torch.manual_seed(seed)
0029
0030
0031 def _batch_loss(model: nn.Module, batch: dict, device: torch.device) -> tuple[torch.Tensor, torch.Tensor]:
0032     history = batch["history"].to(device)
0033     future_inputs = batch["future_inputs"].to(device)
0034     targets = batch["future_targets"].to(device)
0035     if isinstance(model, RSSMWorldModel):
0036         predictions, kl = model(history, future_inputs, future_targets=targets)
0037         reconstruction = torch.nn.functional.smooth_l1_loss(predictions, targets)
0038         # KL weight is intentionally modest for a small educational dataset.
0039         loss = reconstruction + 0.02 * kl
0040     else:
0041         predictions = model(history, future_inputs)
0042         loss = torch.nn.functional.smooth_l1_loss(predictions, targets)
0043     return loss, predictions
0044

```

Line	Code signal	What it is doing / what to verify
25	def seed_everything(seed: int) -> None:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
26	random.seed(seed)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
27	np.random.seed(seed)	Fixes a random seed to make the experiment easier to reproduce and compare.
28	torch.manual_seed(seed)	Fixes a random seed to make the experiment easier to reproduce and compare.

31	<code>def _batch_loss(model: nn.Module, batch: dict, device: torch.device) -> tuple[to</code>	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
32	<code>history = batch["history"].to(device)</code>	Creates or updates `history`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
33	<code>future_inputs = batch["future_inputs"].to(device)</code>	Creates or updates `future_inputs`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
34	<code>targets = batch["future_targets"].to(device)</code>	Creates or updates `targets`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
35	<code>if isinstance(model, RSSMWorldModel):</code>	Branches only when this condition is true; verify both the true and false paths in tests.
36	<code>predictions, kl = model(history, future_inputs, future_targets=targets)</code>	Creates or updates `predictions, kl`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
37	<code>reconstruction = torch.nn.functional.smooth_l1_loss(predictions, targets)</code>	Creates or updates `reconstruction`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
38	<code># KL weight is intentionally modest for a small educational dataset.</code>	Documents an assumption, intent or constraint that should remain aligned with the implementation.
39	<code>loss = reconstruction + 0.02 * kl</code>	Creates or updates `loss`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
40	<code>else:</code>	Handles all remaining cases; inspect whether this fallback is intentionally broad.
41	<code>predictions = model(history, future_inputs)</code>	Creates or updates `predictions`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
42	<code>loss = torch.nn.functional.smooth_l1_loss(predictions, targets)</code>	Creates or updates `loss`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
43	<code>return loss, predictions</code>	Returns the result to the caller; this is the function output boundary that tests should assert.

Lines 45-66

```

0045
0046 def train_world_model(
0047     model: nn.Module,
0048     train_loader: DataLoader,
0049     val_loader: DataLoader,
0050     config: ProjectConfig,
0051     run_dir: str | Path,
0052 ) -> TrainingResult:
0053     seed_everything(config.seed)
0054     run_path = Path(run_dir)
0055     run_path.mkdir(parents=True, exist_ok=True)
0056     device = torch.device(config.training.device)
0057     model.to(device)
0058     optimizer = torch.optim.AdamW(
0059         model.parameters(),
0060         lr=config.training.learning_rate,

```

```

0061     weight_decay=config.training.weight_decay,
0062 )
0063 history: list[dict[str, float | int]] = []
0064 best_val = float("inf")
0065 stale_epochs = 0
0066 checkpoint = run_path / "world_model.pt"

```

Line	Code signal	What it is doing / what to verify
46	def train_world_model(	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
47	model: nn.Module,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
48	train_loader: DataLoader,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
49	val_loader: DataLoader,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
50	config: ProjectConfig,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
51	run_dir: str Path,	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
52	) -> TrainingResult:	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
53	seed_everything(config.seed)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
54	run_path = Path(run_dir)	Creates or updates `run_path`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
55	run_path.mkdir(parents=True, exist_ok=True)	Creates or updates `run_path.mkdir(parents)`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
56	device = torch.device(config.training.device)	Creates or updates `device`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
57	model.to(device)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
58	optimizer = torch.optim.AdamW(	Creates or updates `optimizer`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
59	model.parameters(),	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
60	lr=config.training.learning_rate,	Creates or updates `lr`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
61	weight_decay=config.training.weight_decay,	Creates or updates `weight_decay`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
62	)	Calls a function or method. Trace its input

		contract, returned value and any persistent side effects.
63	history: list[dict[str, float int]] = []	Creates or updates `history: list[dict[str, float   int]]`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
64	best_val = float("inf")	Creates or updates `best_val`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
65	stale_epochs = 0	Creates or updates `stale_epochs`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
66	checkpoint = run_path / "world_model.pt"	Creates or updates `checkpoint`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 67-88

```

0067
0068     for epoch in range(1, config.training.epochs + 1):
0069         model.train()
0070         train_losses = []
0071         for batch in train_loader:
0072             optimizer.zero_grad(set_to_none=True)
0073             loss, _ = _batch_loss(model, batch, device)
0074             loss.backward()
0075             torch.nn.utils.clip_grad_norm_(model.parameters(), config.training.grad_clip)
0076             optimizer.step()
0077             train_losses.append(float(loss.detach().cpu()))
0078
0079         model.eval()
0080         val_losses = []
0081         with torch.no_grad():
0082             for batch in val_loader:
0083                 loss, _ = _batch_loss(model, batch, device)
0084                 val_losses.append(float(loss.detach().cpu()))
0085         train_loss = float(np.mean(train_losses))
0086         val_loss = float(np.mean(val_losses))
0087         history.append({"epoch": epoch, "train_loss": train_loss, "val_loss": val_loss})
0088

```

Line	Code signal	What it is doing / what to verify
68	for epoch in range(1, config.training.epochs + 1):	Iterates over a collection or time dimension. Confirm order, boundaries and whether state should persist between iterations.
69	model.train()	Enables training behaviour such as dropout and running-statistic updates.
70	train_losses = []	Creates or updates `train_losses`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
71	for batch in train_loader:	Iterates over a collection or time dimension. Confirm order, boundaries and whether state should persist between iterations.
72	optimizer.zero_grad(set_to_none=True)	Clears gradients left from the previous optimization step so they are not accumulated unintentionally.
73	loss, _ = _batch_loss(model, batch, device)	Creates or updates `loss, _`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
74	loss.backward()	Runs reverse-mode automatic differentiation to populate gradients for

		trainable parameters.
75	<code>torch.nn.utils.clip_grad_norm_(model.parameters(), config.training.grad_clip)</code>	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.
76	<code>optimizer.step()</code>	Updates trainable parameters using the optimizer state and current gradients.
77	<code>train_losses.append(float(loss.detach().cpu()))</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
79	<code>model.eval()</code>	Enables deterministic evaluation behaviour for layers that differ between training and inference.
80	<code>val_losses = []</code>	Creates or updates `val_losses`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
81	<code>with torch.no_grad():</code>	Opens a managed resource or context and guarantees cleanup when the block exits.
82	<code>for batch in val_loader:</code>	Iterates over a collection or time dimension. Confirm order, boundaries and whether state should persist between iterations.
83	<code>loss, _ = _batch_loss(model, batch, device)</code>	Creates or updates `loss, _`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
84	<code>val_losses.append(float(loss.detach().cpu()))</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
85	<code>train_loss = float(np.mean(train_losses))</code>	Creates or updates `train_loss`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
86	<code>val_loss = float(np.mean(val_losses))</code>	Creates or updates `val_loss`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
87	<code>history.append({"epoch": epoch, "train_loss": train_loss, "val_loss": val_loss})</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.

Lines 89-101

```

0089     if val_loss < best_val - 1e-6:
0090         best_val = val_loss
0091         stale_epochs = 0
0092         torch.save(model.state_dict(), checkpoint)
0093     else:
0094         stale_epochs += 1
0095         if stale_epochs >= config.training.patience:
0096             break
0097
0098     model.load_state_dict(torch.load(checkpoint, map_location=device, weights_only=True))
0099     history_path = run_path / "training_history.json"
0100     history_path.write_text(json.dumps(history, indent=2), encoding="utf-8")
0101     return TrainingResult(best_val, len(history), checkpoint, history_path)

```

Line	Code signal	What it is doing / what to verify
89	<code>if val_loss < best_val - 1e-6:</code>	Branches only when this condition is true; verify both the true and false paths in tests.
90	<code>best_val = val_loss</code>	Creates or updates `best_val`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

91	<code>stale_epochs = 0</code>	Creates or updates `stale_epochs`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
92	<code>torch.save(model.state_dict(), checkpoint)</code>	Persists an artifact. The surrounding code should include run identity, schema and preprocessing lineage.
93	<code>else:</code>	Handles all remaining cases; inspect whether this fallback is intentionally broad.
94	<code>stale_epochs += 1</code>	Creates or updates `stale_epochs +`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
95	<code>if stale_epochs >= config.training.patience:</code>	Branches only when this condition is true; verify both the true and false paths in tests.
96	<code>break</code>	Stops the enclosing loop after the required condition is reached.
98	<code>model.load_state_dict(torch.load(checkpoint, map_location=device, weights_only=T</code>	Creates or updates `model.load_state_dict(torch.load(checkpoint, map_location`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
99	<code>history_path = run_path / "training_history.json"</code>	Creates or updates `history_path`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
100	<code>history_path.write_text(json.dumps(history, indent=2), encoding="utf-8")</code>	Persists an artifact. The surrounding code should include run identity, schema and preprocessing lineage.
101	<code>return TrainingResult(best_val, len(history), checkpoint, history_path)</code>	Returns the result to the caller; this is the function output boundary that tests should assert.

src/apexsim/ui.py

Builds the interactive race-engineering interface.

The UI is an interpretation layer: it must preserve units, loading/error states, uncertainty and the distinction between recorded and imagined futures.

Contract questions

- Who calls this module?
- What units and shapes enter?
- What persistent state or artifacts change?
- Which failure should be caught here rather than downstream?
- Which test proves compatibility?

Lines 1-22

```
0001 from __future__ import annotations
0002
0003 import json
0004 from pathlib import Path
0005
0006 import gradio as gr
0007 import numpy as np
0008 import pandas as pd
0009 import plotly.graph_objects as go
0010 import torch
0011
0012 from apexsim.config import load_config
0013 from apexsim.contracts import MODEL_INPUT_COLUMNS
0014 from apexsim.data.features import Standardizer
0015 from apexsim.data.windows import TelemetryWindowDataset
0016 from apexsim.pipeline.stages import load_trained_model
0017 from apexsim.simulation import Scenario, rollout
0018
0019
0020 CUSTOM_CSS = """
0021 .gradio-container {max-width: 1500px !important; background: radial-gradient(circle at 15% 5%, #191b22 0%,
0022 #090a0e 48%, #050506 100%);}
0022 #hero {border: 1px solid #30343d; border-radius: 22px; padding: 24px; background: linear-gradient(135deg,
0022 rgba(239,35,60,.18), rgba(255,255,255,.025));}
```

Line	Code signal	What it is doing / what to verify
1	from __future__ import annotations	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
3	import json	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
4	from pathlib import Path	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
6	import gradio as gr	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
7	import numpy as np	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
8	import pandas as pd	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
9	import plotly.graph_objects as go	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.

10	import torch	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
12	from apexsim.config import load_config	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
13	from apexsim.contracts import MODEL_INPUT_COLUMNS	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
14	from apexsim.data.features import Standardizer	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
15	from apexsim.data.windows import TelemetryWindowDataset	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
16	from apexsim.pipeline.stages import load_trained_model	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
17	from apexsim.simulation import Scenario, rollout	Imports a dependency required by the definitions below; keeping imports explicit reveals architectural coupling.
20	CUSTOM_CSS = ""	Creates or updates `CUSTOM_CSS`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
21	.gradio-container {max-width: 1500px !important; background: radial-gradient(cir	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
22	#hero {border: 1px solid #30343d; border-radius: 22px; padding: 24px; background	Documents an assumption, intent or constraint that should remain aligned with the implementation.

Lines 23-44

```

0023 .metric-card {border: 1px solid #2e323b; border-radius: 16px; padding: 14px; background: rgba(255,255,255,.035);}
0024 ""
0025
0026
0027 def _load_bundle(run_dir: Path, config_path: Path):
0028     config = load_config(config_path)
0029     frame = pd.read_csv(run_dir / "canonical_telemetry.csv")
0030     splits = json.loads((run_dir / "splits.json").read_text())
0031     standardizer = Standardizer.from_dict(json.loads((run_dir / "standardizer.json").read_text()))
0032     model = load_trained_model(config, run_dir)
0033     dataset = TelemetryWindowDataset(
0034         frame,
0035         splits["test"],
0036         config.data.sequence_length,
0037         max(config.data.prediction_horizon, 80),
0038         standardizer,
0039         max_windows=200,
0040     )
0041     return config, frame, splits, standardizer, model, dataset
0042
0043
0044 def _track_figure(frame: pd.DataFrame, session_id: str) -> go.Figure:

```

Line	Code signal	What it is doing / what to verify
23	.metric-card {border: 1px solid #2e323b; border-radius: 16px; padding: 14px; bac	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
24	""	Begins or ends documentation describing the module, class or function contract.
27	def _load_bundle(run_dir: Path, config_path: Path):	Defines a callable boundary. Its parameters form the input contract and its return value

		or side effects form the output contract.
28	<code>config = load_config(config_path)</code>	Creates or updates `config`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
29	<code>frame = pd.read_csv(run_dir / "canonical_telemetry.csv")</code>	Loads persisted tabular evidence; validate schema, units and ordering immediately afterward.
30	<code>splits = json.loads((run_dir / "splits.json").read_text())</code>	Creates or updates `splits`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
31	<code>standardizer = Standardizer.from_dict(json.loads((run_dir / "standardizer.json"))</code>	Creates or updates `standardizer`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
32	<code>model = load_trained_model(config, run_dir)</code>	Creates or updates `model`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
33	<code>dataset = TelemetryWindowDataset(</code>	Creates or updates `dataset`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
34	<code>frame,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
35	<code>splits["test"],</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
36	<code>config.data.sequence_length,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
37	<code>max(config.data.prediction_horizon, 80),</code>	Runs model inference using learned parameters without fitting on the requested examples.
38	<code>standardizer,</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
39	<code>max_windows=200,</code>	Creates or updates `max_windows`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
40	<code>)</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
41	<code>return config, frame, splits, standardizer, model, dataset</code>	Returns the result to the caller; this is the function output boundary that tests should assert.
44	<code>def _track_figure(frame: pd.DataFrame, session_id: str) -> go.Figure:</code>	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.

Lines 45-66

```

0045     session = frame[frame.session_id == session_id]
0046     fig = go.Figure()
0047     fig.add_trace(
0048         go.Scatter(
0049             x=session.x_m,
0050             y=session.y_m,
0051             mode="lines",
0052             line=dict(width=6, color="#ef233c"),

```

Source → contract → runtime evidence → failure test


```

0053     hovertemplate="x=%{x:.0f}m<br>y=%{y:.0f}m<extra></extra>",
0054     )
0055 )
0056 fig.update_layout(
0057     template="plotly_dark",
0058     title=f"Track trace - {session_id}",
0059     xaxis_visible=False,
0060     yaxis_visible=False,
0061     height=430,
0062     margin=dict(l=15, r=15, t=50, b=15),
0063     paper_bgcolor="rgba(0,0,0,0)",
0064     plot_bgcolor="rgba(0,0,0,0)",
0065     yaxis_scaleanchor="x",
0066 )

```

Line	Code signal	What it is doing / what to verify
45	session = frame[frame.session_id == session_id]	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
46	fig = go.Figure()	Creates or updates `fig`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
47	fig.add_trace(	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
48	go.Scatter(	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
49	x=session.x_m,	Creates or updates `x`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
50	y=session.y_m,	Creates or updates `y`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
51	mode="lines",	Creates or updates `mode`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
52	line=dict(width=6, color="#ef233c"),	Creates or updates `line`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
53	hovertemplate="x=%{x:.0f}m y=%{y:.0f}m<extra></extra>",	Creates or updates `hovertemplate`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
54	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
55	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
56	fig.update_layout(	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
57	template="plotly_dark",	Creates or updates `template`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
58	title=f"Track trace - {session_id}",	Creates or updates `title`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
59	xaxis_visible=False,	Creates or updates `xaxis_visible`. Check its

		unit, shape, lifetime and whether it is derived only from information available at this point.
60	yaxis_visible=False,	Creates or updates `yaxis_visible`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
61	height=430,	Creates or updates `height`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
62	margin=dict(l=15, r=15, t=50, b=15),	Creates or updates `margin`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
63	paper_bgcolor="rgba(0,0,0,0)",	Creates or updates `paper_bgcolor`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
64	plot_bgcolor="rgba(0,0,0,0)",	Creates or updates `plot_bgcolor`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
65	yaxis_scaleanchor="x",	Creates or updates `yaxis_scaleanchor`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
66	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.

Lines 67-88

```

0067     return fig
0068
0069
0070 def _telemetry_figure(frame: pd.DataFrame, session_id: str) -> go.Figure:
0071     session = frame[frame.session_id == session_id].iloc[:1200]
0072     fig = go.Figure()
0073     fig.add_trace(go.Scatter(x=session.timestamp_s, y=session.speed_mps * 3.6, name="Speed km/h"))
0074     fig.add_trace(go.Scatter(x=session.timestamp_s, y=session.throttle * 100, name="Throttle %", yaxis="y2"))
0075     fig.add_trace(go.Scatter(x=session.timestamp_s, y=session.brake * 100, name="Brake %", yaxis="y2"))
0076     fig.update_layout(
0077         template="plotly_dark",
0078         title="Observed telemetry",
0079         xaxis_title="Time (s)",
0080         yaxis_title="Speed (km/h)",
0081         yaxis2=dict(title="Pedal (%)", overlaying="y", side="right", range=[0, 100]),
0082         height=430,
0083         legend=dict(orientation="h"),
0084         paper_bgcolor="rgba(0,0,0,0)",
0085         plot_bgcolor="rgba(255,255,255,.02)",
0086     )
0087     return fig
0088

```

Line	Code signal	What it is doing / what to verify
67	return fig	Returns the result to the caller; this is the function output boundary that tests should assert.
70	def _telemetry_figure(frame: pd.DataFrame, session_id: str) -> go.Figure:	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
71	session = frame[frame.session_id == session_id].iloc[:1200]	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

72	<code>fig = go.Figure()</code>	Creates or updates `fig`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
73	<code>fig.add_trace(go.Scatter(x=session.timestamp_s, y=session.speed_mps * 3.6, name=</code>	Creates or updates `fig.add_trace(go.Scatter(x`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
74	<code>fig.add_trace(go.Scatter(x=session.timestamp_s, y=session.throttle * 100, name=</code>	Creates or updates `fig.add_trace(go.Scatter(x`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
75	<code>fig.add_trace(go.Scatter(x=session.timestamp_s, y=session.brake * 100, name="Bra</code>	Creates or updates `fig.add_trace(go.Scatter(x`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
76	<code>fig.update_layout(</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
77	<code>template="plotly_dark",</code>	Creates or updates `template`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
78	<code>title="Observed telemetry",</code>	Creates or updates `title`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
79	<code>xaxis_title="Time (s)",</code>	Creates or updates `xaxis_title`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
80	<code>yaxis_title="Speed (km/h)",</code>	Creates or updates `yaxis_title`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
81	<code>yaxis2=dict(title="Pedal (%)", overlaying="y", side="right", range=[0, 100]),</code>	Creates or updates `yaxis2`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
82	<code>height=430,</code>	Creates or updates `height`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
83	<code>legend=dict(orientation="h"),</code>	Creates or updates `legend`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
84	<code>paper_bgcolor="rgba(0,0,0,0)",</code>	Creates or updates `paper_bgcolor`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
85	<code>plot_bgcolor="rgba(255,255,255,.02)",</code>	Creates or updates `plot_bgcolor`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
86	<code>)</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
87	<code>return fig</code>	Returns the result to the caller; this is the function output boundary that tests should

Lines 89-110

```

0089
0090 def create_app(run_dir: str | Path, config_path: str | Path = "configs/fast.yaml") -> gr.Blocks:
0091     run_path = Path(run_dir)
0092     config, frame, splits, standardizer, model, dataset = _load_bundle(run_path, Path(config_path))
0093     metrics = json.loads((run_path / "metrics.json").read_text())
0094     ablations = pd.DataFrame(json.loads((run_path / "ablations.json").read_text()))
0095     sessions = sorted(frame.session_id.unique().tolist())
0096
0097     def update_session(session_id: str):
0098         session = frame[frame.session_id == session_id]
0099         cards = (
0100             f"### Session snapshot\n"
0101             f"- **Driver:** {session.driver_id.iloc[0]}\n"
0102             f"- **Track:** {session.track_id.iloc[0]}\n"
0103             f"- **Frames:** {len(session):}\n"
0104             f"- **Peak speed:** {session.speed_mps.max() * 3.6:.1f} km/h\n"
0105             f"- **Rain peak:** {session.rainfall.max():.2f}"
0106         )
0107         return _track_figure(frame, session_id), _telemetry_figure(frame, session_id), cards
0108
0109     def simulate(index: int, throttle: float, brake: float, grip: float, rain: float, tyre: float):
0110         index = int(np.clip(index, 0, len(dataset) - 1))

```

Line	Code signal	What it is doing / what to verify
90	def create_app(run_dir: str Path, config_path: str Path = "configs/fast.yaml")	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
91	run_path = Path(run_dir)	Creates or updates `run_path`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
92	config, frame, splits, standardizer, model, dataset = _load_bundle(run_path, Pat	Creates or updates `config, frame, splits, standardizer, model, dataset`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
93	metrics = json.loads((run_path / "metrics.json").read_text())	Creates or updates `metrics`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
94	ablations = pd.DataFrame(json.loads((run_path / "ablations.json").read_text()))	Creates or updates `ablations`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
95	sessions = sorted(frame.session_id.unique().tolist())	Creates or updates `sessions`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
97	def update_session(session_id: str):	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
98	session = frame[frame.session_id == session_id]	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
99	cards = (	Creates or updates `cards`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
100	f"### Session snapshot\n"	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
101	f"- **Driver:** {session.driver_id.iloc[0]}\n"	Executes this statement inside the current

		control-flow block; connect it to the nearest function contract and testable invariant.
102	f'- **Track:** {session.track_id.iloc[0]}\n"	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
103	f'- **Frames:** {len(session);,}\n"	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
104	f'- **Peak speed:** {session.speed_mps.max() * 3.6:.1f} km/h\n"	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
105	f'- **Rain peak:** {session.rainfall.max():.2f}"	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
106	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
107	return _track_figure(frame, session_id), _telemetry_figure(frame, session_id), c	Returns the result to the caller; this is the function output boundary that tests should assert.
109	def simulate(index: int, throttle: float, brake: float, grip: float, rain: float	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
110	index = int(np.clip(index, 0, len(dataset) - 1))	Bounds a value to a supported range. Monitor how often clipping occurs because it may hide distribution shift.

Lines 111-132

```

0111     item = dataset[index]
0112     history_z = item["history"].numpy()
0113     future_z = item["future_inputs"].numpy()
0114     history_raw = history_z * standardizer.input_std + standardizer.input_mean
0115     future_raw = future_z * standardizer.input_std + standardizer.input_mean
0116     scenario = Scenario(throttle, brake, grip, rain, tyre)
0117     predicted = rollout(model, history_raw, future_raw, standardizer, scenario)
0118     actual = standardizer.inverse_targets(item["future_targets"].numpy())
0119     seconds = np.arange(len(predicted)) / config.data.sample_hz
0120     fig = go.Figure()
0121     fig.add_trace(go.Scatter(x=seconds, y=actual[:, 0] * 3.6, name="Recorded future", line=dict(width=3)))
0122     fig.add_trace(go.Scatter(x=seconds, y=predicted[:, 0] * 3.6, name="Imagined future", line=dict(width=3,
dash="dash"))))
0123     fig.update_layout(
0124         template="plotly_dark",
0125         title="World-model rollout: recorded vs imagined speed",
0126         xaxis_title="Seconds into future",
0127         yaxis_title="Speed (km/h)",
0128         height=470,
0129         paper_bgcolor="rgba(0,0,0,0)",
0130         plot_bgcolor="rgba(255,255,255,.02)",
0131     )
0132     delta = float((predicted[-1, 0] - actual[-1, 0]) * 3.6)

```

Line	Code signal	What it is doing / what to verify
111	item = dataset[index]	Creates or updates `item`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
112	history_z = item["history"].numpy()	Creates or updates `history_z`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
113	future_z = item["future_inputs"].numpy()	Creates or updates `future_z`. Check its unit, shape, lifetime and whether it is derived only from information available at this

		point.
114	history_raw = history_z * standardizer.input_std + standardizer.input_mean	Creates or updates `history_raw`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
115	future_raw = future_z * standardizer.input_std + standardizer.input_mean	Creates or updates `future_raw`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
116	scenario = Scenario(throttle, brake, grip, rain, tyre)	Creates or updates `scenario`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
117	predicted = rollout(model, history_raw, future_raw, standardizer, scenario)	Creates or updates `predicted`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
118	actual = standardizer.inverse_targets(item["future_ta rgets"].numpy())	Creates or updates `actual`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
119	seconds = np.arange(len(predicted)) / config.data.sample_hz	Creates or updates `seconds`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
120	fig = go.Figure()	Creates or updates `fig`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
121	fig.add_trace(go.Scatter(x=seconds, y=actual[:, 0] * 3.6, name="Recorded future"	Creates or updates `fig.add_trace(go.Scatter(x`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
122	fig.add_trace(go.Scatter(x=seconds, y=predicted[:, 0] * 3.6, name="Imagined futu	Creates or updates `fig.add_trace(go.Scatter(x`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
123	fig.update_layout(	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
124	template="plotly_dark",	Creates or updates `template`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
125	title="World-model rollout: recorded vs imagined speed",	Creates or updates `title`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
126	xaxis_title="Seconds into future",	Creates or updates `xaxis_title`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
127	yaxis_title="Speed (km/h)",	Creates or updates `yaxis_title`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
128	height=470,	Creates or updates `height`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
129	paper_bgcolor="rgba(0,0,0,0)",	Creates or updates `paper_bgcolor`. Check its

		unit, shape, lifetime and whether it is derived only from information available at this point.
130	plot_bgcolor="rgba(255,255,255,0.2)",	Creates or updates `plot_bgcolor`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
131	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
132	delta = float((predicted[-1, 0] - actual[-1, 0]) * 3.6)	Creates or updates `delta`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 133-154

```

0133     summary = (
0134         f"### Scenario result\n"
0135         f"- Rollout horizon: **{len(predicted) / config.data.sample_hz:.1f} s**\n"
0136         f"- Final imagined speed: **{predicted[-1, 0] * 3.6:.1f} km/h**\n"
0137         f"- Difference from recorded future: **{delta:+.1f} km/h**\n"
0138         f"- Model: **{config.model.kind.upper()} world model**"
0139     )
0140     return fig, summary
0141
0142     with gr.Blocks(title="Project APEX") as app:
0143         gr.Markdown(
0144             """
0145 <div id="hero">
0146 <h1>PROJECT APEX</h1>
0147 <h3>F1 Telemetry World-Simulation Engine</h3>
0148 <p>Explore real-shaped telemetry, inspect the learned world model, alter future conditions, and watch the engine
0149 imagine what happens next.</p>
0149 </div>
0150             """
0151         )
0152         with gr.Tabs():
0153             with gr.Tab("Race Engineering Desk"):
0154                 with gr.Row():

```

Line	Code signal	What it is doing / what to verify
133	summary = (	Creates or updates `summary`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
134	f"### Scenario result\n"	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
135	f"- Rollout horizon: **{len(predicted) / config.data.sample_hz:.1f} s**\n"	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
136	f"- Final imagined speed: **{predicted[-1, 0] * 3.6:.1f} km/h**\n"	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
137	f"- Difference from recorded future: **{delta:+.1f} km/h**\n"	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
138	f"- Model: **{config.model.kind.upper()} world model**"	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
139	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
140	return fig, summary	Returns the result to the caller; this is the function output boundary that tests should

		assert.
142	with gr.Blocks(title="Project APEX") as app:	Opens a managed resource or context and guarantees cleanup when the block exits.
143	gr.Markdown()	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
144	"""	Begins or ends documentation describing the module, class or function contract.
145	<div id="hero">	Creates or updates `<div id`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
146	PROJECT APEX	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
147	F1 Telemetry World-Simulation Engine	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
148	<p>Explore real-shaped telemetry, inspect the learned world model, alter future	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
149	</div>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
150	"""	Begins or ends documentation describing the module, class or function contract.
151	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
152	with gr.Tabs():	Opens a managed resource or context and guarantees cleanup when the block exits.
153	with gr.Tab("Race Engineering Desk"):	Opens a managed resource or context and guarantees cleanup when the block exits.
154	with gr.Row():	Opens a managed resource or context and guarantees cleanup when the block exits.

Lines 155-176

```

0155         session_select = gr.Dropdown(sessions, value=sessions[0], label="Session")
0156         session_summary = gr.Markdown()
0157     with gr.Row():
0158         track_plot = gr.Plot(_track_figure(frame, sessions[0]))
0159         telemetry_plot = gr.Plot(_telemetry_figure(frame, sessions[0]))
0160         session_select.change(update_session, session_select, [track_plot, telemetry_plot,
session_summary])
0161         initial = frame[frame.session_id == sessions[0]]
0162         session_summary.value = f"Frames: {len(initial):,} | Peak speed:
{initial.speed_mps.max()*3.6:.1f} km/h"
0163
0164     with gr.Tab("World Rollout Simulator"):
0165         gr.Markdown("Change the future controls and environment. The model rolls forward from a real
test-window history.")
0166         with gr.Row():
0167             with gr.Column(scale=1):
0168                 sample_index = gr.Slider(0, max(0, len(dataset) - 1), value=0, step=1, label="Test
window")
0169                 throttle = gr.Slider(0.7, 1.3, value=1.0, step=0.02, label="Throttle multiplier")
0170                 brake = gr.Slider(0.7, 1.3, value=1.0, step=0.02, label="Brake multiplier")
0171                 grip = gr.Slider(0.6, 1.2, value=1.0, step=0.02, label="Grip multiplier")
0172                 rain = gr.Slider(-0.2, 0.5, value=0.0, step=0.02, label="Rain change")
0173                 tyre = gr.Slider(0.5, 2.0, value=1.0, step=0.05, label="Tyre degradation multiplier")
0174                 run_button = gr.Button("IMAGINE FUTURE", variant="primary")
0175             with gr.Column(scale=2):
0176                 rollout_plot = gr.Plot()

```


Line	Code signal	What it is doing / what to verify
155	<code>session_select = gr Dropdown(sessions, value=sessions[0], label="Session")</code>	Creates or updates `session_select`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
156	<code>session_summary = gr.Markdown()</code>	Creates or updates `session_summary`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
157	<code>with gr.Row():</code>	Opens a managed resource or context and guarantees cleanup when the block exits.
158	<code>track_plot = gr.Plot(_track_figure(frame, sessions[0]))</code>	Creates or updates `track_plot`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
159	<code>telemetry_plot = gr.Plot(_telemetry_figure(frame, sessions[0]))</code>	Creates or updates `telemetry_plot`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
160	<code>session_select.change(update_session, session_select, [track_plot, telemetry_plo</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
161	<code>initial = frame[frame.session_id == sessions[0]]</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
162	<code>session_summary.value = f"***Frames:** {len(initial);} **Peak speed:** {initia</code>	Creates or updates `session_summary.value`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
164	<code>with gr.Tab("World Rollout Simulator"):</code>	Opens a managed resource or context and guarantees cleanup when the block exits.
165	<code>gr.Markdown("Change the future controls and environment. The model rolls forward</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
166	<code>with gr.Row():</code>	Opens a managed resource or context and guarantees cleanup when the block exits.
167	<code>with gr.Column(scale=1):</code>	Opens a managed resource or context and guarantees cleanup when the block exits.
168	<code>sample_index = gr.Slider(0, max(0, len(dataset) - 1), value=0, step=1, label="Te</code>	Creates or updates `sample_index`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
169	<code>throttle = gr.Slider(0.7, 1.3, value=1.0, step=0.02, label="Throttle multiplier"</code>	Creates or updates `throttle`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
170	<code>brake = gr.Slider(0.7, 1.3, value=1.0, step=0.02, label="Brake multiplier")</code>	Creates or updates `brake`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
171	<code>grip = gr.Slider(0.6, 1.2, value=1.0, step=0.02, label="Grip multiplier")</code>	Creates or updates `grip`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
172	<code>rain = gr.Slider(-0.2, 0.5, value=0.0, step=0.02, label="Rain change")</code>	Creates or updates `rain`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
173	<code>tyre = gr.Slider(0.5, 2.0, value=1.0, step=0.05, label="Tyre degradation multipl</code>	Creates or updates `tyre`. Check its unit, shape, lifetime and whether it is derived only from information available at this

		point.
174	<code>run_button = gr.Button("IMAGINE FUTURE", variant="primary")</code>	Creates or updates `run_button`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
175	<code>with gr.Column(scale=2):</code>	Opens a managed resource or context and guarantees cleanup when the block exits.
176	<code>rollout_plot = gr.Plot()</code>	Creates or updates `rollout_plot`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.

Lines 177-198

```

0177         rollout_summary = gr.Markdown()
0178         run_button.click(simulate, [sample_index, throttle, brake, grip, rain, tyre], [rollout_plot,
rollout_summary])
0179
0180         with gr.Tab("Model & Evaluation"):
0181             gr.Markdown(
0182                 f"""
0183     ### Test performance
0184     - **Overall MAE:** {metrics['overall_mae']:.4f}
0185     - **Speed MAE:** {metrics['speed_mae_mps'] * 3.6:.2f} km/h
0186     - **Speed RMSE:** {metrics['speed_rmse_mps'] * 3.6:.2f} km/h
0187     - **Test windows:** {metrics['samples']:,}
0188
0189     The horizon curve matters more than a single one-step number: simulation error compounds as the model consumes
its own predictions.
0190             """
0191             )
0192             horizon = pd.DataFrame({"step": np.arange(1, len(metrics["horizon_rmse"]) + 1), "rmse":
metrics["horizon_rmse"]})
0193             fig = go.Figure(go.Scatter(x=horizon.step, y=horizon.rmse, mode="lines+markers"))
0194             fig.update_layout(template="plotly_dark", title="Rollout RMSE by horizon",
xaxis_title="Prediction step", yaxis_title="Normalized RMSE")
0195             gr.Plot(fig)
0196
0197         with gr.Tab("Ablation Lab"):
0198             gr.Markdown("Ablations answer one causal engineering question at a time by removing information
while holding the evaluation protocol fixed.")

```

Line	Code signal	What it is doing / what to verify
177	<code>rollout_summary = gr.Markdown()</code>	Creates or updates `rollout_summary`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
178	<code>run_button.click(simulate, [sample_index, throttle, brake, grip, rain, tyre], [r</code>	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
180	<code>with gr.Tab("Model & Evaluation"):</code>	Opens a managed resource or context and guarantees cleanup when the block exits.
181	<code>gr.Markdown(</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
182	<code>f"""</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
183	<code>### Test performance</code>	Documents an assumption, intent or constraint that should remain aligned with the implementation.
184	<code>- **Overall MAE:** {metrics['overall_mae']:.4f}</code>	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
185	<code>- **Speed MAE:** {metrics['speed_mae_mps'] * 3.6:.2f} km/h</code>	Executes this statement inside the current control-flow block; connect it to the nearest

		function contract and testable invariant.
186	- **Speed RMSE:** {metrics['speed_rmse_mps'] * 3.6:.2f} km/h	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
187	- **Test windows:** {metrics['samples'];,}	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
189	The horizon curve matters more than a single one-step number: simulation error c	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
190	"""	Begins or ends documentation describing the module, class or function contract.
191	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
192	horizon = pd.DataFrame({"step": np.arange(1, len(metrics["horizon_rmse"]) + 1),	Creates or updates `horizon`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
193	fig = go.Figure(go.Scatter(x=horizon.step, y=horizon.rmse, mode="lines+markers"))	Creates or updates `fig`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
194	fig.update_layout(template="plotly_dark", title="Rollout RMSE by horizon", xaxis	Creates or updates `fig.update_layout(template)`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
195	gr.Plot(fig)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
197	with gr.Tab("Ablation Lab"):	Opens a managed resource or context and guarantees cleanup when the block exits.
198	gr.Markdown("Ablations answer one causal engineering question at a time by remov	Calls a function or method. Trace its input contract, returned value and any persistent side effects.

Lines 199-220

```

0199         gr.Dataframe(ablations[["ablation", "speed_mae_mps", "overall_mae"]], interactive=False)
0200         fig = go.Figure(go.Bar(x=ablentions.ablation, y=ablentions.speed_mae_mps * 3.6))
0201         fig.update_layout(template="plotly_dark", title="One-step speed MAE by feature ablation",
yaxis_title="MAE (km/h)")
0202         gr.Plot(fig)
0203
0204         with gr.Tab("System Architecture"):
0205             gr.Markdown(
0206                 """
0207                 ```text
0208                 FastF1 / OpenF1 / Synthetic / Future F1 25 UDP
0209                     ↓
0210                 Source-specific adapters
0211                     ↓
0212                 Canonical telemetry contract
0213                     ↓
0214                 Validation → alignment → session splits
0215                     ↓
0216                 Sequence windows + train-only scaling
0217                     ↓
0218                 Persistence / Linear / GRU / RSSM / SSM models
0219                     ↓
0220                 Horizon evaluation + physical checks

```

Line	Code signal	What it is doing / what to verify
199	gr.Dataframe(ablations[["ablation", "speed_mae_mps", "overall_mae"]],	Creates or updates `gr.Dataframe(ablations[["ablation",

	interactiv	"speed_mae_mps", "overall_mae"]], interactive'. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
200	fig = go.Figure(go.Bar(x=ablations.ablation, y=ablations.speed_mae_mps * 3.6))	Creates or updates `fig`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
201	fig.update_layout(template="plotly_dark", title="One-step speed MAE by feature a	Creates or updates `fig.update_layout(template`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
202	gr.Plot(fig)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
204	with gr.Tab("System Architecture"):	Opens a managed resource or context and guarantees cleanup when the block exits.
205	gr.Markdown(	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
206	"""	Begins or ends documentation describing the module, class or function contract.
207	```text	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
208	FastF1 / OpenF1 / Synthetic / Future F1 25 UDP	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
209	↓	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
210	Source-specific adapters	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
211	↓	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
212	Canonical telemetry contract	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
213	↓	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
214	Validation → alignment → session splits	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
215	↓	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
216	Sequence windows + train-only scaling	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
217	↓	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
218	Persistence / Linear / GRU / RSSM / SSM models	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
219	↓	Executes this statement inside the current control-flow block; connect it to the nearest

		function contract and testable invariant.
220	Horizon evaluation + physical checks	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.

Lines 221-236

```

0221         ↓
0222     Scenario interventions + imagined rollouts
0223         ↓
0224     Registry / artifacts / API / interactive UI
0225     ```
0226         """
0227     )
0228     return app
0229
0230
0231 def launch(run_dir: str | Path, config_path: str | Path = "configs/fast.yaml", share: bool = False) -> None:
0232     create_app(run_dir, config_path).launch(share=share, css=CUSTOM_CSS)
0233
0234
0235 if __name__ == "__main__":
0236     launch("artifacts/runs/reference_gru")

```

Line	Code signal	What it is doing / what to verify
221	↓	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
222	Scenario interventions + imagined rollouts	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
223	↓	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
224	Registry / artifacts / API / interactive UI	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
225	```	Executes this statement inside the current control-flow block; connect it to the nearest function contract and testable invariant.
226	"""	Begins or ends documentation describing the module, class or function contract.
227	)	Calls a function or method. Trace its input contract, returned value and any persistent side effects.
228	return app	Returns the result to the caller; this is the function output boundary that tests should assert.
231	def launch(run_dir: str Path, config_path: str Path = "configs/fast.yaml", s	Defines a callable boundary. Its parameters form the input contract and its return value or side effects form the output contract.
232	create_app(run_dir, config_path).launch(share=share, css=CUSTOM_CSS)	Creates or updates `create_app(run_dir, config_path).launch(share`. Check its unit, shape, lifetime and whether it is derived only from information available at this point.
235	if __name__ == "__main__":	Branches only when this condition is true; verify both the true and false paths in tests.
236	launch("artifacts/runs/reference_gru")	Calls a function or method. Trace its input contract, returned value and any persistent side effects.

Educational Labs and Mini-Projects - Source Map

labs/01_units_and_shapes/solution.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```
0001 import numpy as np
0002
0003 speed_kmh = np.array([0.0, 72.0, 144.0])
0004 speed_mps = speed_kmh / 3.6
0005 telemetry = np.stack([speed_mps, np.array([0.0, 0.5, 1.0])], axis=1)
0006 print("speed_mps:", speed_mps)
0007 print("telemetry shape [time, features]:", telemetry.shape)
0008 assert telemetry.shape == (3, 2)
```

labs/02_vectorized_kinematics/solution.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```
0001 import numpy as np
0002
0003 dt = 0.2
0004 time = np.arange(0, 5, dt)
0005 speed = 20 + 4*np.sin(time)
0006 accel = np.gradient(speed, dt)
0007 reconstructed = speed[0] + np.cumsum(accel)*dt
0008 print("first accelerations:", accel[:5].round(3))
0009 print("max reconstruction error:", float(np.max(np.abs(reconstructed-speed))))
```

labs/03_force_model/solution.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```
0001 from education.core import CarState, step_car
0002
0003 state = CarState(0.0, 0.0, 40.0)
0004 for i in range(5):
0005     state = step_car(state, throttle=0.7, brake=0.0, dt=0.1)
0006     print(i, state)
```

labs/04_sampling_aliasing/solution.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```
0001 import numpy as np
0002
0003 for hz in [5, 12, 40]:
0004     t=np.arange(0,2,1/hz)
0005     signal=np.sin(2*np.pi*8*t)
0006     print(hz, "Hz samples:", np.round(signal[:8],3))
```

labs/05_regression_baseline/solution.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```
0001 import numpy as np
0002 from sklearn.linear_model import Ridge
0003
0004 rng=np.random.default_rng(4)
0005 speed=rng.uniform(20,80,1000)
0006 throttle=rng.uniform(0,1,1000)
0007 brake=rng.uniform(0,1,1000)
```

```

0008 y=speed + 0.7*throttle - 1.1*brake - 0.0008*speed**2 + rng.normal(0,0.05,1000)
0009 X=np.column_stack([speed,throttle,brake])
0010 model=Ridge(alpha=1.0).fit(X[:800],y[:800])
0011 print("coefficients:", model.coef_)
0012 print("test MAE:", np.mean(np.abs(model.predict(X[800:])-y[800:])))

```

labs/06_autograd_drag/solution.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```

0001 import torch
0002
0003 speed=torch.linspace(5,70,200)
0004 true_drag=0.002
0005 target=-true_drag*speed**2
0006 drag=torch.tensor(0.01,requires_grad=True)
0007 opt=torch.optim.SGD([drag],lr=1e-7)
0008 for step in range(200):
0009     opt.zero_grad()
0010     pred=-drag*speed**2
0011     loss=((pred-target)**2).mean()
0012     loss.backward(); opt.step()
0013 print("learned drag:", drag.item(), "loss:", loss.item())

```

labs/07_pytorch_module/solution.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```

0001 import torch
0002 from torch import nn
0003
0004 class Transition(nn.Module):
0005     def __init__(self):
0006         super().__init__()
0007         self.net=nn.Sequential(nn.Linear(5,16),nn.ReLU(),nn.Linear(16,2))
0008     def forward(self,x): return self.net(x)
0009
0010 model=Transition(); x=torch.randn(4,5); y=model(x)
0011 print(model); print("input",x.shape,"output",y.shape)

```

labs/08_dataset_windows/solution.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```

0001 import numpy as np
0002 from education.core import sliding_windows
0003
0004 values=np.arange(30,dtype=float).reshape(10,3)
0005 x,y=sliding_windows(values,history=4,horizon=2)
0006 print("x",x.shape,"y",y.shape)
0007 print("first history\n",x[0]); print("first future\n",y[0])

```

labs/09_loss_comparison/solution.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```

0001 import torch
0002 from torch.nn import functional as F
0003
0004 pred=torch.tensor([0.,1.,2.,20.]); target=torch.tensor([0.,1.,2.,3.])
0005 print("MSE",F.mse_loss(pred,target).item())
0006 print("MAE",F.l1_loss(pred,target).item())
0007 print("Huber",F.huber_loss(pred,target).item())

```

labs/10_training_loop/solution.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in ``source_code_companion/line_by_line/``.

```
0001 import torch
0002 from torch import nn
0003
0004 torch.manual_seed(1); x=torch.randn(128,3); y=x.sum(1,keepdim=True)
0005 model=nn.Linear(3,1); opt=torch.optim.Adam(model.parameters(),lr=0.05)
0006 for epoch in range(8):
0007     model.train(); opt.zero_grad(); pred=model(x); loss=((pred-y)**2).mean(); loss.backward()
0008     grad=float(model.weight.grad.norm()); opt.step()
0009     model.eval();
0010     with torch.no_grad(): val=((model(x)-y)**2).mean()
0011     print(epoch,float(loss),grad,float(val))
```

labs/11_rnn_from_scratch/solution.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in ``source_code_companion/line_by_line/``.

```
0001 import torch
0002
0003 torch.manual_seed(0); W=torch.randn(3,2); U=torch.randn(3,3); b=torch.zeros(3); h=torch.zeros(3)
0004 sequence=torch.tensor([[1.,0.],[0.,1.],[1.,1.]])
0005 for t,x in enumerate(sequence):
0006     h=torch.tanh(W@x+U@h+b); print(t,h.numpy().round(3))
```

labs/12_gru_gates/solution.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in ``source_code_companion/line_by_line/``.

```
0001 import torch
0002
0003 torch.manual_seed(2); cell=torch.nn.GRUCell(2,4); h=torch.zeros(1,4)
0004 for x in [torch.tensor([[1.,0.]],torch.tensor([[0.,1.]],torch.tensor([[0.,1.]])]:
0005     h=cell(x,h); print(h.detach().numpy().round(3))
```

labs/13_linear_ssm/solution.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in ``source_code_companion/line_by_line/``.

```
0001 import numpy as np
0002 A=np.array([[0.95,0.0],[0.1,0.85]]); B=np.array([[0.2],[0.05]]); x=np.zeros(2)
0003 for u in [1,1,0,0,0]:
0004     x=A@x+B[:,0]*u; print(x.round(3))
```

labs/14_selective_ssm/solution.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in ``source_code_companion/line_by_line/``.

```
0001 import torch
0002 from torch import nn
0003
0004 class SelectiveCell(nn.Module):
0005     def __init__(self,d):
0006         super().__init__(); self.decay=nn.Linear(d,d); self.write=nn.Linear(d,d)
0007     def forward(self,x,h):
0008         a=torch.sigmoid(self.decay(x))*0.99
0009         b=torch.tanh(self.write(x))
0010         return a*h+(1-a)*b
0011 cell=SelectiveCell(3); h=torch.zeros(1,3)
0012 for x in torch.eye(3): h=cell(x[None],h); print(h.detach().numpy().round(3))
```


labs/15_autoencoder/solution.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```
0001 import torch
0002 from torch import nn
0003
0004 torch.manual_seed(3); x=torch.randn(256,12)
0005 enc=nn.Linear(12,3); dec=nn.Linear(3,12); opt=torch.optim.Adam(list(enc.parameters())
+list(dec.parameters()),lr=0.03)
0006 for _ in range(80):
0007     opt.zero_grad(); z=torch.tanh(enc(x)); recon=dec(z); loss=((recon-x)**2).mean(); loss.backward(); opt.step()
0008 print("latent",z.shape,"loss",float(loss))
```

labs/16_vae_reparameterization/solution.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```
0001 import torch
0002
0003 mu=torch.tensor([[0.2,-0.4]]); logvar=torch.tensor([[ -1.0,0.5]])
0004 std=torch.exp(0.5*logvar); eps=torch.randn_like(std); z=mu+std*eps
0005 kl=-0.5*torch.sum(1+logvar-mu.pow(2)-logvar.exp(),dim=1)
0006 print("mu",mu,"std",std,"sample",z,"KL",kl)
```

labs/17_rssm_step/solution.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```
0001 import torch
0002 from torch import nn
0003
0004 cell=nn.GRUCell(5,8); prior=nn.Linear(8,4); posterior=nn.Linear(8+3,4)
0005 h=torch.zeros(2,8); prev_z=torch.zeros(2,2); action=torch.randn(2,3); obs_embed=torch.randn(2,3)
0006 h=cell(torch.cat([prev_z,action],-1),h)
0007 prior_stats=prior(h); post_stats=posterior(torch.cat([h,obs_embed],-1))
0008 print("h",h.shape,"prior",prior_stats.shape,"posterior",post_stats.shape)
```

labs/18_cem_planning/solution.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```
0001 import numpy as np
0002
0003 rng=np.random.default_rng(0); horizon=6; mean=np.full(horizon,0.5); std=np.full(horizon,0.3)
0004 for it in range(5):
0005     actions=np.clip(rng.normal(mean,std,(500,horizon)),0,1)
0006     speed=40+np.cumsum(2*actions-0.4,axis=1)
0007     score=-(speed[:,-1]-48)**2-0.1*np.sum(np.diff(actions,axis=1)**2,axis=1)
0008     elite=actions[np.argsort(score)[-50:]]
0009     mean,std=elite.mean(0),elite.std(0)+1e-3
0010 print(it,mean.round(2),score.max().round(3))
```

labs/19_time_alignment/solution.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```
0001 import pandas as pd
0002
0003 car=pd.DataFrame({'t':[0.0,0.2,0.4,0.6],'speed':[10,12,14,16]})
0004 weather=pd.DataFrame({'t':[0.05,0.55],'rain':[0.0,0.3]})
0005 aligned=pd.merge_asof(car.sort_values('t'),weather.sort_values('t'),on='t',direction='backward',tolerance=0.3)
0006 print(aligned)
```

labs/20_pipeline_contract/solution.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```
0001 from pathlib import Path
0002 import json
0003
0004 run=Path('artifacts/education_pipeline'); run.mkdir(parents=True,exist_ok=True)
0005 stages=['ingest','validate','window','train','evaluate','publish']
0006 for i,name in enumerate(stages):
0007     path=run/f'{i:02d}_{name}.json'; path.write_text(json.dumps({'stage':name,'status':'succeeded'}))
0008     print(path)
```

projects/01_car_integrator/main.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```
0001 from education.core import CarState, step_car
0002
0003 def run():
0004     state=CarState(0,0,0)
0005     trace=[]
0006     for i in range(100):
0007         state=step_car(state,0.8 if i<60 else 0.0,0.0 if i<60 else 0.5,0.1)
0008         trace.append(state)
0009     assert trace[-1].speed_mps >= 0
0010     print(trace[-1])
0011 if __name__=='__main__': run()
```

projects/02_telemetry_quality/main.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```
0001 import pandas as pd
0002
0003 REQUIRED={'time_s','speed_mps','throttle','brake'}
0004 def validate(df):
0005     errors=[]
0006     if not REQUIRED.issubset(df): errors.append('missing columns')
0007     if not df.time_s.is_monotonic_increasing: errors.append('time not monotonic')
0008     if (df.speed_mps<0).any(): errors.append('negative speed')
0009     if not df.throttle.between(0,1).all(): errors.append('throttle range')
0010     return errors
0011 def run():
0012     df=pd.DataFrame({'time_s':[0,0.1,0.2],'speed_mps':[0,3,4],'throttle':[0,1,0.5],'brake':[0,0,0]})
0013     assert validate(df)==[]; print('quality passed')
0014 if __name__=='__main__': run()
```

projects/03_lap_physics/main.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```
0001 import numpy as np
0002
0003 def run(n=600):
0004     progress=np.linspace(0,1,n,endpoint=False); curvature=0.02+0.12*(np.sin(6*np.pi*progress)**2)
0005     target=np.clip(90/(1+10*curvature),20,90); speed=np.empty(n); speed[0]=30
0006     for i in range(1,n): speed[i]=speed[i-1]+0.08*np.clip(target[i]-speed[i-1],-8,3)
0007     print('speed range',float(speed.min()),float(speed.max()))
0008     return progress,curvature,speed
0009 if __name__=='__main__': run()
```

projects/04_mlp_transition/main.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```

0001 import torch
0002 from torch import nn
0003
0004 class Transition(nn.Module):
0005     def __init__(self):
0006         super().__init__()
0007         self.net=nn.Sequential(nn.Linear(5,16),nn.ReLU(),nn.Linear(16,2))
0008     def forward(self,x): return self.net(x)
0009
0010 model=Transition(); x=torch.randn(4,5); y=model(x)
0011 print(model); print("input",x.shape,"output",y.shape)
0012
0013 print("Extend this module with the dataset from Project 3.")

```

projects/05_track_encoder/main.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```

0001 import torch
0002 from torch import nn
0003
0004 class TrackEncoder(nn.Module):
0005     def __init__(self):
0006         super().__init__(); self.net=nn.Sequential(nn.Conv1d(3,8,3,padding=1),nn.ReLU(),nn.AdaptiveAvgPool1d(1))
0007     def forward(self,x): return self.net(x).squeeze(-1)
0008 if __name__=='__main__':
0009     x=torch.randn(4,3,64); print(TrackEncoder()(x).shape)

```

projects/06_gru_forecaster/main.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```

0001 import torch
0002 from torch import nn
0003
0004 class GRUForecaster(nn.Module):
0005     def __init__(self,f=6,h=32,o=2):
0006         super().__init__(); self.gru=nn.GRU(f,h,batch_first=True); self.head=nn.Linear(h,o)
0007     def forward(self,x): return self.head(self.gru(x)[0][:,-1])
0008 if __name__=='__main__': print(GRUForecaster()(torch.randn(8,20,6)).shape)

```

projects/07_selective_ssm/main.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```

0001 import torch
0002 from torch import nn
0003
0004 class SelectiveCell(nn.Module):
0005     def __init__(self,d):
0006         super().__init__(); self.decay=nn.Linear(d,d); self.write=nn.Linear(d,d)
0007     def forward(self,x,h):
0008         a=torch.sigmoid(self.decay(x))*0.99
0009         b=torch.tanh(self.write(x))
0010         return a*h+(1-a)*b
0011 cell=SelectiveCell(3); h=torch.zeros(1,3)
0012 for x in torch.eye(3): h=cell(x[None],h); print(h.detach().numpy().round(3))

```

projects/08_rssm_imagination/main.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```

0001 import torch
0002 from torch import nn
0003
0004 cell=nn.GRUCell(5,8); prior=nn.Linear(8,4); posterior=nn.Linear(8+3,4)
0005 h=torch.zeros(2,8); prev_z=torch.zeros(2,2); action=torch.randn(2,3); obs_embed=torch.randn(2,3)
0006 h=cell(torch.cat([prev_z,action],-1),h)
0007 prior_stats=prior(h); post_stats=posterior(torch.cat([h,obs_embed],-1))
0008 print("h",h.shape,"prior",prior_stats.shape,"posterior",post_stats.shape)

```

projects/09_latent_mpc/main.py

This small executable isolates one concept. Predict its output, run it, then use the generated line-by-line Markdown file in `source\_code\_companion/line\_by\_line/`.

```

0001 import numpy as np
0002
0003 rng=np.random.default_rng(0); horizon=6; mean=np.full(horizon,0.5); std=np.full(horizon,0.3)
0004 for it in range(5):
0005     actions=np.clip(rng.normal(mean,std,(500,horizon)),0,1)
0006     speed=40+np.cumsum(2*actions-0.4,axis=1)
0007     score=-(speed[:,-1]-48)**2-0.1*np.sum(np.diff(actions,axis=1)**2,axis=1)
0008     elite=actions[np.argsort(score)[-50:]]
0009     mean,std=elite.mean(0),elite.std(0)+1e-3
0010     print(it,mean.round(2),score.max().round(3))

```